Robot Reinforcement Learning

From First Principles to Real Robots

Bruce Lu

2026-08-31

Contents

How to Use This Book

This book is designed for active self-study. Read with a Python prompt, a notebook, or a robot simulator nearby. When an equation appears, first name every symbol and predict how changing it would affect an experiment. Then work the numerical example before looking at the answer. When code appears, run it, change one input, and explain the output before continuing.

The material has three connected layers:

1. **Foundations** explain the reinforcement-learning problem, the mathematics, and the major algorithm families.
2. **The Microduck laboratory** shows where those abstractions live in a real GPU-parallel robot-learning project, from environment configuration through ONNX deployment.
3. **Modern robot intelligence** connects locomotion to demonstrations, offline data, world models, vision-language-action policies, hierarchical planning, and reproducible research.

Microduck is the running hands-on case study, not the boundary of the subject. Its main walking policy is a command-conditioned local controller: it does not see a camera or obstacle map and it does not choose a destination. That clear boundary makes it a useful base for learning how perception and planning can be added without placing cloud latency inside a real-time balance loop.

On GitHub, chapter-end answers are collapsed so that you can attempt each exercise first. In this PDF edition they are expanded and placed at the end of their chapter. The primary-source index at the end records the papers and official open-source projects behind the research chapters. Treat every state-of-the-art claim as scoped to its dated task, data, embodiment, and evaluation protocol.

Suggested routes

- **Theory first:** Chapters 1–7, the four small programs in `examples/`, and the worked problems in Chapter 20.
- **Build and deploy:** Chapters 1–13, alongside a pinned Microduck checkout.
- **Research literacy:** Chapters 14–20 after the foundations, recording the evidence boundary of every paper before accepting its headline result.

Do not measure progress by pages read. Advance when you can produce the deliverable at the end of a chapter and explain what observation, action, reward, data, and safety contract a robot-learning result actually used.

1. Reinforcement Learning Foundations

1.1 The central idea

Reinforcement learning trains a decision-making rule by letting it interact with an environment. The learner is not given the correct motor command for every possible situation. Instead, it tries actions, observes consequences, and adjusts its policy so that useful outcomes become more likely.

For Microduck, one interaction looks like this:

```
simulated robot and command
        |
        | observation: orientation, joint state, previous action, command
        v
neural-network policy
        |
        | action: 14 normalized joint-position targets
        v
Better Actuator Models (BAM) motor model + MuJoCo physics
        |
        | next robot state, reward, termination flag
        +-----------------------------------------> repeat
```

An **actuator** is the component that turns a command into physical force or motion; here it is a Dynamixel servo represented by BAM. During training, thousands of simulated robots execute this loop in parallel. During deployment, one real robot executes only the learned policy; PPO and the critic are no longer updating weights.

1.2 RL compared with nearby ideas

Supervised learning starts with labeled examples such as “for this image, the answer is cat.” RL usually has no label saying “the correct 14 motor targets are these.” It has an objective, such as tracking velocity while staying upright, and must discover a sequence of actions.

Classical control starts from an explicit model or error law, such as a PID controller. RL can learn a nonlinear controller where writing the full law by hand is difficult. Classical controllers remain valuable as baselines, safety layers, and inner loops.

Planning chooses goals or sequences over a longer horizon. A walking policy is normally below planning: a planner asks for a local velocity, and the policy produces stable joint targets. Asking a cloud model for raw motor torques would collapse these layers and remove the timing and safety boundary.

1.3 A small guide to the notation

Math notation is compact language. Read the symbols rather than trying to memorize the visual shape:

Notation	Read it as	Example meaning here
x_t	“x at time step t”	robot state now
x_{t+1}	“x at the next step”	robot state 20 ms later
$a \in \mathcal{A}$	“a is an element of set A”	this action is allowed
$p(x \mid y)$	“probability of x given y”	next-state probability given state/action
$\sum_i x_i$	“sum x over index i”	add reward terms or future rewards
$\mathbb{E}[X]$	“expected or long-run average X”	average return across sampled rollouts
$x \sim p$	“sample x from distribution p”	sample a randomized mass
θ	“theta,” a parameter collection	all actor network weights
$\approx$	approximately equal	critic estimate versus unknown true value

A **vector** is an ordered list of numbers. A **tensor** is the general multidimensional form used by PyTorch: a scalar is 0D, a vector is 1D, a table or batch is 2D, and images/batched histories use more dimensions. In a batch of 4,096 actor observations, the observation tensor has shape `(4096, 61)`.

1.4 The Markov Decision Process

The standard mathematical model is a Markov Decision Process (MDP):

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, R, \gamma)$$

Symbol	General meaning	Microduck example
$s_t \in \mathcal{S}$	complete state at time t	all simulated positions, velocities, contacts, and randomized physics
$a_t \in \mathcal{A}$	action selected at time t	14 joint-position commands
$P(s_{t+1} \mid s_t, a_t)$	transition dynamics	MuJoCo plus the BAM actuator, contacts, delay, and noise
$R(s_t, a_t, s_{t+1})$	reward	velocity tracking, uprightness, foot behavior, and regularization
γ	discount factor	`0.99` in the main PPO configuration

The Markov property says the current state contains enough information to model the distribution of the next state. The policy does not receive the complete simulator state, however. It receives an observation o_t.

The main actor observation has 61 values:

```
base angular velocity          3
projected gravity              3
joint position                14
joint velocity                14
previous action               14
twist command                  3
head-pose command              4
body-pose command              6
                              --
total                         61
```

This makes the real problem partially observable. For example, the actor does not directly receive true base linear velocity. Previous actions and physical dynamics provide some short-term context without making the policy recurrent.

1.5 Policy, trajectory, and return

A policy is a conditional distribution over actions:

$$\pi_\theta(a_t \mid o_t)$$

θ represents all neural-network weights. The vertical bar means "conditioned on," so the formula reads: "the probability of action a_t when the observation is o_t." During training, the actor defines a Gaussian distribution so it can explore nearby actions. A rollout is a trajectory:

$$\tau = (o_0, a_0, r_0, o_1, a_1, r_1, \ldots)$$

The discounted return from time t is:

$$G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \cdots$$

The capital sigma means "add all the following terms." With $\gamma = 0.99$, a reward one step later counts almost as much as a reward now, while very distant rewards gradually matter less. Discounting helps make long sums finite and expresses a preference for reliable, sooner outcomes.

The learning objective is to find weights with high expected return:

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}\left[\sum_{t=0}^{T-1} \gamma^t r_t\right]$$

Read this as: "adjust network weights θ to maximize the average discounted reward over trajectories sampled from the policy." "Expected" is important. Initial poses, commands, contact events, actuator parameters, and sampled actions vary. A policy should work across that distribution, not memorize one rollout.

1.6 Reward is a specification, not a feeling

The robot has no concept of "walk naturally" unless the measurable reward and termination conditions make natural walking the best available strategy. If a policy can earn more reward by shuffling, leaning, falling into a stable pose, or repeating a violent motion, PPO may discover exactly that.

A typical per-step reward is a weighted sum:

$$r_t = \sum_i w_i f_i(s_t, a_t, s_{t+1})$$

For example, a velocity-tracking term may be positive while action-rate and foot-slip terms are costs. The sign convention is a software contract, not a mathematical universal: some functions return a nonnegative cost and therefore need a negative weight; some Microduck penalty helpers already return a negative value and therefore need a positive weight. The observable invariant is that every logged penalty contribution should be at most zero.

The most common beginner mistake is to optimize a proxy and then judge the policy by the intended goal. Judge both:

- Does the logged reward term improve?
- Does the rollout visibly perform the desired behavior?
- Does it work across commands, resets, and randomized conditions?

1.7 Commands make one policy represent many behaviors

The velocity policy does not learn one fixed walk. A command c_t is part of the observation, so the policy is more accurately written as:

$$\pi_\theta(a_t \mid o_t, c_t)$$

Training samples many forward, lateral, turn, stand, and head-pose commands. Playback also resamples commands, which can look like a "random walk." The actions are not random at deployment: an application can deliberately set the command.

This distinction separates skill from intention:

```
planner or operator: where/why to move
          |
          | local velocity and pose command
          v
RL locomotion policy: how to move the joints stably
```

The current velocity task has no global waypoint, obstacle observation, or navigation memory. Those belong in an upper layer unless a new observation contract and task are intentionally designed.

1.8 Episodes and termination

Training divides experience into episodes. An episode resets when it reaches a time limit or a terminal failure condition. Resetting gives the policy fresh initial states and bounds the return.

Termination design changes the problem. Ending immediately on a fall teaches the locomotion policy to avoid falling. A stand-up policy must not terminate merely because it begins on the ground; the ground state is its task. This is why task templates differ even when they use the same robot.

1.9 On-policy learning

PPO is on-policy: each update uses recent data sampled by the current or very recent policy. After several optimization epochs, that rollout is discarded and new behavior is collected.

Consequences:

- simulation throughput matters;
- a rare state may receive too little training data;
- curricula and reset distributions control what data the policy sees; and
- old demonstrations or checkpoints are not automatically replayed as an off-policy dataset.

Reverse-curriculum resets are useful when a maneuver learns its beginning but never reaches its final state: starting some episodes near completion creates on-policy data at the missing frontier.

1.10 Check your understanding

1. Why is the 3D velocity command part of the observation rather than part of the action?
2. If an obstacle is visible in the renderer but absent from the actor observation, can the policy deliberately avoid it?
3. What is the difference between a high instantaneous reward and a high expected return?
4. Why can a reward function produce behavior its author did not intend?
5. Which state should terminate a walking episode but be a valid initial state for stand-up training?

Continue with the math and neural-network toolkit.

1.11 Folded solutions

Show answers to Section 1.10

1. The velocity command describes the behavior requested *of* the policy, so it must be an input. The action is the policy's response: fourteen immediate joint targets. If forward velocity were an action, the network would merely repeat the request instead of deciding how to move the mechanism.

2. No—not deliberately before contact. Pixels in a viewer are not numbers in the actor observation. The policy could learn a generic cautious gait or a post-contact reaction, but pre-contact avoidance needs an obstacle feature, map, range sensor, or an upper planner that changes the local command.
3. Instantaneous reward evaluates one transition. Expected return averages a discounted sequence of rewards over trajectories and uncertainty. A policy can accept one small immediate cost when it produces a much better future, or exploit a repeated small reward whose long-run sum is large.
4. A reward is a proxy written in code. Any unspecified orientation, contact, timing, or terminal condition may provide a cheaper way to score. The policy follows the mathematical signal, not the author's unstated visual intent.
5. A fallen body should normally terminate walking because continued fallen transitions are outside the skill. The same fallen pose is a legitimate reset state for stand-up, where recovery from it is the task.

2. Math and Neural-Network Toolkit

Deep reinforcement learning combines probability, calculus, linear algebra, and optimization. You do not need to master each subject before beginning. You do need to know what the symbols mean, what shape each quantity has, and what an equation asks the computer to do.

This chapter builds that minimum toolkit. Keep a pencil nearby: predicting a number before running code is much more educational than only reading it.

2.1 Scalars, vectors, matrices, and tensors

A **scalar** is one number, such as the reward at one instant:

$$r_t = 1.7.$$

A **vector** is an ordered list. A simple mobile-robot command might be

$$c_t = [v_x, v_y, \omega_z] = [0.2, 0.0, -0.4].$$

The order is part of the interface. Swapping v_x and v_y does not merely rename two values; it asks for a different physical motion.

A **matrix** is a rectangular array. A neural-network layer maps an input vector x to an output vector z using a weight matrix W and bias b:

$$z = Wx + b.$$

If x has 61 values and the layer has 512 neurons, then

$$x \in \mathbb{R}^{61}, \quad W \in \mathbb{R}^{512\times 61}, \quad b \in \mathbb{R}^{512}, \quad z \in \mathbb{R}^{512}.$$

Read $\mathbb{R}^{61}$ as "a vector of 61 real numbers." The matrix shape says 512 rows, one per output, and 61 columns, one per input. Shape reasoning catches many bugs before training.

A **tensor** is the programming term for a multidimensional numeric array. A Microduck rollout tensor might have shape

```
(time steps, parallel environments, observation values)
             (24, 4096, 61)
```

The word tensor is often used casually in deep-learning code; here it does not require advanced tensor calculus.

Units are an invisible part of every vector

Shape alone is not enough. A value might be radians, radians per second, meters, normalized units, or a Boolean encoded as 0/1. Write interfaces as both shape and meaning:

```
base angular velocity: shape (3,), unit rad/s, expressed in body frame
projected gravity:      shape (3,), dimensionless, body frame
```

```
joint position delta:  shape (14,), unit rad, relative to HOME
```

This discipline matters because a numerically valid 61-vector with the wrong units or frame still produces a physically wrong action.

2.2 Functions and parameters

A **function** maps an input to an output. A policy is a function:

$$a_t = \pi_\theta(o_t).$$

- o_t is the observation at time t;
- a_t is the action;
- π is the policy; and
- θ is the collection of trainable weights and biases.

The subscript θ means changing those parameters changes the function. Training searches for parameters that make useful trajectories more likely.

A **hyperparameter** is chosen by the experimenter rather than learned by the optimizer: learning rate, discount factor, network width, PPO clip value, or rollout length are examples.

2.3 Probability is how RL represents uncertainty

Robots face uncertainty from sensor noise, contact variation, delayed state, random commands, randomized simulation, and exploration.

A **random variable** is a numeric outcome of an uncertain process. We write

$$X \sim p(x)$$

as "X is sampled from probability distribution p." A Gaussian (normal) distribution is written

$$X \sim \mathcal{N}(\mu, \sigma^2),$$

where μ is the mean and σ is the standard deviation. Larger σ means more spread.

During training, a continuous policy commonly produces a mean action and a standard deviation, then samples:

$$a_t \sim \pi_\theta(\cdot \mid o_t) = \mathcal{N}(\mu_\theta(o_t), \operatorname{diag}(\sigma^2)).$$

Read this as: "given observation o_t, the network proposes the center of an action distribution, and an action is sampled from it." During deployment, the deterministic mean is often used instead.

Conditional probability

$p(a \mid o)$ means "the probability of action a given observation o." The vertical bar means "conditioned on." A policy must respond differently to a tilting-left observation and a tilting-right observation, so it represents a conditional distribution rather than one fixed action distribution.

Expectation

The **expectation** $\mathbb{E}[X]$ is a probability-weighted average. For a six-sided die,

$$\mathbb{E}[X] = \frac{1+2+3+4+5+6}{6} = 3.5.$$

No roll produces 3.5. Expectation describes the long-run average, not a guaranteed single result. Likewise, maximizing expected return does not guarantee that every robot episode succeeds. Tail failures need explicit measurement and safety handling.

Variance

Variance measures spread around the mean:

$$\mathrm{Var}(X) = \mathbb{E}[(X - \mathbb{E}[X])^2].$$

Two policies can have the same mean return while one fails violently in 10% of trials. Reporting only the mean hides that distinction.

2.4 Why logarithms appear in policy gradients

For independent events, probabilities multiply. Products of many numbers smaller than one become numerically tiny. Logarithms turn products into sums:

$$\log(xy) = \log x + \log y.$$

The log-probability of a sampled action is convenient to differentiate. A key identity is

$$\nabla_\theta \log \pi_\theta(a \mid o) = \frac{\nabla_\theta \pi_\theta(a \mid o)}{\pi_\theta(a \mid o)}.$$

You do not need to memorize the fraction yet. The practical idea is that the gradient can adjust the probability of actions that were actually sampled. Positive advantage pushes their log-probability up; negative advantage pushes it down.

2.5 Derivatives: sensitivity to change

The derivative of a scalar function says how its output changes for a small input change. If

$$y = x^2,$$

then

$$\frac{dy}{dx} = 2x.$$

At $x = 3$, the derivative is 6. Increasing x by approximately 0.01 increases y by approximately $6 \times 0.01 = 0.06$.

A **partial derivative** changes one input while holding others fixed. For

$$f(x, y) = x^2 + 3xy,$$

$$\frac{\partial f}{\partial x} = 2x + 3y, \qquad \frac{\partial f}{\partial y} = 3x.$$

A **gradient** collects all partial derivatives:

$$\nabla f = \begin{bmatrix} \partial f/\partial x \\ \partial f/\partial y \end{bmatrix}.$$

The gradient points toward the locally steepest increase. To minimize a loss $L(\theta)$, gradient descent takes a small step in the opposite direction:

$$\theta_{new} = \theta_{old} - \alpha \nabla_\theta L,$$

where α is the **learning rate**, the step-size hyperparameter.

Numerical example

Let $L(w) = (w - 4)^2$ and start at $w = 1$.

$$\frac{dL}{dw} = 2(w - 4) = -6.$$

With $\alpha = 0.1$:

$$w_{new} = 1 - 0.1(-6) = 1.6.$$

The update moves w toward 4. A huge learning rate could overshoot; a tiny one could make progress impractically slow.

2.6 The chain rule and backpropagation

Neural networks compose functions. Suppose

$$u = wx, \qquad y = \tanh(u), \qquad L = (y - y^*)^2.$$

The **chain rule** multiplies local sensitivities along the path:

$$\frac{dL}{dw} = \frac{dL}{dy}\frac{dy}{du}\frac{du}{dw}.$$

Substitute each derivative:

$$\frac{dL}{dw} = 2(y - y^*)\left(1 - \tanh^2(u)\right)x.$$

Backpropagation is an efficient application of this chain rule through a computation graph. Automatic differentiation libraries such as PyTorch compute it, but understanding the path helps diagnose detached tensors, exploding gradients, saturation, and unintended objectives.

2.7 A neural network is a learned nonlinear function

A feed-forward layer is

$$h = \phi(Wx + b),$$

where ϕ is an **activation function** applied element by element. Without nonlinear activations, many stacked linear layers collapse to one linear map.

Common activations include:

Activation	Formula or behavior	Practical note
ReLU	$\max(0, x)$	cheap; negative side has zero gradient

Activation	Formula or behavior	Practical note
ELU	x if positive, smooth negative saturation otherwise	used by the Microduck actor/critic
tanh	maps to $(-1, 1)$	useful for bounded outputs; can saturate
sigmoid	maps to $(0, 1)$	common for probabilities or gates

The Microduck actor is conceptually

```
61 observations -> 512 ELU -> 256 ELU -> 128 ELU -> 14 action means
```

The numbers 512, 256, and 128 are hidden-layer widths. This network is not a database of motions. Its weights define a smooth high-dimensional mapping from observations and commands to actions.

2.8 Loss, objective, reward, and return are different

These words are easy to mix up:

- **reward**: one scalar emitted by the environment at one step;
- **return**: a discounted sum of future rewards along a trajectory;
- **objective**: the quantity the algorithm conceptually wants to maximize;
- **loss**: a quantity the optimizer minimizes in code.

An implementation may minimize the negative policy objective, add a value loss, and subtract an entropy bonus. Seeing a negative sign in code does not by itself mean the robot is punished.

2.9 Stochastic gradient descent, minibatches, and epochs

Computing a gradient over all possible trajectories is impossible. RL estimates it from sampled experience.

A **batch** is the collected set of samples. A **minibatch** is one subset used for an optimizer step. An **epoch** is one pass through the batch. PPO can reuse one on-policy rollout for several epochs, but too much reuse makes the updated policy drift away from the policy that generated the data.

Optimizers such as Adam maintain moving estimates of gradient scale. Adam can make training easier, but it does not repair a wrong reward, missing observation, impossible target, or simulator error.

2.10 Normalization and numerical scale

Suppose one observation ranges around 0.01 and another around 1000. The same weight scale treats them very differently. Observation normalization maintains running mean μ and variance σ^2, then computes approximately

$$\hat{o} = \frac{o - \mu}{\sqrt{\sigma^2 + \epsilon}}.$$

ϵ is a small constant that avoids division by zero. Normalization can improve optimization, but it becomes part of the learned input contract. If training uses $\hat{o}$ and deployment feeds raw o, the actor receives a different problem. That is why Microduck's export path bakes the normalizer into ONNX.

Reward scaling also matters. Multiplying every reward by 100 leaves the mathematical optimal policy unchanged in an ideal tabular setting, but it changes gradient magnitude, value targets, clipping interactions, and numerical behavior in a practical deep-RL implementation.

2.11 Bias and variance: a recurring tradeoff

An estimator has **bias** when its average estimate is systematically shifted. It has **variance** when estimates fluctuate strongly across samples.

- Monte Carlo return uses a complete sampled future: low modeling bias, often high variance.
- A one-step temporal-difference target bootstraps from a learned estimate: potentially more bias, usually lower variance.
- Generalized Advantage Estimation introduces λ to move along this tradeoff.

"Unbiased" does not automatically mean "better." A very noisy gradient may need so much data that a slightly biased, lower-variance estimator learns more reliably.

2.12 A tiny gradient check

For a differentiable scalar function, compare the analytic derivative with a finite difference:

```
def f(w: float) -> float:
    return (w - 4.0) ** 2

w = 1.0
eps = 1e-5
finite_difference = (f(w + eps) - f(w - eps)) / (2.0 * eps)
analytic = 2.0 * (w - 4.0)
print(finite_difference, analytic)
```

They should be close to –6. Finite differences are slow in large networks, but the idea is valuable for checking a new reward derivative or small custom operation.

2.13 Exercises

1. A batch has shape `(24, 4096, 61)`. How many observation scalars does it contain?
2. A Gaussian policy has $\mu = 0.3$ and $\sigma = 0.1$. Is action 0.31 more or less typical than action 0.8? Why?
3. Compute one gradient-descent step for $L(w) = (w + 2)^2$, starting at $w = 1$ with learning rate 0.25.
4. Why can two policies with equal expected return have different hardware safety risk?
5. What breaks if an exported policy receives joint angles in degrees while training used radians?
6. Explain backpropagation without using the phrase "the computer learns."
7. Why does an observation normalizer belong in the deployment contract?

The small bandit example in `examples/bandit_incremental_mean.py` uses expectation, sampling, and an incremental mean without any neural network. Run it before continuing with Bellman methods from tables to DQN.

2.14 Folded solutions

Show answers to Section 2.13

1. Multiply every axis:

$$24 \times 4096 \times 61 = 5{,}996{,}544$$

These are scalar entries, not independent trajectories; the batch still has 24 correlated time positions per environment. 2. Action 0.31 is only 0.01/0.1 = 0.1 standard deviations from the mean. Action 0.8 is (0.8 – 0.3)/0.1 = 5 standard deviations away, so 0.31 is vastly more typical under that Gaussian. 3. The gradient at $w = 1$ is $2(w + 2) = 6$. Gradient descent gives $w_{new} = 1 - 0.25(6) = -0.5$. A direct check is:

```
w = 1.0
learning_rate = 0.25
gradient = 2.0 * (w + 2.0)
w -= learning_rate * gradient
```

```
assert w == -0.5
```

4. Equal means do not imply equal distributions. One policy may score consistently while another mixes excellent trials with rare destructive failures. Report quantiles, failure categories, interventions, and raw trials in addition to expected return.
5. One radian is about 57.3 degrees. Feeding degree-valued numbers into a model trained on radians changes its input scale and distribution, often driving normalized features far outside training experience and producing unsafe actions.
6. Backpropagation applies the chain rule from the final loss toward earlier operations. It multiplies each downstream sensitivity by each local derivative, accumulating how a small change in every weight would change the loss. The optimizer then uses those gradients to update the weights.
7. The actor learned a function of normalized observations, not raw sensor numbers. Mean, variance, clipping, epsilon, order, and units therefore form part of the deployed function and must travel with or be embedded in the exported model.

3. Bellman Methods: From Tables to Deep Q-Learning

Before PPO, learn the simpler idea that organizes much of reinforcement learning: the value of a decision is its immediate reward plus the value of what follows. This is the **Bellman principle**.

The tabular setting in this chapter is deliberately small. When every state and action can be listed, the algorithms expose their logic without a neural network hiding mistakes.

3.1 Start with a bandit: action without state transition

A **multi-armed bandit** has several actions ("arms"). Each arm produces reward from an unknown distribution. There is no changing state and no delayed consequence. The learner must balance:

- **exploration**: try uncertain actions to gain information; and
- **exploitation**: choose the action currently estimated to be best.

If action a has been selected $N(a)$ times, an incremental sample-average estimate is

$$Q_{new}(a) = Q_{old}(a) + \frac{1}{N(a)}\left(r - Q_{old}(a)\right).$$

Read the term in parentheses as **prediction error**: observed reward minus the old prediction. The $1/N(a)$ step size shrinks as evidence accumulates.

An ϵ-greedy policy chooses a random action with probability ϵ and the estimated best action otherwise. If $\epsilon = 0.1$, about 10% of decisions explore.

Robotics connection: choosing among calibration routines or high-level skills can resemble a contextual bandit, but balancing a moving robot is sequential. An action changes the next state and therefore future choices.

3.2 State value and action value

For a policy π, the **state-value function** is the expected discounted return starting from state s:

$$V^{\pi}(s) = \mathbb{E}_{\pi}[G_t \mid S_t = s].$$

The **action-value function**, usually called the Q-function, also fixes the first action:

$$Q^{\pi}(s, a) = \mathbb{E}_{\pi}[G_t \mid S_t = s, A_t = a].$$

Plain language:

- $V^{\pi}(s)$ asks, "How good is this situation if I continue with policy π?"
- $Q^{\pi}(s, a)$ asks, "How good is taking this action here, then continuing with π?"

These are expectations over future transition randomness and future policy sampling. They are not promises for one rollout.

3.3 The Bellman expectation equation

Split the return into the next reward and the remaining future:

$$G_t = R_{t+1} + \gamma G_{t+1}.$$

Taking expectations gives

$$V^\pi(s) = \sum_a \pi(a \mid s) \sum_{s',r} p(s', r \mid s, a) \left[r + \gamma V^\pi(s') \right].$$

Do not let the sums obscure the idea:

1. consider each action the policy might take;
2. consider each next state and reward the environment might produce;
3. add immediate reward to discounted next-state value; and
4. average using their probabilities.

This is a self-consistency equation: a correct value estimate agrees with a one-step lookahead using itself.

Numerical example

Suppose a state has one action. It gives reward 2 and moves deterministically to a state worth 5. With $\gamma = 0.9$:

$$V(s) = 2 + 0.9 \times 5 = 6.5.$$

If the next state were terminal, its future value would be zero and the answer would be 2.

3.4 Bellman optimality

The optimal action-value function chooses the best next action:

$$Q^*(s, a) = \sum_{s',r} p(s', r \mid s, a) \left[r + \gamma \max_{a'} Q^*(s', a') \right].$$

Once Q^* is known, a greedy optimal policy chooses

$$\pi^*(s) = \arg\max_a Q^*(s, a).$$

arg max returns the action producing the largest value, not the value itself.

3.5 Dynamic programming: when the model is known

Dynamic programming (DP) assumes the transition/reward model is known. Value iteration repeatedly applies an optimal Bellman backup:

$$V_{k+1}(s) = \max_a \sum_{s',r} p(s', r \mid s, a) \left[r + \gamma V_k(s') \right].$$

Repeated backups propagate information backward from goals and hazards.

Run:

```
python examples/gridworld_value_iteration.py
```

The program has the full grid transition model, so it can evaluate every one-step outcome without collecting experience. In most real robotics tasks, the exact stochastic model is unknown and continuous state makes a table impossible. DP still provides the conceptual reference.

3.6 Monte Carlo learning: wait for the outcome

Monte Carlo (MC) methods estimate value from complete sampled returns. If a state is visited and the later rewards are 1, 0, 3, with $\gamma = 0.9$:

$$G_t = 1 + 0.9(0) + 0.9^2(3) = 3.43.$$

The estimate can move toward 3.43 after the episode finishes. MC does not need a transition model and does not bootstrap from its own current value estimate. But it must wait for an outcome and can have high variance.

3.7 Temporal-difference learning: learn before the episode ends

Temporal-difference (TD) learning updates from one transition:

$$V(S_t) \leftarrow V(S_t) + \alpha\delta_t,$$

where

$$\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t).$$

δ_t is the **TD error**:

```
new one-step target - old prediction
```

The next state's current estimate supplies the unfinished future. Reusing an estimate inside its own target is called **bootstrapping**.

Numerical example

Let $V(S_t) = 4$, reward $R_{t+1} = 1$, $V(S_{t+1}) = 6$, $\gamma = 0.9$, and $\alpha = 0.1$.

$$\delta_t = 1 + 0.9(6) - 4 = 2.4,$$

$$V(S_t) \leftarrow 4 + 0.1(2.4) = 4.24.$$

The experience was better than predicted, so the estimate rises.

3.8 SARSA and Q-learning

SARSA is an on-policy TD control method. Its name lists the transition tuple: $S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1}$. Its update target follows the action the behavior policy actually selects:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\left[R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)\right].$$

Q-learning is off-policy. Its target assumes the greedy next action even if the behavior policy explored:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\left[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t)\right].$$

On-policy means learning about the behavior policy generating the data. **Off-policy** means the policy being learned can differ from the behavior policy. Off-policy learning enables data reuse but introduces harder stability and distribution-shift problems.

Run:

```
python examples/tabular_q_learning.py
```

Try three values of ϵ. Record both final success and the number of episodes needed. Exploration is not free, and no single setting is universally best.

3.9 From Q-table to Deep Q-Network

A robot camera produces too many possible observations for a table. A Deep Q-Network (DQN) approximates all discrete action values with a neural network:

$$Q_\theta(o, a).$$

The squared TD loss for one sample is

$$L(\theta) = \left(r + \gamma \max_{a'} Q_{\bar{\theta}}(o', a') - Q_\theta(o, a)\right)^2.$$

$\bar{\theta}$ denotes a slowly updated **target network**. Two important DQN mechanisms are:

- **experience replay**: store transitions and sample shuffled minibatches, which reuses data and reduces adjacent-sample correlation;
- **target network**: hold the target parameters fixed temporarily so the prediction does not chase a target changing on every optimizer step.

The original DQN paper demonstrated value learning from pixels on Atari. That is historically important evidence for discrete-action domains; it is not evidence that DQN is the natural choice for 14 simultaneous continuous joint targets.

3.10 Why continuous robot action changes the algorithm

For four discrete actions, computing $\max_a Q(s, a)$ is easy: evaluate all four. A 14D continuous action space contains infinitely many possible vectors. One cannot enumerate them.

Continuous-control families handle this differently:

- policy-gradient methods directly learn an action distribution;
- deterministic actor-critic methods learn an actor that approximately maximizes the critic;
- stochastic off-policy methods such as SAC optimize value plus entropy; and
- model-based methods plan through known or learned dynamics.

Microduck uses PPO because its simulator can generate large fresh on-policy batches and its action is continuous. This is an engineering fit, not a theorem that PPO is always the best robot algorithm.

3.11 The “deadly triad” warning

Learning can become unstable when three ingredients combine:

1. **function approximation**: a neural network generalizes across states;
2. **bootstrapping**: targets depend on current estimates; and
3. **off-policy learning**: training distribution differs from the target policy’s distribution.

This combination is called the **deadly triad**. It does not mean every such algorithm fails. It explains why replay buffers, target networks, double critics, conservative objectives, and careful evaluation exist.

3.12 Partial observability and memory

Bellman equations are written in terms of Markov state: all information needed to predict the future distribution. A deployed robot usually receives an observation o_t, not full state s_t. Hidden terrain compliance, actuator temperature, backlash, payload, and delayed contacts can make the problem a Partially Observable MDP (POMDP).

Common responses are:

- stack recent observations;
- add an explicit estimator;
- use a recurrent network with internal memory;
- train an adaptation module that infers latent environment properties; or
- add sensors.

No optimizer can reconstruct information that leaves no trace in the available history.

3.13 Exercises

1. With rewards $2, 1, 4$ and $\gamma = 0.5$, compute the three-step return.
2. Calculate the TD error when $V(s) = 3$, $r = -1$, $V(s') = 5$, and $\gamma = 0.9$. Does the value rise or fall for positive α?
3. Explain the difference between a model, a value function, and a policy.
4. Why can Q-learning learn a greedy target policy while collecting with an ϵ-greedy behavior policy?
5. Name the three parts of the deadly triad and one mitigation used by DQN.
6. Why is ordinary DQN inconvenient for a 14D continuous action vector?
7. Give one hidden physical property that can make a robot observation non-Markov and one way to expose or infer it.

Continue with the RL algorithm map, where these dimensions become a practical selection framework.

3.14 Folded solutions

Show answers to Section 3.13

1. The three-step return is $2 + 0.5(1) + 0.5^2(4) = 2 + 0.5 + 1 = 3.5$.
2. The TD error is $-1 + 0.9(5) - 3 = 0.5$. Because it is positive, an update with positive learning rate raises $V(s)$.
3. A **model** predicts what the environment will do after an action. A **value function** predicts accumulated future reward. A **policy** selects or distributes actions. A system may learn any one, two, or all three.
4. Q-learning's target uses the greedy next value, $\max_a Q(s', a)$, regardless of which exploratory action the behavior policy actually selects next. The behavior policy supplies coverage; the backup defines the target policy.
5. The deadly triad is function approximation, bootstrapping, and off-policy learning. DQN mitigates instability with a replay buffer and a temporarily fixed target network; neither is a universal convergence guarantee.
6. A 14D continuous vector has infinitely many candidates, so ordinary DQN cannot enumerate every action to compute a maximum. Continuous actor-critic methods learn an actor that proposes the maximizing action instead.
7. Hidden ground friction is one example. Recent wheel/contact history or a learned adaptation latent can infer its effect; an additional force/slip sensor can expose it more directly. Motor temperature, payload, and backlash are other valid examples.

The arithmetic behind answers 1 and 2 can be checked with:

```
discount = 0.5
three_step_return = 2 + discount * 1 + discount**2 * 4
td_error = -1 + 0.9 * 5 - 3
assert three_step_return == 3.5
assert td_error == 0.5
```

4. The Reinforcement-Learning Algorithm Map

Algorithm names can make RL feel like a collection of unrelated inventions. Most methods can be located along a few design axes. Learning those axes is more durable than memorizing a leaderboard.

4.1 First ask what data interaction is allowed

Online RL

In **online RL**, the agent collects new experience while learning. The data distribution changes as the policy changes.

- Simulation makes online interaction cheap and safe enough for millions or billions of transitions.
- Physical online learning may consume robot time, wear actuators, or discover unsafe actions.

Offline RL

In **offline RL**, learning uses a fixed logged dataset and cannot request new transitions. It is attractive when robot data already exists or new exploration is dangerous. It is difficult because value estimates for actions absent from the dataset can be arbitrarily wrong.

Imitation learning

Imitation learning learns from demonstrations, often by treating expert actions as supervised labels. It may not use a reward or Bellman backup at all. Modern robot-policy research mixes imitation, offline RL, online fine-tuning, and pretrained representations, so “robot learning” is broader than RL.

4.2 Model-free versus model-based

A **dynamics model** predicts how state changes:

$$\hat{s}_{t+1} = f_\psi(s_t, a_t).$$

- **Model-free RL** learns a policy and/or value without using a learned transition model for planning.
- **Model-based RL** uses a known or learned model to evaluate or optimize future action sequences.

The labels do not mean model-free robot design has no physics knowledge. A PPO policy trained in MuJoCo relies heavily on a simulator model to generate data; the learning algorithm itself simply does not differentiate through or plan with a learned f_ψ at runtime.

Model-based methods can be more sample efficient because one transition helps learn dynamics reusable by many plans. Their danger is **model bias**: planning can exploit small prediction errors and imagine impossible success.

4.3 Value-based, policy-based, and actor-critic

Value-based

Value-based methods learn V or Q and derive behavior by choosing a high-value action. DQN is the canonical deep discrete-action example.

Policy-based

Policy-gradient methods directly adjust a parameterized policy $\pi_\theta(a \mid s)$ toward actions associated with higher return. Continuous actions are natural because the policy can output distribution parameters.

Actor-critic

An **actor-critic** has both:

- actor: produces actions;
- critic: evaluates states or state-action pairs to improve the actor.

PPO, SAC, and TD3 are actor-critic methods, but their data use and objectives differ substantially.

4.4 On-policy versus off-policy

An on-policy method updates from data collected by the current or very recent policy. PPO is approximately on-policy. Old data becomes stale after a policy change, so it is normally discarded.

An off-policy method can update a target policy using data from other behavior policies. SAC, TD3, DQN, IQL, and CQL are off-policy in different settings. A replay buffer improves sample reuse but makes the learning distribution less like current deployment.

The tradeoff is not simply “off-policy is better because it reuses data”:

Property	On-policy	Off-policy
data reuse	low	high
implementation/stability	often simpler	often more delicate
massively parallel simulation	excellent fit	possible, needs care
scarce real interaction	expensive	often preferable
distribution correction	limited by fresh data	central challenge

4.5 Stochastic versus deterministic policies

A **stochastic policy** describes a distribution over actions. Exploration can come from sampling that distribution. PPO and SAC train stochastic policies.

A **deterministic policy** maps a state to one action. TD3 learns a deterministic actor and adds noise during data collection. A deployed stochastic-policy actor may also use its mean deterministically.

Stochastic does not mean careless or random behavior. The distribution is conditioned on the observation, and its spread is learned or configured.

4.6 Core families at a glance

Method	Action	Data	Learned objects	Characteristic idea	Typical fit
tabular Q-learning	discrete	online, off-policy	Q-table	greedy Bellman target	small teaching/control problem
DQN	discrete	online replay, off-policy	Q-network	replay + target network	games or finite skill choices
PPO	discrete or continuous	online, on-policy	actor + value critic	clipped policy-ratio objective	high-throughput simulation
TD3	continuous	online replay, off-policy	deterministic actor + twin critics	clipped double-Q + delayed actor	sample-efficient state control
SAC	continuous; discrete variants exist	online replay, off-policy	stochastic actor + critics	maximize reward and entropy	continuous control with costly data
IQL/CQL	usually continuous	fixed offline data	values/critics + policy	avoid optimistic unseen actions	logged robot datasets
DreamerV3	varied	online replay, model-based	latent world model + actor/critic	learn through imagined rollouts	pixels/general domains
TD-MPC2	continuous	online replay, model-based	latent model + values/policy	local latent trajectory optimization	sample-efficient continuous control
behavior cloning	any represented in data	demonstrations	policy	supervised action prediction	strong expert data
diffusion policy	continuous action chunks	demonstrations	conditional diffusion model	model multimodal action sequences	visual manipulation

This table is a map, not a ranking. Performance depends on implementation, task, data, compute, observation/action choices, and evaluation.

4.7 Four influential objectives in plain language

DQN: make Q agree with a bootstrapped target

$$\min_{\theta} \left(r + \gamma \max_{a'} Q_{\theta}(s', a') - Q_{\theta}(s, a) \right)^2 .$$

Move the current action value toward immediate reward plus the target network's best next value.

PPO: improve sampled actions without moving probability too far

$$\max_{\theta} \mathbb{E}\left[\min(r_t A_t, \text{clip}(r_t, 1 - \epsilon, 1 + \epsilon) A_t) \right].$$

Increase probability for better-than-expected sampled actions and decrease it for worse ones, while clipping the incentive for a large probability-ratio change. Chapter 5 derives every term.

TD3: trust the smaller of two learned critics

$$y = r + \gamma \min_{i=1,2} Q_{\phi_i}(s', \pi_{\theta}(s') + \text{clipped noise}).$$

Using the smaller critic reduces overestimation; delayed actor updates let the critics settle between policy changes. See the primary TD3 paper.

SAC: value both reward and action entropy

$$J(\pi) = \mathbb{E}\left[\sum_t \gamma^t \big(r_t + \alpha \mathscr{H}(\pi(\cdot \mid s_t))\big)\right].$$

Entropy $\mathscr{H}$ measures distributional spread. SAC rewards task success and, weighted by temperature α, retaining multiple plausible actions. This often improves exploration and robustness. See the primary SAC paper.

4.8 Exploration is part of the system design

Exploration mechanisms include:

- ϵ-greedy random discrete actions;
- action noise around a deterministic actor;
- sampling a stochastic actor;
- an entropy bonus;
- randomized initial states and goals;
- curricula that expose increasingly difficult regions;
- demonstrations or reverse-curriculum starts that place the agent near rare useful states; and
- intrinsic rewards for novelty or prediction error.

Robot exploration must respect hardware. “Let the agent discover it” is not a safety plan. Most aggressive motor-skill exploration belongs in simulation, with a separately enforced hardware envelope.

4.9 A practical selection procedure

Ask in this order:

1. **Is the action discrete or continuous?** DQN does not naturally solve a 14D continuous joint command.
2. **Can you collect fresh interaction cheaply?** If yes, on-policy simulation may be simple and effective. If no, consider replay, offline data, or a model.
3. **Is the observation low-dimensional state or high-dimensional vision?** Pixels increase representation and data demands.
4. **Do you have demonstrations?** A supervised initialization may remove the hardest exploration phase.
5. **Is a reliable model available?** Model-predictive control or model-based RL may exploit it.
6. **Does the task need memory/adaptation?** Select history, recurrence, or latent inference explicitly.
7. **What is the safety boundary?** Constraints and an independent runtime supervisor may matter more than algorithm choice.
8. **What baseline must learning beat?** Start with a scripted or classical controller when one exists.

4.10 Why PPO is sensible for Microduck

Microduck has:

- a 14D continuous action;
- thousands of GPU-parallel simulated robots;
- inexpensive fresh rollouts;
- dense task/recovery signals; and
- a small actor needed for 50 Hz deployment.

These properties fit PPO. One iteration with 4,096 environments and 24 steps collects 98,304 fresh transitions. Low sample reuse is less painful when simulation generates this much data quickly.

PPO does not cause walking by itself. Robot model, observation, command distribution, reward, reset distribution, actuator dynamics, randomization, and curriculum define the experience from which PPO learns.

4.11 Why a different robot may need a different method

Consider three projects:

A simulated biped locomotion controller

Continuous action, cheap massively parallel experience, 50–100 Hz inference. PPO is a strong baseline.

A real arm with 200 successful demonstrations

Physical trial cost is high and demonstrations already specify behavior. Behavior cloning or a diffusion/action-chunk policy is a sensible first baseline; offline RL may improve it if reward labels and dataset coverage are credible.

A wheeled robot choosing among “follow,” “dock,” and “stop”

The high-level choice is discrete and slow, but each skill has its own continuous controller. A hierarchical architecture may use a planner or discrete policy above classical/RL motor skills. One monolithic algorithm is not required.

4.12 Common selection mistakes

- Choosing the newest paper before defining the observation and action.
- Comparing algorithms with different reward, model, environment count, or compute budgets.
- Calling behavior cloning “RL” because the policy is a neural network.
- Calling a simulator-trained model “model-based RL” merely because the simulator has physics.
- Assuming sample efficiency implies wall-clock efficiency.
- Reporting the best seed instead of a distribution.
- Using a generalist visual model in a hard realtime loop without measuring worst-case latency.

4.13 Exercises

For each scenario, select a starting algorithm and state what evidence would change your mind:

1. Cart-pole with left/right discrete actions and unlimited simulation.
2. Microduck velocity tracking in 4,096 parallel environments.
3. A robot arm with a fixed dataset and no permission for online exploration.
4. An image-based manipulation task with several valid grasp trajectories.
5. A rover with a known dynamics model, strict constraints, and a 20 Hz local planner.
6. A legged robot whose payload changes during an episode.

Then make a two-column list: what the learning algorithm decides versus what the system architecture must decide. Continue with PPO from equations to code.

4.14 Folded solutions

Show reference answers to Section 4.13

1. **Cart-pole:** begin with tabular Q-learning after discretization or DQN for a neural baseline. Unlimited simulation and two discrete actions fit value learning. A continuous-action variant or poor pixel sample efficiency would motivate an actor-critic or representation change.
2. **Microduck:** begin with PPO. Its continuous action and 4,096 cheap parallel simulators make fresh on-policy data practical. A matched-budget experiment showing better robustness or wall-clock cost from SAC/TD3 would change the choice.

3. **Fixed arm dataset:** start with behavior cloning, then compare IQL or CQL if rewards and coverage support offline improvement. Permission for safe online interaction would open a separate fine-tuning phase.
4. **Multimodal visual manipulation:** start with a diffusion or other multimodal imitation policy. Plain squared-error BC may average incompatible grasps. Additional reward-bearing data could justify offline RL.
5. **Known constrained rover:** start with model-predictive control (MPC), which can use the known model and express constraints. Learn a residual/model only if held-out traces reveal systematic error the baseline cannot handle.
6. **Changing payload:** start with a history-conditioned/adaptive locomotion actor, using privileged teacher information only during training. If the payload is measured directly, explicit estimation plus a robust controller may be simpler.

The learning algorithm decides how experience changes a policy, value, or model. The architecture still decides sensor/calibration ownership, control rate, action semantics, planner decomposition, hard limits, watchdog, E-stop, compute placement, data privacy, and fallback. Algorithm selection cannot make those system decisions disappear.

5. PPO from Equations to Code

This chapter explains how Proximal Policy Optimization turns a batch of Microduck rollouts into improved actor and critic networks.

5.1 Actor and critic

The actor chooses actions. The critic estimates how much discounted reward is still available from the current situation.

$$\text{actor: } \pi_\theta(a_t \mid o_t)$$

$$\text{critic: } V_\phi(x_t) \approx \mathbb{E}[G_t \mid x_t]$$

The actor must use observations available on the real robot. The critic is needed only during training and may receive privileged simulator information. In the main velocity task:

Network	Input	Hidden layers	Output
Actor	61	512 → 256 → 128, ELU	14 action means
Critic	76	512 → 256 → 128, ELU	1 value estimate

The critic's extra information includes true base linear velocity and contact features. This asymmetric actor-critic pattern can make training easier without creating a deployment dependency.

The actor uses a Gaussian distribution during training. Its network predicts the mean action, and an exploration standard deviation allows nearby actions to be sampled. Deployment uses the inference policy rather than deliberately sampling exploratory motor commands.

The layer widths, learning rate, discount, and similar choices are **hyperparameters**: settings chosen by the engineer rather than weights learned by gradient descent. A **gradient** describes how a small parameter change would change the loss; backpropagation efficiently computes those gradients through the network.

5.2 Why a critic helps

Suppose a robot receives reward 1 in a state. Is that good? If similar states normally produce reward 5, it was worse than expected. If they normally produce reward 0, it was better than expected.

The advantage measures this relative quality:

$$A_t = Q(o_t, a_t) - V(o_t)$$

A positive advantage means the sampled action led to a better outcome than the critic expected. A negative advantage means it was worse. Policy-gradient learning increases the probability of positive-advantage actions and decreases the probability of negative-advantage actions.

5.3 Temporal-difference error and GAE

The one-step temporal-difference residual is:

$$\delta_t = r_t + \gamma V_\phi(x_{t+1}) - V_\phi(x_t)$$

Generalized Advantage Estimation (GAE) combines a sequence of these residuals:

$$\hat{A}_t = \delta_t + (\gamma\lambda)\delta_{t+1} + (\gamma\lambda)^2\delta_{t+2} + \cdots$$

Microduck uses $\gamma = 0.99$ and $\lambda = 0.95$. Lower λ relies more on the critic and usually lowers variance; higher λ uses longer sampled returns and usually lowers bias. The chosen values are a common compromise, not a law of robotics.

The value target is commonly formed from the advantage and old value estimate:

$$\hat{G}_t = \hat{A}_t + V_{\text{old}}(x_t)$$

The critic learns by reducing error between $V_\phi(x_t)$ and this target.

5.4 The policy ratio

After collecting a rollout with policy $\pi_{\theta_{old}}$, PPO asks how the new policy changes the probability of each sampled action:

$$r_t(\theta) = \frac{\pi_\theta(a_t \mid o_t)}{\pi_{\theta_{old}}(a_t \mid o_t)}$$

- $r_t = 1$ means no probability change.
- $r_t > 1$ means the sampled action became more likely.
- $r_t < 1$ means it became less likely.

A basic policy-gradient objective would maximize $r_t\hat{A}_t$. Reusing the same rollout for aggressive updates can move the policy so far that the data no longer represents it. PPO limits the incentive for a large move.

5.5 The clipped PPO objective

The clipped surrogate objective is:

$$L^{CLIP}(\theta) = \mathbb{E}_t\left[\min\left(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t\right)\right]$$

Microduck uses $\epsilon = 0.2$, so the clipped interval is $[0.8, 1.2]$.

Example: an action has advantage +2, its old probability was 0.10, and its new probability is 0.13. The ratio is 1.3. The unclipped objective is 2.6, but the clipped positive-advantage objective is $1.2 \times 2 = 2.4$. Increasing its probability further offers no clipped benefit for this sample.

Clipping does not mathematically guarantee a small update. RSL-RL also tracks the approximate Kullback–Leibler (KL) divergence, a measure of how different the old and new action distributions are, and uses an adaptive learning-rate schedule with desired KL `0.01`. Gradient norm is capped at `1.0`.

5.6 Value, entropy, and the combined update

The optimizer conceptually combines three goals:

```
maximize clipped policy improvement
minimize critic value error
preserve enough action-distribution entropy to explore
```

One common minimization form is:

$$L = -L^{CLIP} + c_v L^{value} - c_e H[\pi_\theta]$$

The main configuration uses value-loss coefficient `1.0` and entropy coefficient `0.01`. Entropy is not "randomness is always good." Early in training it prevents the policy from collapsing before discovering useful motion. Too much entropy can keep behavior noisy; too little can freeze a poor strategy.

5.7 What one Microduck iteration contains

The default runner uses:

```
parallel environments       4096 in a normal full run
steps per environment         24
rollout transitions       98,304 per iteration
minibatches                    4
learning epochs                5
initial learning rate       0.001
```

The same rollout is shuffled into four **minibatches** (smaller parts of the full collected batch) and visited for five optimization **epochs** (complete passes over that rollout). Then the updated policy collects a new rollout.

At a 50 Hz policy rate, 24 steps represent 0.48 seconds per environment. The large parallel batch contains many commands, initial conditions, contacts, and randomized robot instances even though each individual fragment is short.

The logical loop is:

```
for iteration in range(max_iterations):
    rollout = collect_current_policy(num_envs=4096, steps=24)
    advantages, returns = generalized_advantage_estimation(rollout)

    for epoch in range(5):
        for minibatch in split_and_shuffle(rollout, count=4):
            update_actor_with_clipped_ppo(minibatch)
            update_critic(minibatch)

    log_metrics_and_periodically_save_checkpoint()
```

This is explanatory pseudocode. RSL-RL owns the actual buffers, distribution, losses, optimizer, and checkpoint format.

5.8 Observation normalization

Joint angles, angular velocities, gravity components, and commands have different numerical scales. The actor and critic use empirical observation normalizers so one large-magnitude feature does not dominate merely because of its units.

Conceptually, each feature becomes:

$$\tilde{o}_i = \frac{o_i - \mu_i}{\sqrt{\sigma_i^2 + \varepsilon}}$$

The running mean and variance are learned from training observations and saved in the checkpoint. The ONNX exporter includes the actor's normalizer in the graph. A hand-converted network that omits it receives a different numerical problem and may fail on hardware even if playback from the checkpoint looked correct.

5.9 The important hyperparameters in this repository

Setting	Main value	Practical effect
`gamma`	0.99	how strongly later rewards count
`lam`	0.95	GAE bias/variance tradeoff
`clip_param`	0.2	clips the policy-ratio incentive
`learning_rate`	0.001	initial optimizer step scale
`desired_kl`	0.01	target used by adaptive schedule
`entropy_coef`	0.01	exploration pressure
`num_learning_epochs`	5	reuse passes over each rollout
`num_mini_batches`	4	subdivisions per epoch
`max_grad_norm`	1.0	gradient clipping threshold
actor/critic normalization	on	normalizes each observation stream

Do not tune these merely because a reward curve looks noisy. First verify the environment, reward signs, resets, observation validity, and actual rollout. Most project failures have been specification or physics failures rather than a need for exotic PPO settings.

5.10 Why PPO does not understand intent

PPO only sees sampled data and scalar objectives. It does not know that a forward roll should be sagittal rather than a shoulder roll, or that “stand” should not mean balancing on the head. Hard state-based gates and correct task distributions translate that intent into the optimization problem.

This is also why total reward can improve while the task fails. The agent may learn cheaper regularizers without improving the main skill. Always inspect individual weighted reward terms and rollouts.

5.11 Check your understanding

1. Why can the critic use information that is unavailable on the real robot?
2. What does an advantage of zero mean for a sampled action?
3. What problem does PPO clipping reduce?
4. How many transitions are collected in one 4,096-environment iteration?
5. Why must observation normalization be included in the deployed ONNX graph?

Continue with off-policy and model-based reinforcement learning.

5.12 Folded solutions

Show answers to Section 5.11

1. The critic is used to estimate returns/advantages during training and is not part of the deployed actor. Simulator-only truth can therefore reduce critic noise while the actor remains restricted to reproducible sensors. Leakage into actor inputs would break this argument.
2. Zero advantage means the sampled result matched the critic’s baseline for that observation. The policy-gradient term has no first-order reason to make that action more or less likely, though entropy and other loss terms may still update the policy.
3. Clipping reduces the incentive for one batch to move action probabilities too far from the behavior policy that collected it. It is a practical trust- region surrogate, not a proof that every update improves the real task.
4. One iteration collects $4096 \times 24 = 98{,}304$ transitions.
5. Normalization changes the function’s actual input. Omitting the saved mean and variance makes the ONNX actor see a different numeric distribution than the network optimized in training.

A minimal transition-count check is:

```
num_envs = 4096
steps_per_env = 24
```

```
assert num_envs * steps_per_env == 98_304
```

6. Off-Policy and Model-Based Reinforcement Learning

PPO throws away old rollouts after a few update epochs. That can be entirely reasonable in fast parallel simulation, but it feels wasteful when one physical robot transition is expensive. Off-policy and model-based methods try to learn more from each interaction.

This chapter explains the motivation and failure modes before the algorithms.

6.1 Replay buffers: experience as a reusable dataset

An off-policy learner commonly stores transitions

$$(s_t, a_t, r_t, s_{t+1}, d_t)$$

in a **replay buffer**, where d_t indicates termination. Training samples minibatches from the buffer rather than only the newest trajectory.

Replay provides:

- reuse of expensive experience;
- shuffled samples instead of strongly correlated time neighbors; and
- a mixture of behavior from several policy versions.

That mixture is also the difficulty. The current actor may visit states and actions differently from the buffer. A critic can be asked about actions for which the data provides weak evidence.

6.2 State-action critics

PPO usually uses a state-value critic $V(s)$. Continuous off-policy actor-critic methods often learn

$$Q_\phi(s, a),$$

the expected return for taking action a in state s and following a policy afterward.

A one-step critic target has the form

$$y = r + \gamma(1 - d)\,\text{next-value}.$$

The factor $(1 - d)$ removes future value after a true terminal transition. A time-limit truncation may need different handling because the underlying task could have continued. Confusing termination with truncation biases targets.

6.3 Why critics become overoptimistic

Suppose a critic has small random errors across actions. Selecting the maximum tends to select not only a genuinely good action but also a positive estimation error. Repeating this in bootstrapped targets can inflate values.

Robot consequence: the actor may exploit critic errors by proposing an out-of-distribution action that the critic imagines is excellent, even though the dataset never demonstrated it safely.

Two widely used responses are:

- learn two critics and use the smaller estimate; and
- constrain or regularize actions toward regions covered by data.

6.4 TD3: deterministic continuous control

Twin Delayed Deep Deterministic Policy Gradient (TD3) learns:

- deterministic actor $a = \pi_\theta(s)$;
- two critics Q_{ϕ_1} and Q_{ϕ_2}; and
- slowly moving target copies.

Its target is approximately

$$y = r + \gamma(1 - d) \min_{i=1,2} Q_{\bar{\phi}_i}(s', \pi_{\bar{\theta}}(s') + \epsilon),$$

where target noise ϵ is clipped.

The three ideas encoded in the name/paper are:

1. **clipped double Q**: use the smaller of two critics to reduce optimistic error;
2. **delayed policy update**: update the actor less often than critics; and
3. **target policy smoothing**: train the value of a neighborhood, not a narrow action spike.

The actor is optimized to choose actions its critic values:

$$\max_\theta \mathbb{E}_{s\sim D}[Q_{\phi_1}(s, \pi_\theta(s))].$$

The TD3 paper isolates these mechanisms. Its benchmark findings support those mechanisms in the evaluated continuous control tasks; they do not erase the need for robot-specific evaluation.

6.5 SAC: maximum-entropy actor-critic

Soft Actor-Critic (SAC) learns a stochastic policy and adds entropy to the return:

$$J(\pi) = \mathbb{E}\left[\sum_{t=0}^{T-1} \gamma^t \left(r_t + \alpha \mathcal{H}(\pi(\cdot \mid s_t))\right)\right].$$

$\mathcal{H}$ is entropy and α is a temperature. In plain language, the policy values reward while retaining action diversity when several actions are plausible.

For continuous actions, implementations commonly sample an unconstrained Gaussian variable using a differentiable reparameterization, then apply `tanh` to bound it:

$$u = \mu_\theta(s) + \sigma_\theta(s) \odot \xi, \quad \xi \sim \mathcal{N}(0, I), \quad a = \tanh(u).$$

$\odot$ means elementwise multiplication. Writing randomness as an input ξ allows gradients to flow through μ and σ.

A soft critic target includes the next action's entropy term:

$$y = r + \gamma(1 - d)\left[\min_i Q_{\phi_i}(s', a') - \alpha \log \pi_\theta(a' \mid s')\right].$$

The actor minimizes approximately

$$J_\pi = \mathbb{E}\left[\alpha \log \pi_\theta(a \mid s) - \min_i Q_{\phi_i}(s, a)\right].$$

Interpretation: prefer actions the critics value, while paying a cost for collapsing the action distribution too sharply. Modern SAC often tunes α toward a target entropy instead of fixing it.

The primary SAC paper motivated stable, sample-efficient continuous control. "Sample efficient" still depends on what counts as a sample, update-to-data ratio, environment, and compute.

6.6 PPO versus SAC as an engineering decision

Question	PPO tendency	SAC tendency
Fresh simulation is cheap?	excellent	also possible
Each transition is expensive?	wastes more data	replay is attractive
Thousands of synchronous environments?	natural fit	replay/update design needs care
Simplicity of distributed rollout?	relatively simple	buffer and target networks add state
Old data from prior policies?	mostly discarded	reusable if relevant
Primary instability concern	policy update size	critic error/distribution shift

Do not compare only final reward. Match environment steps, wall time, compute, network size, evaluation protocol, and number of seeds.

6.7 What a model adds

A dynamics model predicts the next state distribution and possibly reward:

$$p_\psi(s_{t+1}, r_t \mid s_t, a_t).$$

If the model is known, Model Predictive Control (MPC) can repeatedly:

1. observe the current state;
2. simulate candidate action sequences;
3. score predicted futures;
4. execute only the first action; and
5. replan from the next observation.

Executing only the first action is **receding-horizon control**. Replanning limits damage from prediction error because reality corrects the plan at every step.

Model-based RL learns some or all of the model and uses it to improve behavior.

6.8 Model error compounds through imagination

Let a learned model have a small one-step error. Rolling it forward for 100 steps feeds predictions back as inputs, so error can compound. The policy or planner may also seek regions where the model is wrong in a favorable way.

Mitigations include:

- short planning horizons;

- model ensembles to estimate disagreement;
- uncertainty penalties;
- frequent replanning from real observations;
- training the model on the policy's current distribution;
- latent representations that focus on control-relevant information; and
- value functions to approximate outcomes beyond the short model horizon.

6.9 Latent world models

Images contain far more pixels than control-relevant state. A **latent model** compresses observations into a learned representation z_t, predicts latent dynamics, and learns reward/value in that space.

Conceptually:

```
camera/history -> encoder -> latent state z_t
                                  |
                            learned dynamics
                      (z_t, a_t) -> z_{t+1}
                                  |
                          imagined trajectories
                                  |
                            actor/value updates
```

The latent state is not guaranteed to correspond to human-named variables. It is optimized to support reconstruction, prediction, reward, value, or a combination.

6.10 DreamerV3

DreamerV3 learns a world model from replay, then trains actor and critic from trajectories imagined in its latent space. The paper evaluates a single configuration across more than 150 tasks in several domains and emphasizes normalization and scale-robust objectives.

The important conceptual contribution for a beginner is not "Dreamer replaces PPO." It is the separation:

```
real/sim experience -> learn predictive latent world
learned world       -> generate imagined futures
imagined futures    -> improve policy/value
```

Questions to ask before applying it to a robot:

- Does the latent model predict contacts and discontinuities well enough?
- What observation history resolves partial observability?
- Does imagined success correlate with real rollout success?
- What is the runtime actor size and latency?
- How is uncertainty outside the replay distribution handled?

6.11 TD-MPC2

TD-MPC2 combines a learned latent model, temporal-difference values, a policy prior, and local trajectory optimization. Rather than decode future images, it learns task-relevant latent predictions for control. The paper reports results across 104 online RL tasks and explores multi-task scaling.

At each decision, planning conceptually samples/refines short action sequences in latent space, uses learned reward and terminal value to score them, and executes the first action. The policy helps propose actions and can also serve as a fast behavior prior.

This offers a different runtime tradeoff from a small feed-forward PPO actor: planning may improve data efficiency or adaptation but consumes more per-step compute and introduces planner/model failure modes.

6.12 Known physics, learned residuals, and hybrid control

Robot learning need not choose between “all classical” and “all neural.” A hybrid can use:

- known rigid-body dynamics for nominal prediction;
- a learned residual for unmodeled friction or compliance;
- MPC for explicit constraints;
- an RL policy for fast local correction; and
- a hard safety supervisor outside both.

A **residual policy** outputs a bounded correction:

$$u = u_{nominal} + \Delta u_\theta, \qquad |\Delta u_\theta| \le u_{residual,max}.$$

This can reduce exploration scope and preserve an interpretable baseline. However, a poorly defined residual can still destabilize the nominal loop.

6.13 A fair comparison experiment

To compare PPO and SAC on one simulated robot:

1. freeze robot model, observations, actions, reward, resets, randomization, control rate, and evaluator;
2. give both enough network capacity but report parameter counts;
3. report environment transitions and wall-clock/GPU time;
4. evaluate deterministic policies on identical held-out seeds and physics;
5. show median and uncertainty across training seeds;
6. include failure categories, not only average return; and
7. preserve resolved configs and checkpoints.

Changing algorithm and reward together answers no clean question.

6.14 Exercises

1. Why can a replay buffer improve sample efficiency but worsen distribution mismatch?
2. Explain why taking the minimum of two noisy critics can reduce optimistic targets. What new pessimistic bias can it introduce?
3. In SAC, what happens qualitatively as α approaches zero?
4. A true terminal transition has $d = 1$. Evaluate $y = r + \gamma(1 - d)V(s')$.
5. Why does planning farther into a learned model sometimes reduce real-world performance?
6. Compare actor-only inference with MPC through a learned model for a 100 Hz loop. What must be measured?
7. Design a bounded residual action for a wheeled-leg robot with a classical balance controller.

Continue with robot dynamics, control, and estimation.

6.15 Folded solutions

Show answers to Section 6.14

1. Replay reuses each expensive transition and decorrelates adjacent samples, improving transition efficiency. But old data came from older policies and possibly older task/robot distributions, so its state-action coverage can differ from the policy currently being optimized.
2. If critic noise is sometimes optimistically high, taking the smaller of two estimates makes that error less likely to enter the target. The minimum can instead be systematically low, creating conservative or pessimistic bias.
3. As SAC temperature α approaches zero, entropy contributes less and the objective approaches ordinary reward-maximizing actor-critic behavior. Exploration/diversity pressure weakens.

4. With $d = 1$, the bootstrap multiplier is zero, so $y = r$. Bootstrapping beyond a true terminal state would incorrectly assign value after the episode has ended.
5. Small one-step model errors compound as predicted states become inputs to later predictions. Farther plans can exploit model mistakes or cross contact events the model predicts poorly; shorter receding horizons refresh from reality more often.
6. Measure worst-case—not just average—inference/planning time, jitter, deadline misses, horizon/sample count, model error, memory, observation age, action continuity, and fallback behavior. A 100 Hz loop has only 10 ms for the entire sensor-to-command path.
7. Let a classical balance controller output wheel target u_c and constrain the learned correction:

```
residual_limit_rad_s = 1.0
residual = max(-residual_limit_rad_s,
               min(residual_limit_rad_s, actor_output))
wheel_target = classical_target + residual
```

   Expire the residual on stale input or deadline miss, rate-limit the combined target, preserve MCU current/speed/tilt limits, and prove that residual zero returns to the accepted baseline.

7. Robot Dynamics, Control, and State Estimation

An RL policy becomes a robot controller only through mechanics, electronics, sensors, timing, and lower-level control loops. This chapter supplies the robotics background needed to understand those interfaces.

7.1 Configuration and degrees of freedom

A robot's **configuration** describes its pose. Joint coordinates are commonly collected in a vector q; joint velocities are $\dot{q}$ and accelerations are $\ddot{q}$.

A **degree of freedom** (DoF) is an independent coordinate needed to describe motion. A single revolute hinge has one DoF: its angle. A free rigid body in 3D has six: three position coordinates and three orientation coordinates.

Microduck has 14 actuated joint coordinates, but its simulated state also includes the free base pose and velocity. "14-action policy" therefore does not mean the full physical state has dimension 14.

7.2 Position, velocity, acceleration, force, and torque

- position tells where something is;
- velocity is the rate of position change;
- acceleration is the rate of velocity change;
- force changes linear momentum; and
- torque is the rotational counterpart of force.

For a revolute joint,

$$\tau = Ia$$

is the simplest analogy: torque τ produces angular acceleration a according to rotational inertia I. A whole robot is coupled, so moving one joint can accelerate many links.

7.3 The robot dynamics equation

A standard compact rigid-body equation is

$$M(q)\ddot{q} + C(q, \dot{q})\dot{q} + g(q) + f(\dot{q}) = \tau + J(q)^T F_{ext}.$$

Read it term by term:

- $M(q)\ddot{q}$: inertia resisting acceleration;
- $C(q, \dot{q})\dot{q}$: velocity-dependent Coriolis/centrifugal effects;
- $g(q)$: gravity torque;
- $f(\dot{q})$: friction and other losses;
- τ: actuator torque;

- $J(q)^T F_{ext}$: external/contact force mapped into joint torques.

You do not need to symbolically solve this equation to train PPO. You do need to understand that mass, center of mass, inertia, friction, contact, voltage, and delay change how the same command moves the robot.

7.4 Coordinate frames

A vector is incomplete without a frame. "Velocity [1, 0, 0]" could mean world east or robot-forward.

Common frames are:

- **world frame**: fixed to the environment;
- **base/body frame**: moves with the trunk;
- **sensor frame**: aligned with an IMU or camera mounting; and
- **joint frame**: defined by the robot model's joint axis.

A rotation matrix R_{WB} can map a body-frame vector into world coordinates:

$$v_W = R_{WB} v_B.$$

A reversed transform, wrong axis sign, or degrees/radians mismatch can look like a failed policy even if the network is correct.

7.5 Orientation representations

Common orientation representations include:

- roll-pitch-yaw/Euler angles: intuitive but singular at some orientations;
- rotation matrices: 9 numbers with orthogonality constraints;
- quaternions: 4 numbers with unit-length constraint; and
- projected gravity: gravity direction expressed in the body frame.

Projected gravity is useful for locomotion because it tells the policy which way is "down" without exposing a fragile Euler-angle parameterization. A quaternion and its negation represent the same physical rotation, which must be considered in errors and interpolation.

7.6 Contacts make robot dynamics hybrid

A walking robot alternates between continuous flight/swing motion and discrete contact events. Impact can abruptly change velocity. Contact includes normal force, friction, compliance, collision geometry, and numerical solver choices.

This is a **hybrid system**: continuous dynamics plus mode switches. Small differences in foot geometry or timing can produce large rollout differences. That is one reason long sim and real trajectories need not be bit-identical even when the control interface matches.

Coulomb friction intuition

A simple contact model limits tangential friction force:

$$|F_t| \le \mu F_n,$$

where F_n is normal force and μ is a friction coefficient. If requested tangential force exceeds the limit, the foot slips. Real tires/feet also show compliance, velocity dependence, surface variation, and wear.

7.7 Actuators are not ideal commands

An **actuator** converts electrical energy into mechanical motion or force. Common robot actuators include DC/BLDC motors with transmissions and integrated servos.

An ideal simulator position actuator instantly producing any required torque is physically impossible. Real behavior depends on:

- voltage and battery sag;
- motor torque/current limits;
- speed-torque relationship;
- gear ratio and efficiency;
- friction, backlash, and compliance;
- embedded-controller gains and rate;
- command/communication latency; and
- thermal protection.

BAM in the Microduck project

BAM means **Better Actuator Models**, the actuator-modeling approach used by the project for Dynamixel XL330 servos. Here it refers to a voltage-aware servo model with its own friction behavior, not a generic RL algorithm.

That distinction has a concrete consequence: randomizing MuJoCo's ordinary joint `dof_frictionloss` does not randomize the effective friction authority when BAM computes it inside the actuator. The project scales BAM's `friction_scale` instead.

7.8 Feedback control and PD control

A feedback controller compares a target to a measurement. A proportional- derivative (PD) joint controller is

$$\tau = K_p(q_{target} - q) - K_d\dot{q}.$$

- proportional term pushes against position error;
- derivative term damps motion;
- K_p and K_d are gains.

High gains can track tightly but amplify noise, excite unmodeled dynamics, or hit current limits. Low gains are compliant but may need target overshoot to produce enough torque. Thus a policy's position target can intentionally lie beyond the actual desired joint position while the physical joint remains within limits.

7.9 Choosing the policy action

An RL action need not equal motor current. Common choices are:

Action	Advantage	Risk/requirement
torque/current	direct dynamic authority	hard transfer and safety; needs fast loop
joint velocity target	natural for wheels	requires matched velocity controller
absolute joint position	interpretable	can jump if not rate-limited
position delta from HOME	centered, normalized	target scale and HOME must match
residual on nominal controller	bounded learning scope	nominal/residual interaction
task-space target	meaningful geometry	needs IK/controller underneath

Microduck uses 14 normalized actions that become joint-position targets around the default pose. The lower-level actuator dynamics turn targets into motion.

7.10 Nested control loops

A safe robot usually has several rates and responsibilities:

```
high-level planner / behavior selection        ~10 Hz
            |
local navigation / learned skill command      ~50 Hz
            |
RL locomotion policy                          ~50200 Hz
            |
joint position/velocity control             ~1001000 Hz
            |
current/field-oriented motor control          ~40 kHz
            |
mechanics and contacts                      continuous
```

These are illustrative ranges, not universal specifications. Each robot must measure timing and stability requirements.

The fastest loop should not depend on a cloud round trip. A network outage must not remove motor current limits, watchdogs, balance protection, or emergency stop.

7.11 Sampling rate, latency, and delay

A 50 Hz policy period is

$$T = \frac{1}{50} = 0.02 \text{ s} = 20 \text{ ms}.$$

If physics simulates at 5 ms and the action is held for four physics steps, the policy sees a new observation every 20 ms. This hold count is called **decimation** in many robotics simulators.

Latency is time between measurement/decision and effect. **Jitter** is variation in that latency. A constant 10 ms delay and a delay varying from 0–20 ms can affect stability differently.

Deployment must define:

- when sensors are sampled;
- whether measurements have different timestamps;
- inference worst-case execution time (WCET), not only average time;
- when commands reach actuators; and
- what happens on deadline miss.

7.12 State estimation

The policy rarely observes the true state. A **state estimator** combines sensor readings and a process model to estimate quantities such as orientation or velocity.

Typical inputs:

- encoders: joint/motor position and sometimes velocity;
- IMU gyroscope: angular velocity;
- IMU accelerometer: specific force, including gravity effects;
- contact/force sensors;
- cameras, depth sensors, or LiDAR; and
- motor current/voltage/temperature.

An estimator may use complementary filtering, a Kalman-filter family, optimization, or a learned model. Every estimate has bandwidth, bias, noise, delay, and frame conventions.

Observation is not state

An observation o_t is what the policy receives. Simulator state s_t may also contain exact ground-truth velocity, contact forces, terrain parameters, or future commands unavailable on hardware.

Using richer information for a training-only critic is called **asymmetric actor-critic** or **privileged critic** training. It is valid only if the actor input remains deployable.

7.13 System identification before randomization

System identification estimates model parameters from measured input-output data. For an actuator:

1. command a safe excitation;
2. log target, encoder position/velocity, current, voltage, and timestamps;
3. fit delay, gain, friction, damping, or other parameters;
4. validate on a held-out motion; and
5. randomize around the residual uncertainty.

Randomization is not a substitute for calibration. A wildly wrong nominal model plus a huge randomization range can teach unnecessarily conservative or unphysical behavior.

7.14 Safety is a separate objective and mechanism

A reward penalty is a preference in expectation. It is not a hard guarantee. Hardware safety needs independent mechanisms:

- mechanical stops and suitable structure;
- current, voltage, speed, position, and temperature limits;
- watchdog and command timeout;
- communication validity and sequence checks;
- bounded policy outputs and rate limits;
- fall/tilt detection;
- physical emergency stop;
- tether/test stand during early trials; and
- a known rollback controller/artifact.

Constrained RL can represent expected costs separately from reward. For example, Constrained Policy Optimization optimizes return under expected constraints. Its theoretical/empirical properties do not replace electrical or realtime interlocks.

7.15 Exercises

1. A policy uses 5 ms physics and decimation 4. Compute its rate.
2. Explain each term in the rigid-body equation in one sentence.
3. Why might a low-K_p servo need a command outside the desired physical angle, and why must the real joint still have a safety limit?
4. Choose an action representation for a wheel and for a servo-height joint.
5. Draw the frames for a body IMU mounted rotated 90° about yaw. Which transform must the observation pipeline apply?
6. Give one constant bias, one random noise source, and one latency source on a real robot.
7. Explain why a reward penalty cannot serve as an emergency stop.
8. Design a system-identification trial that is informative but safe.

Continue with the Microduck software and control architecture, where these concepts become concrete interfaces.

7.16 Folded solutions

Show answers to Section 7.15

1. The policy period is 5 ms × 4 = 20 ms, so its rate is 1/0.020 = 50 Hz.
2. In $M(q)\ddot{q} + C(q, \dot{q})\dot{q} + g(q) = \tau + J^T f_{ext}$: M maps joint acceleration to inertial torque; $C\dot{q}$ contains velocity-dependent Coriolis/centrifugal effects; g is gravity torque; τ is commanded or actuator torque; and $J^T f_{ext}$ maps external/contact forces into joint torque.
3. With low proportional gain, static load may leave a position error; a target beyond the desired angle can create enough torque to hold the actual angle. The physical joint still needs independent limits because target overshoot can otherwise request collision, overload, or hard-stop impact.
4. A wheel naturally accepts bounded velocity or torque/current target under a lower wheel loop. A height servo naturally accepts a bounded position target with rate/limit enforcement. The exact choice follows measured actuator and controller semantics.
5. Define sensor frame S and body frame B. For a sensor mounted +90° about body yaw, apply the calibrated rotation R_{BS} to every sensor-frame vector before constructing body-frame observations. Test known gravity and angular- rate directions; do not repair a fixed mounting transform with randomization.
6. Constant bias: gyro zero offset. Random noise: encoder quantization or IMU measurement noise. Latency: filtering plus bus/queue/inference delay.
7. A reward penalty changes expected optimization preference. It can be traded against positive reward, has approximation error, and acts only when the policy runs. An E-stop must independently and deterministically remove or bound actuator authority.
8. Clamp the robot in a fixture, begin at low voltage/current, and command a small multisine or step sequence inside accepted travel. Log command, encoder, current, voltage, load, and synchronized timestamps. Fit delay, gain, damping/friction on one sequence; validate on a different sequence; stop on current, temperature, position, velocity, or watchdog limits.

8. Software and Control Architecture

An RL experiment is not just a neural network. It is a chain of models, interfaces, clocks, and artifacts. A policy can be mathematically correct and still fail if one link changes between training and deployment.

8.1 The project stack

```
Onshape/CAD and measured robot facts
        |
        v
MuJoCo XML model (MJCF) + collision models + HOME pose
        |
        v
MuJoCo physics through MuJoCo Warp on the GPU
        |
        +---- BAM XL330 actuator model
        +---- sensors, contacts, delays, noise, domain randomization
        |
        v
mjlab manager-based RL environment
        |
        v
RSL-RL PPO actor/critic and rollout storage
        |
        +---- checkpoints and metrics
        v
ONNX export with observation normalizer
        |
        v
CPU runtime at 50 Hz on the real robot
```

The major dependencies have distinct jobs:

Component	Responsibility
MuJoCo	rigid-body dynamics, joints, contacts, sensors
MuJoCo Warp / Warp	CUDA/GPU-parallel simulation of many environments
`mjlab`	environment managers, task registry, vectorized wrapper, viewer, export support
RSL-RL	PPO, neural networks, rollout buffers, checkpoints
BAM (Better Actuator Models)	voltage-aware Dynamixel XL330 actuator and friction behavior
this repository	Microduck robot models, tasks, rewards, randomization, tests, export and rehearsal
Microduck runtime repository	real sensors, command sources, ONNX inference, motor communication

MJCF is MuJoCo's XML-based robot/model description format. CUDA is NVIDIA's GPU-computing platform. ONNX (Open Neural Network Exchange) is a portable neural-network file format. **Inference** means running a frozen trained network to obtain an output; unlike training, it does not update weights.

8.2 Repository map as a learning map

```
src/mjlab_microduck/├──
 robot/│
   ├── microduck_constants.py       robot choice, HOME pose, actuator setup│
   └── microduck/                   MJCF, scenes, meshes, backlash generator├──
 actuator/│
   └── friction_dr_bam.py           actuator friction DR and backlash feedback├──
 tasks/│
   ├── __init__.py                  task IDs and runner registration│
   ├── mdp.py                       custom rewards, events, observations, curricula│
   ├── symmetry.py                  optional 61D mirror transform│
   ├── backlash.py                  base-task to backlash-task wrapper│
   └── microduck_*_env_cfg.py       task-family configuration factories└──
 train_cli.py                      local/Hugging Face training entry point

scripts/├──
 export.py                         checkpoint -> normalized ONNX├──
 infer_policy.py                   CPU MuJoCo deployment rehearsal└──
 testbench_sim2real.py             measured actuator comparison tools

tests/                               CPU regression and configuration tests
logs/rsl_rl/<experiment>/<run>/      checkpoints, parameters, metrics, videos
```

Read in dependency order. Start with the task registration, then its configuration factory, then follow only the custom functions it references into `mdp.py`. Reading all of the large MDP module from top to bottom is not a good beginner exercise.

8.3 Task registration

Every user-facing task ID binds four things:

```
# Abridged from tasks/__init__.py
register_mjlab_task(
    task_id="Mjlab-Velocity-Flat-MicroDuck",
    env_cfg=make_microduck_velocity_env_cfg(),
    play_env_cfg=make_microduck_velocity_env_cfg(play=True),
    rl_cfg=MicroduckRlCfg,
    runner_cls=MicroduckOnPolicyRunner,
)
```

- `env_cfg` is the training environment.
- `play_env_cfg` makes one policy easier to inspect, often with fewer or more visible disturbances.
- `rl_cfg` defines the networks and PPO hyperparameters.
- `runner_cls` connects the environment to RSL-RL.

`uv run list-envs` reads this live registry. Documentation can become stale; the registry is executable truth.

8.4 Manager-based environment anatomy

`mjlab` divides environment behavior into managers:

Manager/config section	Question it answers
`scene`	What robot, terrain, sensors, and spacing exist?
`actions`	How do policy outputs become simulator controls?
`observations`	What numeric information reaches actor and critic?
`commands`	What behavior is requested this episode?
`events`	What changes at startup, reset, or intervals?
`rewards`	What outcomes produce learning signal?
`terminations`	When is an episode over?

Manager/config section	Question it answers
`curriculum`	How does difficulty or a parameter change with training steps?
`metrics`	What diagnostic values are logged without changing learning?

The velocity factory starts from `mjlab`'s closest locomotion template and modifies it:

```
def make_microduck_velocity_env_cfg(play=False, rough=False):
    cfg = make_velocity_env_cfg()
    cfg.scene.entities = {"robot": MICRODUCK_WALK_ROBOT_CFG}
    cfg.scene.sensors = (
        feet_ground_cfg,
        self_collision_cfg,
        foot_height_scan_cfg,
    )
    # Then specialize actions, observations, commands, rewards, DR, and terrain.
    return cfg
```

Building on a maintained template retains important sensor and safety wiring. A standalone task must recreate that wiring deliberately.

8.5 One 20 ms policy step

The physics timestep is 5 ms and action **decimation** is 4, meaning one policy decision is reused for four smaller physics steps. One policy action is therefore held across four physics steps:

```
t = 0 ms     assemble observation; run actor; set action target
t = 5 ms     physics + actuator substep
t = 10 ms    physics + actuator substep
t = 15 ms    physics + actuator substep
t = 20 ms    physics + actuator substep; compute next observation/reward
```

This gives a 50 Hz policy rate while resolving contacts and actuator dynamics at 200 Hz. Changing either clock changes the control problem and must be matched in deployment.

8.6 Observation contract

The actor layout is intentionally fixed across the policy family:

```
[0:3]    base angular velocity
[3:6]    projected gravity
[6:20]   14 relative joint positions
[20:34]  14 joint velocities
[34:48]  14 previous actions
[48:51]  twist: vx, vy, yaw rate
[51:55]  head pose: four joint deltas
[55:61]  body pose: x, y, z, roll, pitch, yaw deltas
```

Unused command slots remain present and are zero-padded or sampled over tiny ranges. Deleting a term would change the input size and prevent runtime policy hot-swapping.

The actor excludes true base linear velocity because the real robot does not have a direct perfect measurement. The critic includes it because privileged information can improve value estimates during simulation training.

Observation corruption models hardware imperfections. The broader practice of sampling physical parameters and disturbances is called **domain randomization**: training across a range of simulated domains so the real robot is less likely to fall outside the learned experience.

- small additive sensor noise;
- encoder bias;

- IMU mounting variation;
- delayed angular velocity, gravity, joint velocity, and actuation;
- backlash-specific encoder views.

If the actor sees a biased or backlash-adjusted quantity, a tracking reward on that quantity must use the same view. Otherwise the policy is punished for correcting the measurement it receives.

8.7 Action contract

The actor emits 14 floating-point values. The joint-position action uses `scale=1.0` and the model's default joint pose as its offset. Conceptually:

$$q^{target} = q^{HOME} + a$$

The exact action manager also applies its configured ordering and any limits. The target then enters the BAM actuator model; it is not an ideal instantaneous joint angle.

The 14 servo order is:

```
0..4    left hip yaw, hip roll, hip pitch, knee, ankle
5..8    neck pitch, head pitch, head yaw, head roll
9..13   right hip yaw, hip roll, hip pitch, knee, ankle
```

Roller and backlash MJCF models contain interleaved passive joints. Custom MDP code must use `_servo_joint_ids` and `_servo_joint_pos` rather than assuming that MuJoCo joint index equals servo index. All unactuated joints begin with `passive_`, and actuator/observation selectors exclude that prefix.

8.8 The actuator is part of the environment

The real XL330 is not an ideal position source. The BAM configuration includes firmware position-loop behavior, motor electrical effects, torque limits, friction, battery voltage, load-dependent voltage sag, and command delay.

The main configuration randomizes values such as:

```
FrictionDRBamActuatorCfg(
    motor_name="xl330",
    model="m6",
    target_names_expr=(r"^(?!passive_).*",),
    kp_fw=200.0,
    vin_range=(6.5, 8.2),
    vin_drop_gain_range=(0.0, 0.2),
    vin_min=6.0,
    delay_min_lag=3,
    delay_max_lag=6,
)
```

This is a real project example of model-based sim-to-real engineering: the policy learns to control a family of plausible actuators rather than a perfect one. Under BAM, MuJoCo's ordinary `dof_frictionloss` is not the friction authority; friction randomization must scale the actuator's own field.

8.9 Three robot models, one policy interface

Model	Why it exists
`robot_walk.xml`	light walking model; trunk/head ground contacts are stripped
`robot_allcollisions.xml`	lying, stand-up, tricks, and body contacts
`robot_allcollisions_rollers.xml`	full body plus passive wheel joints

Backlash versions add an unactuated hinge in series with every servo while keeping actor and action dimensions unchanged. A base and backlash A/B test must use otherwise matching robot models or the comparison is confounded.

8.10 Training artifacts and provenance

A run directory normally contains:

```
params/env.yaml       resolved environment used for the run
params/agent.yaml     resolved PPO/network configuration
model_<n>.pt          actor, critic, normalizers, optimizer, counters
events.out...         local scalar logs
*.onnx                exported inference policy, when exported
videos/play/          finite recorded rollouts, when requested
```

The resolved YAML is valuable. Source code may change after a run; the saved parameters show what the checkpoint actually trained against. Keep it with any policy you evaluate or deploy.

8.11 Lab: trace a task without training

1. List the registry:

   ```
   uv run list-envs
   ```

2. Find the registration of `Mjlab-Velocity-Flat-MicroDuck` in `src/mjlab_microduck/tasks/__init__.py`.
3. Open `make_microduck_velocity_env_cfg` and identify one scene sensor, one command, one reward, one termination, and one curriculum.
4. Find the actor and critic hidden layers in `MicroduckRlCfg`.
5. Explain which of those objects exists during real ONNX inference.

Continue with Microduck setup and the first experiment.

8.12 Folded lab solution

Show a reference trace for Section 8.11

1. `uv run list-envs` proves registration through the installed project rather than a filename guess. Preserve the exact task ID it prints.
2. `tasks/__init__.py` registers `Mjlab-Velocity-Flat-MicroDuck` with `make_microduck_velocity_env_cfg()`, a play configuration, `MicroduckRlCfg`, and `MicroduckOnPolicyRunner`.
3. Valid examples include the `foot_height_scan` terrain ray sensor; `twist` command; `track_linear_velocity` reward; `nan_state` termination; and `action_rate_weight` curriculum. The important result is the path from factory to a named manager term, not memorizing these examples.
4. Actor and critic both use hidden dimensions `(512, 256, 128)` with ELU activation and observation normalization. Their inputs differ because the critic may receive privileged observations.
5. Real ONNX inference deploys the actor and its actor-observation normalizer. The critic, reward/termination/event/curriculum managers, rollout storage, PPO losses, optimizer, and simulator randomization exist only to generate or improve the checkpoint.

A repeatable source trace is:

```
rg -n 'Mjlab-Velocity-Flat-MicroDuck' src/mjlab_microduck/tasks/__init__.py
rg -n 'MicroduckRlCfg|hidden_dims|obs_normalization' \
  src/mjlab_microduck/tasks/microduck_velocity_env_cfg.py
```

9. Setup and First Experiment

This chapter builds confidence in layers. Do not begin with a 20-hour training run. First prove the checkout, dependency lock, task registry, CPU tests, GPU simulation, short PPO loop, checkpoint load, and viewer.

9.1 Requirements

For local GPU training you need:

- Linux;
- a CUDA-capable NVIDIA GPU and working driver;
- enough disk space for Python/CUDA packages and checkpoints;
- Git; and
- `uv` for the locked Python 3.12 environment.

The project requires Python `>=3.12,<3.13`; let `uv` honor the repository configuration rather than installing packages into an unrelated system Python.

Training can also be submitted to Hugging Face Jobs. Playback and deployment rehearsal still need a local environment that can load the model.

9.2 Clone and synchronize

```
git clone https://github.com/pollen-robotics/microduck_rl
cd microduck_rl
uv sync
```

`uv sync` is the reproducibility test. A package that exists only because it was manually installed into a developer's environment will be absent on a clean machine or remote job.

On Linux AArch64, set a longer first-download timeout if needed:

```
export UV_HTTP_TIMEOUT=600
uv sync
```

The project pins Torch directly and selects the CUDA 12.9 wheel index on AArch64. Do not remove the apparently redundant direct pin; `uv` source routing depends on it.

9.3 Verify the GPU stack

```
nvidia-smi

uv run python - <<'PY'
import torch
import warp as wp

print("torch:", torch.__version__)
print("torch CUDA runtime:", torch.version.cuda)
print("CUDA available:", torch.cuda.is_available())
print("GPU count:", torch.cuda.device_count())
```

```
if torch.cuda.is_available():
    print("GPU 0:", torch.cuda.get_device_name(0))
wp.init()
PY
```

Do not proceed to a long run if `torch.cuda.is_available()` is false. A visible GPU in `nvidia-smi` is necessary but not sufficient; the Python wheel must also include compatible CUDA support.

9.4 Discover the live tasks

```
uv run list-envs
```

Look for `Mjlab-Velocity-Flat-MicroDuck`. If it is missing, task package entry points were not installed or importing `mjlab_microduck.tasks` failed. Fix that before debugging PPO.

9.5 Run the CPU regression suite

```
uv run --with pytest pytest tests/
```

These tests are more than unit-test decoration. They protect properties such as:

- the 61D observation family contract;
- servo indices on roller/backlash models;
- reward sign conventions;
- NaN handling;
- reset and curriculum wiring; and
- task registration.

A passing suite does not prove a good policy, but a failing invariant makes an expensive run untrustworthy.

9.6 Run the five-iteration smoke train

```
uv run train Mjlab-Velocity-Flat-MicroDuck \
    --env.scene.num-envs 64 \
    --agent.max_iterations 5
```

What this should prove:

1. the robot and sensors compile;
2. Warp can execute on the selected GPU;
3. actor observation shape is 61 and action shape is 14;
4. every reward and termination can run;
5. PPO can collect data, compute losses, and update weights;
6. a checkpoint and resolved parameters can be written; and
7. values remain finite for this short run.

What it does **not** prove: that the robot has learned to walk. Five iterations are a software smoke test.

The default logger uses Weights & Biases. Authenticate with `wandb login` when you want cloud tracking, or use W&B's offline mode for a local-only experiment:

```
WANDB_MODE=offline uv run train Mjlab-Velocity-Flat-MicroDuck \
    --env.scene.num-envs 64 \
    --agent.max_iterations 5
```

9.7 Inspect the run artifacts

```
find logs/rsl_rl/velocity -maxdepth 3 -type f | sort | tail -40
```

Open the newest run directory and inspect:

```
sed -n '1,160p' logs/rsl_rl/velocity/<run>/params/agent.yaml
sed -n '1,220p' logs/rsl_rl/velocity/<run>/params/env.yaml
```

Questions to answer:

- How many environments actually ran?
- How many iterations were requested?
- Is actor normalization on?
- What is the simulator timestep and action decimation?
- Which robot spec function was resolved?

This habit prevents evaluating a checkpoint under a task you merely assumed it used.

9.8 Play a local checkpoint

```
uv run play Mjlab-Velocity-Flat-MicroDuck \
    --checkpoint-file logs/rsl_rl/velocity/<run>/model_<iteration>.pt \
    --num-envs 1 \
    --viewer native
```

Or load a run from W&B:

```
uv run play Mjlab-Velocity-Flat-MicroDuck \
    --wandb-run-path <entity>/mjlab_microduck/<run_id> \
    --num-envs 1
```

The viewer resamples training-style commands, so the robot may change direction. The command arrow is the requested local velocity, not an obstacle sensor or global route.

Verify these startup facts in the terminal:

```
checkpoint name is the one you intended
environment device is cuda:0 (or the explicitly selected device)
actor observation shape is (61,)
action shape is 14
actor first layer has 61 inputs
```

An actuator-selector warning that says 14 joints were matched while similarly named sites also match is informational in the current stack; the resolved action dimension must still be 14.

9.9 Record a finite rollout

A video is durable evidence and exits after the requested number of policy steps:

```
uv run play Mjlab-Velocity-Flat-MicroDuck \
    --checkpoint-file logs/rsl_rl/velocity/<run>/model_<iteration>.pt \
    --num-envs 1 \
    --video True \
    --video-length 500
```

At 50 Hz, 500 policy steps represent 10 seconds of simulated behavior. The video is written under the checkpoint run's `videos/play/` directory.

When comparing checkpoints, keep task ID, commands, random seed or evaluation battery, camera, and video length consistent.

9.10 Start a full run only after the smoke test

```
uv run train Mjlab-Velocity-Flat-MicroDuck \
    --env.scene.num-envs 4096
```

Environment count is a throughput/memory tradeoff. Do not assume 4,096 fits every GPU or task. Reduce it if compilation or memory fails; do not change the task merely to hide a resource problem.

Remote alternative:

```
uv run train Mjlab-Velocity-Flat-MicroDuck \
    --env.scene.num-envs 4096 \
    --hf-jobs
```

See the Microduck repository's `scripts/hf/README.md` for authentication, job flavor, namespace, and retrieval details.

9.11 Resume rather than discard a useful run

```
uv run train Mjlab-Velocity-Flat-MicroDuck \
    --env.scene.num-envs 4096 \
    --agent.run-name resume \
    --agent.load-checkpoint model_29999.pt \
    --agent.resume True
```

Resume only when the environment and observation/action contract are compatible with the checkpoint. A checkpoint is not a generic starting point for an arbitrary task.

9.12 Common first-run failures

GPU visible to the driver but not Torch

Symptom: `nvidia-smi` works, while Torch reports zero devices or `mjlab` fails while selecting a GPU. Inspect the installed Torch build and CUDA runtime. On AArch64, preserve the repository's direct Torch pin and CUDA index routing.

Viewer does not open

Check `DISPLAY` on X11 or use a finite video/remote viewer appropriate to the machine. Do not interpret a display failure as a policy failure; first confirm the simulation process and checkpoint load.

Observation-size mismatch

The current family expects 61 actor inputs. A 51D legacy ONNX policy or a task that deleted command slots is incompatible. Use the correct checkpoint/task or intentionally version the runtime contract.

NaN during training

Record the task, iteration, offending term, and resolved configuration. Run the CPU NaN regression tests, reduce the case to a reset/step reproduction, and inspect contacts and sensors. Simply restarting an unchanged long run loses the most useful evidence.

A successful smoke policy falls immediately

Expected. Five iterations proved the pipeline, not locomotion.

9.13 Lab report template

Record this after the first experiment:

```
commit:
task ID:
GPU and Torch/CUDA versions:
command:
run directory:
checkpoint:
actor/action shapes:
tests result:
smoke result:
viewer or video result:
first anomaly observed:
```

This small report is the beginning of reproducible RL work.

Continue with the anatomy of a Microduck environment.

9.14 Folded example report

Show an annotated first-experiment report

This is a format example, not evidence from your machine. Replace every value and retain the commands/logs that support it.

```
commit:                 FULL_GIT_SHA (plus dirty patch, if any)
task ID:                Mjlab-Velocity-Flat-MicroDuck
GPU and Torch/CUDA:     exact device / torch build / CUDA runtime
command:                uv run train ... --num-envs 64 --max-iterations 5
run directory:          logs/<experiment>/<timestamp-or-run>
checkpoint:             exact model_*.pt and SHA-256
actor/action shapes:    61 / 14, confirmed from runtime output
tests result:            N passed, M skipped; paste command and date
smoke result:            completed 5 iterations; finite observations/rewards
viewer or video result:  pipeline rendered; locomotion quality not claimed
first anomaly observed:  exact warning, metric, frame, or "none observed"
```

The important distinctions are observed versus assumed, pipeline success versus learned-skill success, and a mutable filename versus a content hash.

10. Anatomy of a Microduck Environment

An environment is an executable task specification. It defines what the robot can observe and do, how the world changes, what counts as progress, and which experience PPO collects.

This chapter uses `Mjlab-Velocity-Flat-MicroDuck` as the main worked example.

10.1 Start from the factory

The task registry points to:

```
make_microduck_velocity_env_cfg(play=False, rough=False)
```

The factory begins with `mjlab`'s velocity template and then specializes it for Microduck. This is an important design pattern:

```
cfg = make_velocity_env_cfg()
cfg.scene.entities = {"robot": MICRODUCK_WALK_ROBOT_CFG}
cfg.scene.sensors = (
    feet_ground_cfg,
    self_collision_cfg,
    foot_height_scan_cfg,
)
```

The resulting object is still just configuration. Managers copy and resolve it when an environment instance is constructed.

10.2 Scene: the physical question

The scene selects:

- the robot MJCF and its initial state;
- terrain;
- collision rules;
- contact and ray sensors;
- environment count and spacing; and
- viewer camera.

Flat velocity uses a plane. Rough velocity uses a generator with robot-scale terrain; step heights are capped around 1.5 cm because carrying human-scale terrain assumptions to a 25 cm robot would create a different task.

The walking model has deliberately simplified fall contacts. Tasks that begin or finish on the ground use the all-collisions model. Model choice is part of the task definition, not a rendering preference.

Try it: compare resolved scenes

```
uv run play Mjlab-Velocity-Flat-MicroDuck --agent zero --num-envs 1
uv run play Mjlab-Velocity-Rough-MicroDuck --agent zero --num-envs 1
```

A zero agent is useful for inspecting physics and initial conditions without attributing motion to a policy.

10.3 Actions: the controllable question

The velocity task has one action term with dimension 14:

```
joint_pos_action = cfg.actions["joint_pos"]
joint_pos_action.scale = 1.0
```

The action manager interprets each network output as an offset from the default joint pose. BAM then turns the position target into voltage/torque behavior.

Action design choices include:

- position, velocity, torque, or residual targets;
- scale and offset;
- clipping;
- ordering; and
- delay/filter behavior.

An action space should match what the runtime can reproduce. Training with an action low-pass filter and deploying without it creates a different plant; deploying a filter that training never saw does the same. This policy family is intentionally unfiltered.

10.4 Observations: the information question

An observation term is a function plus optional noise, scale, clipping, delay, and history. Actor terms are concatenated in order.

The main actor gets 48 proprioceptive values plus a 13D command block. The critic receives additional simulator-only signals. Two principles matter:

1. **Availability:** actor inputs must be reproducible on hardware.
2. **Meaning:** training and runtime must use the same units, frame, sign, ordering, offset, delay, and filtering.

For example, “angular velocity” is incomplete documentation. You must know:

```
world or body frame?
rad/s or deg/s?
which axis signs?
raw, calibrated, or filtered?
current sample or delayed sample?
```

The current actor intentionally removes perfect base linear velocity and the general terrain height scan. A per-foot ray sensor supports foot-height features and rewards, but there is no forward obstacle representation in the actor.

10.5 Commands: the intention question

Commands are targets sampled by the training environment and supplied by an operator or planner at deployment.

The velocity task’s twist command uses these ranges:

```
forward velocity vx       -0.4 .. +0.4 m/s
lateral velocity vy       -0.3 .. +0.3 m/s
yaw rate                  -1.0 .. +1.0 rad/s
resampling interval        3.0 .. 8.0 s
```

It also creates explicit buckets:

- standing/zero-command environments, because continuous uniform sampling almost never produces exactly zero;

- forward-command environments; and
- turn-in-place environments with zero linear velocity and a meaningful yaw command.

The turn-in-place code is a useful real example of fixing the data distribution rather than tuning PPO:

```
# Abridged from VelocityCommandCommandOnly._resample_command
turn_ids = env_ids[random_values < rel_turn_in_place_envs]
self.vel_command_b[turn_ids, 0:2] = 0.0
sign = where(random_values < 0.5, -1.0, 1.0)
mag = uniform(0.4 * max_yaw_rate, max_yaw_rate)
self.vel_command_b[turn_ids, 2] = sign * mag
```

Independent uniform sampling made useful spin-in-place examples too rare. An explicit 15% bucket gives PPO enough on-policy data.

The head command is four joint deltas from HOME. Its range widens by curriculum. The six-dimensional body-pose slot is retained, but its tracking reward has weight zero in the main velocity policy; do not expect this checkpoint to obey body-pose commands reliably.

10.6 Rewards: the objective question

The main velocity reward stack includes:

Term	Purpose	Typical configured weight
linear velocity tracking	follow requested planar velocity	+2.0
angular velocity tracking	follow requested turn rate	+2.0
upright	keep trunk orientation useful	+2.0
pose	retain a workable leg posture	+1.0
air time	encourage stepping when commanded	+3.0
head-pose tracking	follow four head targets	+2.0
body angular velocity	discourage unnecessary rotation	-0.05
angular momentum	reduce excess whole-body motion	-0.02
action rate	reduce command jitter	starts at -0.1, ramps stronger
foot slip	reduce sliding without blocking turning	-0.1
self-collision	avoid invalid body contact	-1.0

Weights cannot be compared in isolation. A term's scale, frequency, gate, and the total positive reward mass determine its influence.

Real code example: Gaussian head tracking

The custom reward compares the commanded head delta with measured joint delta:

```
# Abridged from mdp.head_pose_tracking
cmd = env.command_manager.get_command("head_pose")       # (N, 4)
measured = joint_pos[:, neck_ids] + backlash_position
actual = measured - default_joint_pos[:, neck_ids]
error = actual - cmd
per_joint = torch.exp(-((error / std) ** 2))
return per_joint.mean(dim=-1)                            # (N,)
```

At `abs(error) == std`, one joint contributes $e^{-1} \approx 0.37$. A very wide standard deviation weakens the gradient near small errors; a very narrow one can make far states nearly zero-reward and can tax unavoidable walking oscillation.

The implementation reads through simulated backlash so the tracking reward and encoder observation describe the same physical angle.

Reward sign audit

There are two penalty styles in this project:

```
# Style A: function returns nonnegative cost
cost = error.square()
# Configure a NEGATIVE weight.

# Style B: helper returns a nonpositive penalty
penalty = -error.abs()
# Configure a POSITIVE weight.
```

A negative weight on Style B makes violations profitable. The reliable runtime check is:

```
every Episode_Reward/<penalty> contribution must be <= 0
```

Reward gates

A gate defines when a reward is meaningful. Foot air time should activate when walking is commanded, not while standing. A recovery reward should pay for progress toward upright rather than pay continuously for remaining fallen.

Hard state-based gates are often more effective than small penalties because they define what counts as the maneuver. For a roll, support contact and orientation axes can distinguish a real forward roll from a shoulder flop.

10.7 Events and domain randomization: the variation question

Event terms run in modes:

Mode	Example
startup	foot friction, encoder bias fields, mass/inertia setup
reset	root pose, joints, CoM, actuator friction scale, armature
interval	velocity pushes every few seconds

Domain randomization (DR) trains one policy over a distribution of plausible robots:

$$\theta_{physics} \sim p(\theta)$$

Microduck randomizes selected quantities such as friction, mass/inertia, CoM, battery voltage/sag, armature, encoder bias, IMU alignment, delays, and pushes.

DR is not a substitute for calibration. Zero-centered IMU orientation variation teaches tolerance to uncertain mounting magnitude; it cannot remove a fixed systematic mounting bias. The runtime must calibrate that bias.

Custom randomization must restore defaults before applying a sampled change. If a +5% mass change is added to the already-randomized mass every reset, the distribution drifts outside the intended range during long training.

10.8 Terminations: the data-recycling question

The velocity task can terminate for timeout, falling, terrain bounds, or a nonfinite state. A NaN guard checks joints, root state, and named sensor data:

```
bad = ~torch.isfinite(data.joint_pos).all(dim=1)
bad |= ~torch.isfinite(data.joint_vel).all(dim=1)
bad |= ~torch.isfinite(data.root_link_pos_w).all(dim=1)
bad |= ~torch.isfinite(data.root_link_quat_w).all(dim=1)
bad |= ~torch.isfinite(data.root_link_lin_vel_w).all(dim=1)
bad |= ~torch.isfinite(data.root_link_ang_vel_w).all(dim=1)
```

Sensor-derived critic terms are separately sanitized so one invalid contact or ray value does not poison the value network before the environment resets.

Do not reuse walking terminations blindly. A recovery task needs time on the ground. An episodic trick may need a short horizon and a landing criterion.

10.9 Curricula: the pacing question

A curriculum changes the problem as training progresses. In this repository, steps mean environment steps:

```
curriculum step = PPO iteration * num_steps_per_env
                = PPO iteration * 24
```

The action-rate curriculum uses discrete stages:

```
weight_stages = [
    {"step": 0,         "weight": -0.1},
    {"step": 500 * 24,  "weight": -0.2},
    {"step": 750 * 24,  "weight": -0.4},
    {"step": 1000 * 24, "weight": -0.6},
    {"step": 1250 * 24, "weight": -0.8},
    {"step": 1500 * 24, "weight": -1.0},
]
```

The helper mutates the live manager copy:

```
term_cfg = env.reward_manager.get_term_cfg(reward_name)
term_cfg.weight = selected_stage["weight"]
```

Writing to `env.cfg.rewards` after manager initialization is a silent no-op because managers copied their configurations.

A stage should follow demonstrated competence. If the main skill metric drops exactly at every boundary, the schedule is outpacing the policy.

10.10 What the velocity policy can and cannot do

Capability	In the current task?	Reason
commanded forward/lateral walk	yes	command input and active tracking reward
commanded turn or stand	yes	explicit command buckets and tracking
commanded head pose	yes	4D input and active reward
commanded body pose	not reliably	slot exists, reward weight is zero
global waypoint navigation	no	no waypoint/map state or navigation objective
obstacle avoidance	no	no forward obstacle observation or avoidance objective
visual decision making	no	camera pixels/features are not actor inputs

Putting a box in the rendered scene does not create perception. An actor can only condition behavior on information in its observation or inferable from contact after collision.

Two sound extension architectures are:

1. a perception/planning system detects obstacles and sends safe local twist commands to the unchanged 61D locomotion policy; or

2. a deliberately versioned policy family receives exteroceptive features and is trained on obstacle-rich scenes with a new runtime contract.

The first preserves a small, fast, hot-swappable motor policy. The second may learn tighter perception-action coupling but requires new training data, network design, inference budgets, tests, and deployment plumbing.

10.11 Lab: predict behavior from configuration

Without running training, answer these from `microduck_velocity_env_cfg.py`:

1. What command causes standing?
2. Why is turn-in-place sampled in its own bucket?
3. Which head joints are commandable?
4. Why does `body_pose` remain in the observation when its reward is zero?
5. Which randomization changes the actuator rather than MuJoCo joint friction?
6. At which PPO iteration does the action-rate weight first become `-0.6`?

Then run `uv run play ... --agent zero` and identify which scene facts can be verified without a trained policy.

Continue with training, evaluation, and debugging.

10.12 Folded lab answers

Show answers to the Section 10.11 configuration trace

1. Standing is an **exact zero twist command** selected by the command term's `rel_standing_envs` bucket. Near-zero uniform samples are not the same training case.
2. Independent continuous sampling makes simultaneous near-zero translation and meaningful yaw too rare. `rel_turn_in_place_envs` reserves a deliberate fraction of experience for that behavior.
3. The 4D head command controls `neck_pitch`, `head_pitch`, `head_yaw`, and `head_roll`, in that order.
4. The body slot preserves the shared 61D hot-swap interface and keeps those input weights available to policy-family tasks that train them. In the main velocity recipe the ranges remain small but `body_pose_tracking` has weight zero, so the interface does not prove body-pose skill.
5. `randomize_joint_friction` changes the BAM actuator's per-environment `friction_scale`. Randomizing MuJoCo `dof_frictionloss` would be a silent no-op because BAM owns this friction model.
6. The `-0.6` stage begins at `1000 * NUM_STEPS_PER_ENV`; with 24 rollout steps that is environment step 24,000, or PPO iteration 1,000.

With `--agent zero`, you can verify model loading, gravity, collision/contact geometry, passive-joint behavior, reset poses, actuator plumbing, and whether a zero action is numerically finite. You cannot verify learned command tracking, gait quality, recovery, or sim-to-real robustness because no trained actor is making decisions.

11. Training, Evaluation, and Debugging

Training is not “start PPO and wait.” A useful run is a controlled experiment with a hypothesis, resolved configuration, checkpoints, per-term metrics, and rollouts that test the intended behavior.

11.1 The end-to-end loop

```
physics assumption check
        v
CPU config/reward tests
        v
64-env, 5-iteration smoke train
        v
full vectorized run
        v
metric and checkpoint inspection
        v
fixed evaluation battery + video
        v
one evidence-backed change
        +--------------------------> repeat
```

For a target/rest pose, add a physics-only check before training: hold its control target from noisy initial states for several seconds and measure both height and tilt. A fallen body can have a stable height, so height alone cannot prove equilibrium.

11.2 Before launching a full run

Write a short experiment note:

```
hypothesis:
task ID and commit:
baseline run/checkpoint:
one intentional change:
expected main metric effect:
expected visible behavior:
failure criterion:
training budget:
```

Then run:

```
uv run --with pytest pytest tests/

uv run train <TASK_ID> \
    --env.scene.num-envs 64 \
    --agent.max_iterations 5
```

Do not combine reward redesign, new observations, stronger DR, a new robot model, and new PPO hyperparameters in one experiment. Even a successful result will not tell you which change mattered.

11.3 Read the training dashboard in layers

Layer 1: pipeline health

Check:

- iteration advances;
- simulation and learning FPS are plausible;
- losses and KL are finite;
- no NaN termination spike appears;
- checkpoints continue to save; and
- GPU memory remains stable.

Layer 2: episode behavior

The main velocity episode lasts 20 seconds, or 1,000 policy steps at 50 Hz. A mean episode length near 1,000 suggests most episodes reach timeout. It does not prove walking; standing still may also survive.

A rising episode length can be good for locomotion and irrelevant for a fixed short trick. Interpret it against the task.

Layer 3: reward signs

For every penalty contribution:

```
Episode_Reward/<penalty> <= 0
```

If a penalty is positive, stop and inspect its function/weight sign. PPO will farm a double-negated violation.

Layer 4: the main skill

Track the term that directly represents the task:

```
walking      linear/angular tracking and air time
stand-up     upright/height completion and landing quality
sit-stand    commanded transition completion
spin         commanded yaw-rate profile and grounded support
```

Total reward can rise because action rate fell while locomotion remained absent. Main-task metrics and video must agree.

Layer 5: policy distribution

Watch entropy/noise scale, KL, and action statistics. Early entropy collapse can freeze a do-nothing solution. Persistently large action noise can hide a useful mean policy. These are diagnostic signals; do not tune them before checking the task specification.

11.4 Weighted logs and reward mass

`Episode_Reward/<term>` is the weighted contribution. If a term's weight is zero, its log is zero even when the underlying behavior is poor. If a curriculum changes the weight, a jump in the log may reflect the coefficient, not behavior.

Compare reward mass:

$$\text{mass}_i \approx |w_i| \, \mathbb{E}[|f_i|]$$

Suppose one task has 8 points/step of positive reward and another has 2. An action-rate weight of `-0.1` is four times weaker relative to the first task, even though the text of the configuration is identical.

11.5 Evaluation is a separate experiment

Do not judge only the final checkpoint or one random viewer episode. Create a battery that covers the state distribution you care about.

Example velocity battery:

Case	Command	What to measure
idle	`[0, 0, 0]`	stable stance, head bias, jitter
forward	`[+0.2, 0, 0]`	achieved speed, drift, falls
backward	`[-0.2, 0, 0]`	tracking and foot clearance
lateral	`[0, +0.15, 0]`	side tracking and collisions
turn in place	`[0, 0, +0.6]`	yaw rate, translation drift
combined	`[+0.2, 0, +0.5]`	curve tracking
head command	fixed twist + head deltas	per-joint error and gait effect
disturbance	selected push cases	recovery and settling time

The default `play` command generator is useful for qualitative variety, but a fixed battery is better for checkpoint comparison. The deployment rehearsal in `scripts/infer_policy.py` allows explicit initial velocity commands and keyboard changes.

11.6 Watch video and compute state metrics

Video answers geometric questions metrics can miss:

- Which body or collision geometry touched?
- Did a “forward roll” go over the head or shoulder?
- Did the feet step or slide?
- Is the head tracking target or serving as a counterweight?

State metrics answer questions video can mislead:

- exact trunk tilt and height;
- achieved versus commanded speed;
- peak angular rate and acceleration;
- contact timing and impact force;
- per-spawn success rate; and
- end-state clusters.

Use both. A smooth camera angle can hide lateral drift, and a scalar threshold can split one visibly identical behavior cluster.

11.7 Checkpoint selection

The last checkpoint is not automatically best. A policy can discover the skill and later trade it away when a curriculum adds difficulty or regularization.

Evaluate checkpoints around:

- first visible skill discovery;
- every curriculum boundary;
- sudden main-metric changes;
- entropy or KL events; and
- the final iteration.

Load an exact local checkpoint:

```
uv run play <TASK_ID> \
    --checkpoint-file logs/rsl_rl/<experiment>/<run>/model_2000.pt \
    --num-envs 1
```

Preserve the resolved `params/` directory with the selected model.

11.8 Common failure patterns

The robot does nothing

Likely causes:

- attempt penalties or smoothness costs dominate before skill discovery;
- positive task rewards are too sparse or flat;
- command distribution rarely includes the desired case;
- a tracking standard deviation is too tight at initial errors; or
- standing earns most of the same positive stack as moving.

Measure per-term contributions before increasing entropy or learning rate.

The robot moves violently

Check whether an arrival state pays a per-step jackpot, whether impact/contact is under-specified, and whether action-rate regularization has begun. For a small robot, 3.5–5.5 rad/s body rotation may be physically natural during a tumble; penalize impact and thrash rather than importing human-scale angular speed intuition.

It finds the beginning but not the finish

The successful frontier may almost never appear in on-policy data. Add reverse-curriculum resets near later maneuver states and consolidate each slice before widening.

It camps at a waypoint

Per-step waypoint rewards create local jackpots. For an episodic landing task, prefer a fixed final target with progress/landing shaping and let RL discover the path.

It reaches the goal too fast and crashes

An early-arrival state that keeps paying makes speed profitable. Use a slewed target or rate-limited progress so being ahead of the intended transition does not pay extra.

Joints park at hard limits

The default limit cost may activate only near the last portion of the range. Add a qpos-side limit-proximity penalty for the offending joints rather than shrinking the wide command range needed by low-stiffness servos.

Training becomes NaN

Do not merely lower the learning rate. Determine whether the first nonfinite value originated in physics, a sensor, reward math, observation normalization, or PPO. Inspect `Episode_Termination/nan_state`, contact conditions, and a minimal reset/step reproduction.

Simulation succeeds and hardware fails

Audit the interface before redesigning rewards:

```
joint names/order and signs
HOME offsets
units and coordinate frames
observation order and normalization
sensor delay/noise/filtering
```

```
action scaling and filtering
policy frequency and missed deadlines
actuator voltage, friction, saturation, backlash
command-slot writes
```

In-sim playback applies the checkpoint normalizer and can hide an ONNX export mistake. Runtime all-zero commands can select "stand" when an application expected a trick flag. These interface failures look like bad learning.

11.9 Curriculum diagnosis

Plot main metrics against curriculum stages. If a metric drops at exactly the same step every run, inspect the stage before changing PPO.

Example reasoning:

```
observation: air-time reward rises through iteration 700
event: action-rate cost doubles at iteration 750
result: air-time collapses and never recovers

hypothesis: smoothness tax arrived before stepping consolidated
experiment: delay only that stage, preserve every other setting
```

Curricula should introduce challenge after competence, not according to an arbitrary desire to finish sooner.

11.10 A disciplined reward change

Use this sequence:

1. Name the visible failure precisely: "left hip roll remains within 3 degrees of its hard limit for 70% of forward steps."
2. Confirm it from state data, not one frame.
3. Identify the cheapest exploit in the current objective.
4. Add or change one term/gate.
5. Write a pure-function test and a configuration wiring/sign test.
6. Run the five-iteration smoke test.
7. Compare a fixed checkpoint battery with the baseline.
8. Keep or revert based on the intended behavior, not total reward alone.

11.11 Lab: produce an evidence-backed run review

Choose one trained checkpoint and write:

```
Task and checkpoint:
Resolved configuration path:
Main behavior observed:
Failure rate and evaluation cases:
Main weighted reward terms:
All penalty signs valid? yes/no
Episode length interpretation:
Visible exploit or compromise:
Measured state evidence:
Next single hypothesis:
```

Avoid "it works." Prefer statements such as "tracks +0.2 m/s on 18/20 flat starts, but falls in 6/20 turn-in-place trials after the first direction change." That tells the next engineer what to reproduce.

Continue with export, deployment, and sim-to-real.

11.12 Folded lab rubric

Show a reference evidence-backed review

An acceptable answer names a reproducible case and separates facts from its next hypothesis. For example:

```
Task/checkpoint:       exact task + checkpoint SHA-256
Evaluation battery:   20 fixed flat starts at +0.2 m/s; 20 turn-in-place cases
Observed result:      18/20 forward successes; 14/20 turn successes
Raw failure classes:  4 forward falls after direction reversal; 2 initial slips
Penalty signs:        every weighted penalty <= 0 (attach metric export)
Episode length:       timeouts are success only for continuous walking cases
Visible compromise:  large translation drift while tracking yaw
State evidence:       median/p90 yaw error and x-y drift, synchronized to video
Hypothesis:           turn-in-place samples remain underrepresented
Single next change:   change only its explicit command bucket fraction
```

Those numbers are illustrative. Your report must derive them from the selected checkpoint. A strong answer also includes representative success/failure video, resolved configuration, seed list, evaluator commit, and the result that would falsify the proposed hypothesis.

12. Export, Deployment, and Sim-to-Real

A training checkpoint is not the deployed policy. Deployment is a contract between the exported graph, observation producer, command source, timing loop, action mapping, actuator interface, and safety system.

12.1 Artifact flow

```
model_<iteration>.pt
  actor + critic + normalizers + optimizer + training state
        |
        | scripts/export.py
        v
policy.onnx
  actor inference graph + actor observation normalizer + metadata
        |
        | scripts/infer_policy.py
        v
CPU MuJoCo rehearsal using deployment-style observations and commands
        |
        | real runtime integration and staged safety tests
        v
14 joint targets at 50 Hz on Microduck
```

The critic, reward functions, terminations, randomizers, and PPO optimizer are not deployed. Their job was to shape the actor during training.

12.2 Export through the mandatory path

Export a local checkpoint:

```
uv run scripts/export.py Mjlab-Velocity-Flat-MicroDuck \
    --checkpoint-file logs/rsl_rl/velocity/<run>/model_<iteration>.pt \
    --onnx-file walk.onnx
```

Or export from W&B:

```
uv run scripts/export.py Mjlab-Velocity-Flat-MicroDuck \
    --wandb-run-path <entity>/mjlab_microduck/<run_id> \
    --onnx-file walk.onnx
```

The script:

1. loads the play configuration for the exact task ID;
2. constructs the actor and critic architecture;
3. restores the checkpoint;
4. obtains the deterministic inference policy;
5. exports the actor with its `EmpiricalNormalization` submodule; and
6. attaches metadata such as joint names, defaults, action scale, commands, and observation names.

Never hand-convert the checkpoint. In-simulator playback normalizes observations and can conceal a missing-normalizer deployment bug.

12.3 Inspect the ONNX interface

```
uv run python - <<'PY'
import numpy as np
import onnxruntime as ort

path = "walk.onnx"
session = ort.InferenceSession(path, providers=["CPUExecutionProvider"])
input_info = session.get_inputs()[0]
output_info = session.get_outputs()[0]

print("input:", input_info.name, input_info.shape, input_info.type)
print("output:", output_info.name, output_info.shape, output_info.type)
print("metadata:", session.get_modelmeta().custom_metadata_map)

obs = np.zeros((1, 61), dtype=np.float32)
action = session.run([output_info.name], {input_info.name: obs})[0]
print("action shape:", action.shape)
print("finite:", np.isfinite(action).all())
PY
```

For this policy family, expect one batch of 61 float inputs and 14 float outputs. A finite result for an all-zero synthetic observation proves graph execution, not meaningful robot behavior.

12.4 Rehearse the deployment loop

Current policies use the unified 13D command block, so include `--new-cmd-obs`:

```
uv run scripts/infer_policy.py \
    --walking walk.onnx \
    --new-cmd-obs \
    --lin-vel-x 0.20
```

This script runs ONNX Runtime on the CPU and builds the same observation order the robot runtime must build. It also applies action delay and maps actions as:

```
observation = np.concatenate([
    base_angular_velocity,       # 3
    projected_gravity,           # 3
    relative_joint_position,     # 14
    joint_velocity,              # 14
    previous_action,             # 14
    command,                     # 13
]).astype(np.float32)

action = onnx_session.run(..., {input_name: observation[None, :]})[0]
target_position = default_pose + action * action_scale
```

The terminal, not the viewer window, receives keys:

```
UP/DOWN           forward command
LEFT/RIGHT        lateral command for the walking model
A/E               turn command
SPACE             zero velocity commands
H                 enter/leave head-command mode
B                 enter/leave body-command mode
P                 apply a random push
Q                 quit
```

The body-pose UI exists for policy families that train those slots. The main velocity checkpoint has body-pose reward weight zero, so UI availability is not proof of learned body control.

12.5 The exact 61D runtime contract

Slice	Values	Runtime source
base angular velocity	3	calibrated IMU, body frame, rad/s
projected gravity	3	calibrated orientation projected into body frame
relative joint position	14	servo encoders minus exact HOME offsets
joint velocity	14	encoder-derived velocity in matching order/units
previous action	14	last actor output, not measured joint target
twist	3	application/planner command
head pose	4	requested joint deltas from HOME
body pose	6	requested body deltas or zeros

For every slice, verify:

```
dimension
ordering
dtype
units
frame
sign
offset
latency
noise/filter behavior
reset/initial value
```

The ONNX graph normalizes the assembled raw observation. Do not pre-normalize again in the runtime.

12.6 Timing is part of the policy

Microduck training assumes:

```
physics model step        5 ms
policy period            20 ms (50 Hz)
action decimation         4
unfiltered policy output
modeled sensor/action delays
```

A real loop should use a monotonic clock, timestamp sensor snapshots, detect missed deadlines, and choose a documented safe response. Running inference “whenever Linux schedules it” without observing jitter changes the effective delay distribution.

Do not silently reuse stale commands forever. High-level commands need an owner, sequence/timestamp, and expiry behavior. The local safety controller must be able to stop or hold the robot without a cloud round trip.

12.7 Sim-to-real gap

The sim-to-real gap is the difference between the transition distribution used for training and the real robot:

$$P_{sim}(s_{t+1} \mid s_t, a_t) \neq P_{real}(s_{t+1} \mid s_t, a_t)$$

Microduck reduces this gap with:

- measured CAD geometry and mass properties;
- the BAM voltage-controlled XL330 model;
- friction, armature, mass/inertia, and CoM randomization;
- battery voltage and load-sag variation;
- sensor noise, bias, IMU misalignment, and latency;
- command delay;
- backlash models with encoder-consistent observations; and
- pushes and varied initial states.

Randomization should cover plausible uncertainty, not every imaginable value. An unrealistically wide distribution can teach an overly conservative policy or make the task unsolvable.

12.8 Calibrate before randomizing

Use a parameter workflow:

```
measure real subsystem
        v
fit nominal simulator parameter
        v
validate on held-out motion
        v
estimate repeatability and uncertainty
        v
randomize around the nominal range
```

Examples:

- identify actuator friction and torque response on a bench;
- measure battery sag under representative load;
- compare commanded and measured step responses;
- measure encoder bias and delay;
- weigh assemblies and estimate their CoM/inertia; and
- verify foot contact friction on real surfaces.

`scripts/testbench_sim2real.py` and `scripts/validate_bam_testbench.py` support the actuator side of this process.

12.9 Staged physical acceptance

The exact hardware procedure belongs to the Microduck runtime/hardware project, but a safe conceptual progression is:

1. **Static interface test:** no torque; verify joint order, signs, HOME, sensors, command zeros, ONNX shapes, and loop timing.
2. **Supported low-authority test:** robot secured; conservative target/current limits; verify action direction and emergency stop.
3. **Supported policy test:** run short windows with zero or tiny commands; inspect saturation, delay, temperature, and observation ranges.
4. **Tethered behavior test:** flat surface, local operator, hard timeout, recorded telemetry/video.
5. **Bounded untethered test:** only after earlier acceptance criteria pass.
6. **Command and disturbance battery:** expand one measured region at a time.

Never make the cloud, Wi-Fi, or a high-level planner responsible for the fast emergency path.

12.10 Hot-swapping policies

The runtime can switch walking, standing, recovery, and trick policies because they share the 61D input and 14D output contract. Hot-swapping still requires:

- the same joint/action metadata;

- correct command-slot semantics for each skill;
- clearing stale command values when a skill expects zero padding;
- defined previous-action behavior at the boundary; and
- a safe arbitration state machine.

For example, sit/stand uses the first twist slot as a posture flag, while an all-zero twist means stand. Feeding zeros because an application forgot the flag can look like a policy that ignores the button.

Rehearse combinations before hardware:

```
uv run scripts/infer_policy.py \
    --walking walk.onnx \
    --standing stand.onnx \
    --sitstand sitstand.onnx \
    --roulade roulade.onnx \
    --new-cmd-obs
```

12.11 Deployment acceptance record

For each released policy, retain:

```
source commit and task ID
resolved env/agent YAML
checkpoint digest
export command and ONNX digest
ONNX input/output and metadata inspection
fixed simulation evaluation results
CPU rehearsal results
runtime version and observation/action contract version
hardware calibration version
staged physical test logs and video
known operating envelope and known failures
rollback artifact
```

“The file loads” is a format check. “The viewer walks” is a simulation check. Neither is physical release evidence.

12.12 Lab: prove checkpoint-to-ONNX parity

1. Select one checkpoint and record a fixed simulation command case.
2. Export it with `scripts/export.py`.
3. Inspect its 61→14 interface and metadata.
4. Run the same command through `scripts/infer_policy.py`.
5. Compare initial observations and actions for units, signs, and scale.
6. Explain any remaining trajectory difference from simulator/runtime details.

The goal is not bit-identical long trajectories—chaotic contacts diverge—but an evidence-backed interface match.

Continue with Microduck customization labs.

12.13 Folded parity-lab solution

Show the expected evidence and comparison code

The result should include model/checkpoint hashes, exact observation corpus, normalizer provenance, action transform, runtimes, tolerances, and per-element error—not only “both looked similar.” For the same already-normalized or same raw-and-embedded-normalizer input (choose one contract), compare arrays:

```
import numpy as np
```

```
# Shape: (number_of_frozen_cases, 14). These must come from the same 61D
# observations and deterministic actor mode.
checkpoint_actions = np.load("checkpoint_actions.npy")
onnx_actions = np.load("onnx_actions.npy")

assert checkpoint_actions.shape == onnx_actions.shape
assert checkpoint_actions.shape[1] == 14
error = np.abs(checkpoint_actions - onnx_actions)
print("max_abs", error.max())
print("p99_abs", np.quantile(error, 0.99))

# Select tolerances from numeric precision/runtime evidence; do not copy these
# illustrative values blindly for a quantized model.
np.testing.assert_allclose(
    onnx_actions,
    checkpoint_actions,
    rtol=1e-5,
    atol=1e-6,
)
```

First compare one-step outputs; long contact trajectories can diverge from tiny floating-point differences even when the interface is correct. If one-step outputs disagree, inspect observation order, raw versus normalized input, actor mean versus stochastic sample, dtype, HOME/action scaling, and stale previous action before blaming physics.

13. Customization Labs

These labs turn the book into a project course. Work on a branch, keep one question per experiment, and preserve tests/configuration with every result.

Lab 0: reproduce before modifying

Goal: prove that you can run and explain the existing system.

1. Run all CPU tests.
2. Run a 64-environment, five-iteration velocity smoke train.
3. Load an existing trained velocity checkpoint.
4. Record a 10-second rollout.
5. Export ONNX and run the CPU rehearsal with a fixed forward command.
6. Draw the 61D observation and 14D action layout from memory.

Deliverable: the lab report template from Chapter 9 plus one paragraph explaining why playback direction changes are command sampling, not random actions.

Lab 1: change a command distribution

Question: does more turn-in-place data improve turning without hurting forward walking?

The current setting is:

```
TURN_IN_PLACE_FRACTION = 0.15
```

Experiment:

1. Keep a baseline run and fixed evaluation battery.
2. Change only this fraction, for example to `0.25`.
3. Add/update a configuration test that checks the expected value.
4. Run the CPU suite and five-iteration smoke test.
5. Train both settings for the same budget and seed policy.
6. Compare turn-in-place success, translation drift, forward speed tracking, and total command coverage.

Do not conclude that 25% is universally better. Increasing one bucket reduces the fraction of all other experience.

Lab 2: add a small custom reward safely

Goal: learn the pure-helper → environment wrapper → configuration → test pattern.

Suppose a new task needs a self-negating trunk-height error:

```
# In tasks/mdp.py
def height_l1_from_values(height: torch.Tensor, target: float) -> torch.Tensor:
    """Return a nonpositive penalty; zero only at the target."""
    return -torch.abs(height - target)
```

```
def trunk_height_l1_penalty(
    env: ManagerBasedRlEnv,
    target: float,
    asset_cfg: SceneEntityCfg = SceneEntityCfg("robot"),
) -> torch.Tensor:
    asset = env.scene[asset_cfg.name]
    origin_z = env.scene.terrain.env_origins[:, 2]
    height = torch.nan_to_num(
        asset.data.root_link_pos_w[:, 2] - origin_z,
        nan=0.0,
    )
    return height_l1_from_values(height, target)
```

Wire it in the relevant task factory:

```
cfg.rewards["trunk_height_l1"] = RewardTermCfg(
    func=microduck_mdp.trunk_height_l1_penalty,
    weight=0.2,  # positive: the function already returns <= 0
    params={"target": 0.095},
)
```

Test the math and sign without a simulator:

```
def test_height_l1_penalty():
    h = torch.tensor([0.095, 0.105, 0.075])
    out = height_l1_from_values(h, target=0.095)
    torch.testing.assert_close(out, torch.tensor([0.0, -0.01, -0.02]))


def test_height_penalty_weight_is_positive():
    cfg = make_my_task_env_cfg()
    assert cfg.rewards["trunk_height_l1"].weight > 0.0
```

Before keeping the reward, answer:

- Is the target measured from the actual current robot model?
- Is height meaningful in every task phase?
- Could a fallen pose match the height?
- Does this term block motion the task physically requires?
- How large is its weighted mass relative to the positive objective?

This example teaches wiring; it is not a recommendation to add a height penalty to the main walking task.

Lab 3: design a new task from a template

Choose the closest family:

Intended behavior	Starting point
locomotion	velocity
episodic trick ending in a pose	stand-up
commanded two-state transition	sit-stand
dynamic maneuver	roulade

Implementation checklist:

1. Write the task's observable success criterion and plausible exploits.
2. Verify target states in physics before PPO.
3. Create `microduck_<name>_env_cfg.py` by building on the closest factory.
4. Put custom MDP functions in `tasks/mdp.py`, grouped with the task.
5. Preserve the 61D actor command layout and 14D action layout.
6. Select the correct walk/all-collisions/roller model.
7. Give the runner a distinct `experiment_name`.
8. Register train/play variants in `tasks/__init__.py`.

9. Add a backlash twin only if it mirrors the base model correctly.
10. Write config tests for dimensions, signs, selectors, gates, and task registration.
11. Run CPU tests and the five-iteration smoke train.
12. Evaluate the main behavior, every stable flop, and likely reward hacks.

Minimal registration shape:

```
register_mjlab_task(
    task_id="Mjlab-MyTask-Flat-MicroDuck",
    env_cfg=make_microduck_my_task_env_cfg(),
    play_env_cfg=make_microduck_my_task_env_cfg(play=True),
    rl_cfg=MicroduckMyTaskRlCfg,
    runner_cls=MicroduckOnPolicyRunner,
)
```

Lab 4: add a curriculum from evidence

Do not begin by inventing ten stages. First train the easier slice and identify the measured frontier.

Example goal: widen a pose command only after small commands track well.

```
cfg.curriculum["pose_range"] = CurriculumTermCfg(
    func=microduck_mdp.pose_command_range_curriculum,
    params={
        "command_name": "head_pose",
        "range_stages": [
            {"step": 0, "ranges": ((-0.05, 0.05),) * 4},
            {"step": 500 * 24, "ranges": ((-0.2, 0.2),) * 4},
        ],
    },
)
```

For the real head, use per-joint ranges rather than the identical illustrative ranges above because yaw, pitch, and roll have different limits.

Test that:

- stage steps use environment-step units;
- the live command-manager term changes;
- the first and last ranges are intended;
- the policy sees nonzero samples before a reward is introduced; and
- metrics do not collapse at the stage boundary.

Lab 5: obstacle avoidance—choose the architecture first

The current velocity policy cannot avoid obstacles. There are two different projects hidden inside the phrase “add obstacle avoidance.”

Option A: perception/planning above locomotion

```
camera/depth/ToF
      v
local obstacle model + planner
      v
safe local [vx, vy, yaw_rate] command
      v
existing 61D Microduck walking policy
```

This is the recommended first project. It preserves the proven fast motor policy and observation contract. The planner can be classical, learned, or cloud-assisted, but a local collision/stop layer must remain available when network service is absent or stale.

Build a simulation harness that varies obstacles and checks:

- planner command expiry;
- stop distance;
- minimum clearance;
- collision rate;
- progress to goal; and
- behavior on missing/stale perception.

The locomotion policy still needs robustness to the planner's command changes, but it does not need pixels.

Option B: an end-to-end exteroceptive policy

```
proprioception + command + depth/rays/features
      v
new actor architecture
      v
joint actions
```

This requires a new versioned policy family because extra features break the 61D hot-swap contract. Define:

- sensor model and hardware availability;
- feature dimensions/order/units and missing-data semantics;
- model architecture and inference budget;
- obstacle/goal scene generator;
- collision, clearance, progress, and deadlock objectives;
- domain randomization for sensor noise/dropout and obstacle geometry;
- runtime contract/version negotiation; and
- safety behavior independent of learned perception.

Do not merely add boxes and a collision penalty. Without pre-contact observation, the policy learns only collision reaction.

Lab 6: measure one sim-to-real parameter

Choose one subsystem, such as a single XL330 step response.

1. Define a safe bench input and collect timestamped command, position, velocity, current/voltage, and load conditions.
2. Replay the same input in the BAM testbench simulation.
3. Compare rise time, overshoot, steady-state error, and decay/friction.
4. Fit a nominal parameter using training data.
5. Validate it on a different input.
6. Estimate a realistic randomization range from repeated trials.
7. Record the model version and evidence.

The lesson is broader than one actuator: fit first, randomize uncertainty second.

Lab 7: create a policy acceptance battery

Write a headless evaluator for one task that runs a matrix of seeds and command cases. Output a machine-readable summary:

```
{
  "task": "Mjlab-Velocity-Flat-MicroDuck",
  "checkpoint": "model_2000.pt",
  "cases": 100,
  "successes": 92,
  "falls": 5,
  "nonfinite": 0,
  "mean_linear_tracking_error_mps": 0.04,
  "mean_yaw_tracking_error_radps": 0.09
```

```
}
```

Define success before running. Keep the raw per-case data so an aggregate number can be audited. Add video sampling for human-visible quality.

Capstone: a complete evidence-backed customization

Your capstone should include:

```
problem statement and non-goals
architecture choice
physics assumptions and measurements
observation/action/command contract
reward and termination rationale
likely exploits
unit and configuration tests
five-iteration smoke evidence
training configuration and checkpoints
fixed evaluation battery and videos
ONNX export/interface inspection
CPU deployment rehearsal
sim-to-real risk register
known limitations and next experiment
```

The capstone is complete when another person can reproduce the result and understand where it is safe to generalize—not when one attractive video exists.

Where to continue learning

After this book, read code and experiments in this order:

1. the velocity task and its focused tests;
2. sit-stand for command-conditioned state transitions;
3. stand-up for reset distributions and recovery shaping;
4. roulade for dynamic-task reward lessons;
5. backlash for observation/physics consistency; and
6. roller tasks for passive joints and architecture-specific indices.

Return to the book index whenever a new experiment crosses from task design into training or deployment.

Folded reference outcomes

Show the minimum acceptable result for Labs 0–7

- **Lab 0:** a pinned commit/config/checkpoint reproduces the baseline evaluator within declared stochastic uncertainty. A video alone is insufficient.
- **Lab 1:** the resolved command distribution changes exactly one intended bucket/range; exact zero remains explicitly sampled; a configuration test locks the values.
- **Lab 2:** the helper's sign and boundary behavior pass as pure tensor tests, its configuration weight has the intended sign, and every logged weighted penalty remains nonpositive.
- **Lab 3:** the new task inherits the closest proven template, preserves the 61D/14D deployment family contract when intended, resolves all joint/sensor selectors, and passes a five-iteration export smoke.
- **Lab 4:** the stage changes only after measured competence, uses environment steps correctly, and does not create a repeatable metric cliff at its boundary.
- **Lab 5:** either the modular planner proves local stop/clearance/freshness while retaining 61D, or a new exteroceptive schema is explicitly versioned; simply rendering obstacles is rejected.
- **Lab 6:** nominal actuator parameters are fit on one trace and validated on held-out traces; randomization represents residual uncertainty rather than a guess.

- **Lab 7:** a machine-readable per-case result, aggregate with uncertainty, failure taxonomy, and sampled videos can be regenerated from one command.

A pure reward helper and wiring test can follow this pattern:

```
def limit_proximity_cost(q, lower, upper, margin):
    distance = minimum(q - lower, upper - q)
    return maximum(0.0, margin - distance) / margin

def test_limit_cost_is_zero_away_from_limits():
    assert limit_proximity_cost(0.0, -1.0, 1.0, 0.1) == 0.0

def test_limit_cost_grows_near_upper_limit():
    assert limit_proximity_cost(0.95, -1.0, 1.0, 0.1) > 0.0
```

Adapt this to Torch and repository conventions; it illustrates separation of the mathematical value from simulator-manager data extraction.

14. Demonstrations, Imitation, and Offline Robot Learning

Much modern robot learning does not begin with random exploration and a reward. It begins with demonstrations, logs, or a pretrained policy. This chapter shows how those settings relate to—and differ from—reinforcement learning.

14.1 A transition dataset is an interface

A robot-learning dataset may contain

$$D = \{(o_t, a_t, r_t, o_{t+1}, d_t, m_t)\},$$

where m_t is optional metadata such as task language, camera calibration, timestamps, episode ID, or robot embodiment.

Before selecting an algorithm, audit:

- exact observation fields, units, frames, and timestamps;
- action semantics: torque, target, delta, end-effector pose, or action chunk;
- who/what generated each action;
- whether failed episodes are present;
- whether reward is measured, inferred, sparse, or absent;
- termination versus time-limit truncation;
- task and environment coverage;
- sensor/action rate and missing samples; and
- train/validation/test split by episode, scene, object, and operator.

A large dataset with inconsistent action semantics can be less useful than a small coherent one.

14.2 Behavior cloning

Behavior cloning (BC) treats demonstration actions as labels. For a continuous action and deterministic policy, a simple objective is

$$\min_{\theta} \; \mathbb{E}_{(o,a)\sim D}\left[\|\pi_\theta(o) - a\|_2^2\right].$$

Plain language: make the policy predict the demonstrator's action for each recorded observation.

BC is supervised learning, not reinforcement learning. It does not need reward, next-state values, or online interaction. It is often a strong baseline because it avoids reward design and difficult exploration.

Why mean-squared error can average incompatible actions

Suppose a demonstrator passes an obstacle equally often on the left and right. At the decision point the action distribution has two modes. A deterministic mean-squared-error policy may average them and drive straight toward the obstacle.

Responses include:

- provide more context so the modes become distinguishable;
- predict a multimodal distribution;
- use a mixture model or discrete latent choice;
- predict an action sequence/chunk; or
- use a generative policy such as diffusion.

14.3 Covariate shift and compounding errors

Training observations come from the demonstrator distribution $d_{expert}(o)$. Deployment observations come from the learned policy $d_{\pi}(o)$. A small prediction error changes the robot state; the next observation may never occur in demonstrations; another error follows.

This is **covariate shift**. In sequential control, errors can compound over time.

The Dataset Aggregation (DAgger) idea alternates:

1. run the current learner;
2. ask an expert for the correct action on states the learner visits;
3. aggregate those labeled states into the dataset; and
4. retrain.

The primary DAgger paper analyzes this interactive reduction. On hardware, expert intervention and safe rollout collection must be engineered; a human may not label a 1 kHz recovery action.

14.4 Action chunking

Instead of predicting one action, an **action-chunk** policy predicts a short sequence:

$$\pi(o_t) \rightarrow (a_t, a_{t+1}, \dots, a_{t+H-1}).$$

Benefits can include temporal consistency and fewer high-level inference calls. But open-loop execution for the entire chunk delays correction. Practical systems often use receding horizon: predict a chunk, execute a small prefix, observe again, and replan.

The ACT paper uses action chunking with a transformer-style conditional variational autoencoder for low-cost bimanual manipulation. Its demonstration-based success should not be generalized to fast balance loops without latency and disturbance tests.

14.5 Diffusion Policy

A diffusion model learns to turn noise into a structured sample through iterative denoising. Diffusion Policy conditions that process on robot observations to generate action sequences.

Conceptually:

```
random action sequence
       |
       v
denoise using image/state condition
       |
       v
more coherent action sequence
       |
       v
execute first action(s), observe, repeat
```

Why this can help:

- several valid action modes can be represented rather than averaged;
- an action horizon captures temporal structure;

- image-conditioned manipulation has genuinely multimodal choices.

Costs include several denoising evaluations, runtime latency, sensitivity to data coverage, and no automatic recovery outside demonstrations.

The primary Diffusion Policy paper reports evaluation across 12 tasks in four manipulation benchmarks and public code/data. Those results support the method in that protocol; they do not imply diffusion is required for all robots.

14.6 Offline RL: improvement without new interaction

Offline RL uses a fixed dataset but has rewards and sequential transitions. It tries to find a policy better than the data-collection behavior while avoiding unsupported actions.

This creates a tension:

```
stay close to data -> reliable estimates but limited improvement
move beyond data   -> possible improvement but uncertain values
```

Ordinary off-policy algorithms can overestimate an unseen action, then train the actor to select it. Because no online rollout corrects the mistake, the error can reinforce itself.

14.7 Conservative Q-Learning

Conservative Q-Learning (CQL) adds pressure for values of actions outside the dataset to be lower than values of observed actions. One conceptual form is

$$\min_Q \underbrace{L_{Bellman}(Q)}_{\text{fit transitions}} + \alpha \left(\underbrace{\mathbb{E}_{s,a\sim\mu}[Q(s,a)]}_{\text{candidate actions}} - \underbrace{\mathbb{E}_{s,a\sim D}[Q(s,a)]}_{\text{dataset actions}} \right).$$

μ is an action proposal distribution. The added term discourages the critic from assigning unjustified high value broadly.

The CQL paper provides theoretical and benchmark evidence for conservative value estimation. Conservative does not mean physically safe; it describes value estimation relative to data support.

14.8 Implicit Q-Learning

Implicit Q-Learning (IQL) avoids evaluating the Q-function at policy actions outside the dataset during its main value-learning stage.

Its key stages are:

1. fit a state value using **expectile regression** toward the upper portion of dataset action values;
2. fit Q with a Bellman target using that state value; and
3. extract a policy with advantage-weighted behavior cloning.

An expectile parameter $\tau > 0.5$ weights positive residuals more heavily, making $V(s)$ reflect better actions present in the dataset without taking an explicit max over unseen actions.

Policy extraction weights demonstrated actions approximately by

$$w(s,a) = \exp(\beta(Q(s,a) - V(s))),$$

usually with clipping. Better-than-baseline dataset actions receive more weight.

The IQL paper reports strong D4RL results and online fine-tuning. A beginner should retain the design principle: improve toward the best supported data without trusting arbitrary unseen actions.

14.9 Dataset coverage is the real boundary

Imagine a walking dataset containing only forward motion on high-friction floor. No offline objective can identify the correct recovery action for an unseen sideways slip purely from that data unless learned structure generalizes correctly. The dataset does not contain counterfactual evidence.

Audit coverage along dimensions that matter physically:

- commands and task goals;
- initial poses and failure/recovery states;
- surfaces, payloads, voltage, and temperature;
- sensor noise/dropout and latency;
- successful and unsuccessful behavior;
- humans/operators and camera viewpoints; and
- action saturation and safety boundaries.

14.10 D4RL and benchmark literacy

D4RL introduced standardized offline-RL datasets with behavior mixtures and evaluation protocols. A normalized score commonly has the form

$$100\frac{J_\pi - J_{random}}{J_{expert} - J_{random}}.$$

This makes scores more comparable within a benchmark, but the reference policies, environment version, termination handling, and dataset composition remain part of the result. A score of 100 is not "100% physically safe" or "solved for every robot."

14.11 Open robot-data ecosystems

Hugging Face LeRobot provides an open-source ecosystem for robot datasets, policies, and real-hardware tools. Its value for study is the full data-to-policy workflow and common dataset format—not an assumption that every supported policy is RL.

Generalist manipulation projects such as Octo and OpenVLA train on large collections of robot demonstrations. They belong primarily to pretrained imitation/foundation-policy research. Chapter 16 studies their architecture and safety boundary.

14.12 Choosing among BC, offline RL, and online RL

Start with behavior cloning when:

- demonstrations are strong and cover deployment;
- reward is missing or unreliable;
- a simple, auditable baseline is needed.

Consider offline RL when:

- transitions and meaningful rewards exist;
- the dataset contains a range of behavior quality;
- policy improvement beyond average demonstration behavior matters;
- you can evaluate conservatively before hardware.

Consider online fine-tuning when:

- a safe simulator or controlled hardware protocol exists;
- deployment states differ from logged data;
- reward is reliable; and
- rollback and exploration limits are enforced.

A common progression is

```
demonstrations -> behavior cloning -> offline RL (optional)
               -> safe simulation/real fine-tuning -> frozen deployment
```

Each arrow needs its own evaluation gate.

14.13 A practical dataset card

Record at least:

```
robot: exact hardware revision
task: operational success definition
episodes: total / success / failure
observation_schema: fields, shapes, units, frames
action_schema: meaning, range, rate, delay
sensors: models, calibration, timestamps
collection_policy: human / scripted / learned, versions
environment_distribution: objects, surfaces, lighting, payload
safety_interventions: what was filtered or stopped
splits: episode/scene/object/operator separation
known_gaps: unsupported conditions
license_and_consent: provenance and permitted use
```

Without provenance, “more data” can mean more untraceable error.

14.14 Exercises

1. Give an example where squared-error BC averages two valid actions into an invalid one.
2. Explain covariate shift using a robot that drifts 2 cm left per step.
3. Why can action chunking help smooth behavior, and when can it hurt reaction time?
4. Distinguish ordinary off-policy online RL from offline RL.
5. What kind of optimism do CQL and IQL try to avoid?
6. Design train/validation/test splits for three tables, ten objects, and two human demonstrators. Which generalization question does each split answer?
7. A dataset contains only successful trials. What information about failure recovery is missing?
8. Write a dataset card for a Microduck playback log. Which additional fields are needed before it could support offline learning?

Continue with modern robot locomotion, adaptation, and sim-to-real research.

14.15 Folded solutions

Show answers to Section 14.14

1. Demonstrations may pass an obstacle on the left with steering −1 or on the right with +1. Squared-error BC predicts their mean, zero, which can drive directly into the obstacle. More context or a multimodal distribution must represent the alternatives.
2. After one 2 cm left error, the robot observes a state slightly outside the expert path. BC was not trained to correct there, so it may produce another error; after n uncompensated steps the simple drift is $0.02n$ metres and the observation distribution moves progressively farther from training.
3. A chunk represents temporally coherent intent and reduces high-frequency switching. A long open-loop chunk also delays reaction to a person, collision risk, or changed contact. Receding-horizon execution and measured chunk age balance these effects.
4. Off-policy **online** RL reuses old data but can still gather new transitions under evolving policies. Offline RL is restricted to a frozen dataset and must avoid relying on actions absent from it.
5. Function approximation can assign unrealistically high Q-values to unseen state-action pairs. CQL penalizes such optimism explicitly; IQL improves from in-dataset action/value structure without querying arbitrary unseen actions in its main value update.

6. Hold out an entire table to test scene transfer, hold out selected objects on otherwise seen tables to test object transfer, and hold out one operator to test demonstrator/style transfer. A final test can combine held-out table, object, and operator; validation cases used for model selection must remain separate from that test.
7. The data omits pre-failure warning states, failed actions, recovery paths, intervention thresholds, and the boundary between recoverable and unsafe. A policy trained only on success cannot infer these merely from labels it never saw.
8. A Microduck log card needs the exact robot/task/commit/checkpoint, 61D field order and normalization, 14D action transform, units/frames/rates/timestamps, command process, randomization, resets, terminations, reward reconstruction, episode IDs, quality/failure labels, and splits. For offline learning it also needs next observations, terminal/truncation distinction, reproducible rewards, coverage analysis, and license/provenance.

15. Modern Robot Locomotion, Adaptation, and Sim-to-Real Research

Recent locomotion systems combine ideas rather than relying on one magical algorithm: GPU-parallel simulation, calibrated actuator models, domain randomization, privileged training information, histories or latent adaptation, curricula, and strict deployment interfaces.

This chapter is an annotated research pathway. Dates and evaluated scope are included because "state of the art" is meaningful only relative to a protocol.

15.1 The recurring locomotion architecture

Many systems have this shape:

```
terrain / navigation / operator
             |
      command or short plan
             |
  learned locomotion policy -(20200 Hz)
             |
 joint targets or torques
             |
 embedded motor control and safety (faster)
             |
        physical robot
```

The learned policy is usually a local motor skill. It does not necessarily choose a destination, recognize an object, build a map, or enforce hard current limits.

15.2 Milestone: actuator-aware sim-to-real locomotion

The 2019 Learning agile and dynamic motor skills for legged robots paper trained policies in simulation and transferred them to ANYmal. A key lesson is that actuator behavior and dynamics identification are first-class parts of the learning system.

What to study:

- how actuator behavior was represented;
- what observations/actions reached the deployed network;
- which parameters were randomized;
- how real tests were evaluated; and
- which skills used separate policies or objectives.

What not to conclude: every robot can copy ANYmal's parameter values or reward weights. Morphology, actuator, sensor, rate, and task differ.

Microduck's BAM XL330 model belongs to the same engineering tradition: train through a more realistic actuator interface instead of an unlimited ideal position source.

15.3 Milestone: massively parallel simulation

The 2021 Learning to Walk in Minutes Using Massively Parallel Deep Reinforcement Learning work showed that thousands of GPU-parallel environments can greatly reduce wall-clock PPO training time and introduced a game-inspired terrain curriculum. It accompanied the open `legged_gym` ecosystem.

The important distinction is:

$$\text{environment steps} \neq \text{wall-clock seconds}.$$

Parallelism produces more samples per second; it does not make each sample more informative. It changes optimization dynamics too: batch size, update count, and policy lag must be understood.

Microduck's 4,096 environments × 24 steps is this pattern applied through MuJoCo Warp and RSL-RL.

15.4 Open training stacks

Useful open projects occupy different layers:

Project	Main role	Study value
`rsl_rl`	robot-focused PPO and related training components	readable runner/algorithm/storage separation
`legged_gym`	Isaac Gym locomotion environments	influential vectorized locomotion recipe; pin compatible versions
Isaac Lab	Isaac Sim robot-learning framework	sensors, randomization, RL/imitation workflows
`mjlab`	manager-based MuJoCo Warp robot learning	Microduck's environment layer
MuJoCo Playground	GPU-accelerated MuJoCo robot-learning suite	open locomotion, manipulation, vision, and sim-to-real examples
Genesis	general robotics physics/simulation platform	alternative parallel simulator; inspect released features, not roadmap claims

These are not drop-in equivalents. Compare physics, renderer, task API, algorithm integration, supported hardware, license, version compatibility, and reproducibility before migrating.

The 2025 MuJoCo Playground paper documents an open MuJoCo-based stack across locomotion and manipulation. Its breadth makes it a valuable comparison project for the Microduck/mjlab design.

15.5 Domain randomization: train a distribution, not one simulator

Let physical parameters be ξ, such as mass, friction, delay, or motor strength. Instead of optimizing one nominal environment, domain randomization optimizes

$$J(\theta) = \mathbb{E}_{\xi \sim p(\xi),\ \tau \sim \pi_\theta, \xi}\left[\sum_t \gamma^t r_t\right].$$

Plain language: sample a plausible robot/world, run the policy, and optimize average performance over that population.

Early primary demonstrations include visual domain randomization and dynamics randomization for control.

What to randomize

Randomize uncertainty that exists and affects behavior:

- link mass, inertia, and center of mass;
- ground friction/restitution/compliance;
- actuator gain, strength, friction, damping, delay, and voltage;
- encoder/IMU bias, noise, mounting error, and dropout;
- command delay and control-period jitter;
- initial state, terrain, disturbances, and payload.

Three failure modes

1. **Wrong center**: randomization surrounds an inaccurate nominal model.
2. **Too narrow**: the real robot lies outside the training distribution.
3. **Too broad**: policy becomes unnecessarily conservative or cannot find a behavior valid across contradictory physics.

Calibrate, quantify residual uncertainty, randomize, then validate held-out corners. Do not select ranges by folklore.

15.6 System identification is becoming more systematic

The open PACE sim-to-real project uses measured encoder data and optimization to identify actuator/joint dynamics for legged robots. It is a useful modern study of the workflow

```
safe excitation -> measured trajectory -> parameter fit
                -> held-out validation -> policy training/transfer
```

The reusable idea is measurement-driven identification. Its exact tooling and models are not automatically suitable for XL330 servos or a wheeled-leg robot; the excitation, parameters, and safety protocol must match the hardware.

15.7 Privileged learning

Simulation exposes information unavailable on the robot: exact base velocity, terrain height, contact force, friction, mass, and external disturbance. **Privileged learning** uses this information during training without requiring it at deployment.

Three patterns are common:

Asymmetric critic

```
actor  <- deployable observations only
critic <- actor observations + privileged state
```

The better-informed critic can estimate value with less ambiguity while the actor remains deployable.

Teacher-student distillation

```
privileged teacher -> target actions or latent representation
deployable student -> imitates using sensor-accessible history/vision
```

Privileged experience for later RL

A privileged policy generates useful trajectories that warm-start a visual or off-policy learner.

The non-negotiable audit is actor information at deployment. A simulator result with height-map truth does not prove a depth-camera student works.

15.8 Online adaptation and RMA

Fixed domain randomization asks one policy to be robust across all sampled physics. An adaptive system tries to infer the current environment.

Rapid Motor Adaptation (RMA) separates:

- a base policy conditioned on an environment latent; and
- an adaptation module that infers that latent from recent proprioceptive history.

Conceptually:

$$z_t = g_\psi(o_{t-H:t}, a_{t-H:t-1}), \qquad a_t = \pi_\theta(o_t, z_t).$$

z_t is not necessarily a human-readable friction coefficient. It is a control-useful summary inferred from how the robot has responded.

Questions for a new robot:

- Which physical changes leave an observable trace in the history?
- How long must the history be relative to the dynamics?
- How quickly does the estimate react versus amplify noise?
- What happens immediately after reset, before enough history exists?
- Was the adaptation distribution broad enough to include hardware?

15.9 Robust perceptive locomotion

Proprioception tells what the robot feels now; **exteroception** senses the external environment before contact, using depth, LiDAR, or vision.

The 2022 Learning robust perceptive locomotion for quadrupedal robots in the wild work integrates exteroceptive and proprioceptive information with a recurrent encoder designed to cope with unreliable terrain perception.

This addresses a different task than Microduck velocity tracking. Obstacles or terrain must appear in the actor input (or in a planner producing safe commands). Rendering an obstacle without sensing it does not create perceptive locomotion.

15.10 From locomotion to parkour

Agile terrain tasks need more than a flat-ground velocity reward:

- terrain/obstacle observations;
- goal/contact representation;
- curricula or privileged teachers;
- collision and constraint definitions;
- recovery from perception error; and
- evaluation by obstacle class and failure mode.

Extreme Parkour with Legged Robots studies low-cost quadruped parkour with a front depth camera. The 2024 SoloParkour work studies constrained visual locomotion, using privileged experience to warm-start depth-based off-policy learning on Solo-12.

Compare what each deploys—not only the video:

- sensor and frame rate;
- observation history;
- teacher information;
- action/control rate;
- constraint definition;
- obstacle distribution; and
- number and type of real trials.

15.11 Hierarchical wheeled-leg locomotion

The 2024 Learning Robust Autonomous Navigation and Locomotion for Wheeled-Legged Robots integrates adaptive locomotion, a learned navigation controller, and large-scale path planning. The paper is especially relevant to Jump Rover because it demonstrates the architectural distinction between local wheel-leg control and navigation.

It does not imply that a new wheeled-leg platform can train before its mechanical, actuator, sensing, and realtime interfaces are characterized. Architecture transfers more readily than weights.

15.12 Recent off-policy scaling is a research direction

The 2025 preprint Learning Sim-to-Real Humanoid Locomotion in 15 Minutes reports massively parallel FastSAC/FastTD3 recipes on humanoids. It is valuable evidence that off-policy continuous-control methods can be engineered at GPU parallel scale, challenging a simplistic "PPO is always the locomotion choice" belief.

Treat it as a recent, reproducible direction with stated hardware and compute, not a universal 15-minute promise. Reproduction must match environment count, GPU, simulator, update-to-data ratio, task, randomization, and success criteria.

15.13 A research-to-project comparison card

For any locomotion paper, fill this before copying code:

```
Paper/project and version:
Robot morphology and mass:
Actuator and low-level controller:
Policy rate / physics rate:
Observation (train actor / deploy actor / critic):
Action representation:
Command/goal representation:
Algorithm and network memory:
Simulator and environment count:
System identification:
Domain randomization ranges:
Curriculum/reset distribution:
Baselines:
Number of training seeds:
Sim evaluator:
Real trials and failure categories:
Public code/checkpoint/data:
What is directly transferable:
What must be re-measured:
```

15.14 Microduck research extensions

High-value controlled studies include:

1. **Robustness versus adaptation**: compare fixed history-free PPO with an adaptation latent under held-out actuator/friction changes.
2. **Calibration versus randomization width**: measure actuators, fit BAM, then ablate narrow/medium/wide residual randomization.
3. **Privileged critic ablation**: keep actor identical and compare critic information across seeds.
4. **PPO versus off-policy at matched cost**: match environment steps, GPU time, evaluator, and model capacity.
5. **Perceptive hierarchy**: compare a local obstacle planner commanding the unchanged walk policy against a new perceptive locomotion actor.

Each is a paper-shaped question because it changes one conceptual factor and defines evidence before training.

15.15 Exercises

1. Explain why parallel simulation improves wall-clock throughput but not necessarily sample efficiency.
2. Give one train-only privileged variable and a way a deployment student could infer its effect.
3. Contrast robustness through domain randomization with online adaptation.
4. Why must randomization ranges be based on calibration and held-out tests?
5. Design a failure taxonomy for perceptive stair climbing.
6. From one paper in this chapter, fill the research-to-project comparison card using only the paper and official repository.
7. Propose one Microduck experiment whose negative result would still teach something specific.

Continue with vision, foundation policies, and hierarchical autonomy.

15.16 Folded solutions

Show reference answers to Section 15.15

1. Parallel simulation generates more transitions per wall-clock second. Sample efficiency asks how much improvement occurs per transition, so batching the same algorithm across more environments does not inherently improve it.
2. A teacher may observe true friction. A deployment student can infer its effect from a history of commands, wheel/foot motion, body response, and slip/contact; the latent should be tested on held-out friction changes.
3. Domain randomization trains one policy to work across a distribution without explicitly identifying the current member. Online adaptation uses recent history or measurements to infer a latent/current dynamics context and changes behavior accordingly.
4. Unmeasured ranges can omit reality, include impossible physics, or make the task so broad that the policy becomes conservative. Calibration supplies the nominal/range hypothesis; held-out tests show whether it generalizes.
5. For stair climbing label at least: missed/late geometry; wrong height/depth; infeasible foothold; correct perception but wrong action; toe/body collision; slip/actuator saturation; stale frame; recovery success/failure; fall; and safety intervention. Report denominators and video/state evidence.
6. A valid comparison card must quote the chosen paper's exact embodiment, inputs, action, rate, data, baseline, seeds/trials, and public artifact. If the source omits an item, write "not reported"; do not infer it from a neighboring project.
7. Example: "Does a history-conditioned actor improve held-out voltage-sag recovery at matched nominal gait quality?" A negative result still bounds the value of history under the tested delay/range and may show that actuator calibration or explicit voltage sensing is the simpler solution.

16. Vision, Foundation Policies, and Hierarchical Autonomy

A robot that walks is not automatically a robot that knows where to go. This chapter separates perception, state estimation, local control, planning, and cloud reasoning, then shows where modern vision-language-action models fit.

16.1 Perception changes the observation problem

Proprioception measures the robot itself: joint encoders, IMU, motor state. **Exteroception** measures the surrounding world: camera, depth, LiDAR, radar, or tactile arrays.

Images are high-dimensional and ambiguous. One RGB frame does not directly state depth, velocity, object identity, traversability, or occlusion. A visual control system commonly needs:

- calibration (intrinsics/extrinsics);
- timestamps and synchronization;
- an encoder or geometric perception pipeline;
- temporal history for motion and occlusion;
- an explicit confidence/validity representation; and
- behavior when the sensor is delayed, dark, blocked, or missing.

16.2 Three ways to connect vision to control

Modular perception and planning

```
camera -> detector/depth/SLAM -> map or local obstacles
       -> planner -> safe velocity command -> locomotion policy
```

Advantages: inspectable intermediate outputs, reusable mapping/planning, easier geometric safety constraints. Risks: interface errors and perception mistakes propagate.

End-to-end visual policy

```
camera + proprioception + goal -> neural policy -> motor/task-space action
```

Advantages: features optimize for the task; may exploit cues omitted by a hand designed representation. Risks: high data demand, harder diagnosis, distribution shift, and latency.

Hybrid

```
camera -> pretrained/learned encoder -> compact latent or terrain map
       + proprioception -> local policy
       + geometric safety layer
```

Most real systems are hybrids somewhere, even if a paper calls one block “end-to-end.” Motor drivers, limits, synchronization, and emergency handling remain outside the learned model.

16.3 Obstacles must affect information and objective

Putting boxes in a simulator does not teach obstacle avoidance. At minimum:

1. the agent must receive pre-contact obstacle information or a planner must convert it into commands;
2. training must vary obstacle layouts enough to prevent memorization;
3. reward/constraints must distinguish safe progress from collision;
4. resets must expose relevant decisions; and
5. evaluation must hold out shapes, positions, textures, and sensor failures.

For Microduck, two valid projects are:

Project A: preserve the locomotion actor

```
depth/camera -> local map -> collision-aware planner -> [vx, vy, yaw rate]
                                                    -> existing velocity actor
```

This is the lower-risk route when the current actor already tracks commands.

Project B: train perceptive locomotion

```
depth/history + proprioception + goal -> new actor -> 14 joint targets
```

This can coordinate footsteps with terrain, but it changes observation contract, training cost, runtime compute, failure modes, and sim-to-real burden.

16.4 Representation learning

A visual encoder maps an image I_t to a smaller feature vector:

$$z_t = f_\psi(I_t).$$

The encoder can be:

- trained end-to-end from task reward;
- pretrained with supervised labels;
- pretrained self-supervised on images/video;
- frozen during policy learning; or
- jointly fine-tuned.

A visually rich latent is not necessarily control-relevant. Conversely, a task-trained latent can discard object attributes needed by future tasks. Probe latents with held-out conditions and downstream success rather than judging a visualization alone.

16.5 Generalist and foundation robot policies

A **generalist robot policy** is trained across many tasks, datasets, or robot embodiments instead of from scratch for one skill. A **foundation model** is a broadly pretrained model intended to adapt to many downstream tasks. These terms describe scope and reuse, not guaranteed general intelligence.

A Vision-Language-Action (VLA) policy typically maps images and language, sometimes proprioception, to robot actions or action tokens:

$$a_{t:t+H} = \pi(I_{t-k:t}, \text{language}, q_t).$$

Most VLA pretraining is imitation/supervised sequence modeling on robot demonstrations, not classical online RL. RL may later fine-tune preferences or task reward, but the terms should not be conflated.

16.6 Octo: an open generalist policy

The 2024 Octo paper presents an open-source transformer-based policy pretrained on 800,000 trajectories from the Open X-Embodiment dataset and studies adaptation across robot setups.

Study questions:

- How are different observation and action spaces tokenized/normalized?
- What part is pretrained versus newly initialized for a robot?
- Which task/robot combinations are evaluated zero-shot versus fine-tuned?
- What inference hardware and rate are used?
- Does the target task exist in the data distribution?

The transferable idea is diverse pretraining plus explicit embodiment adapters. The pretrained weights do not know a new robot's safety limits automatically.

16.7 OpenVLA

The 2024 OpenVLA paper presents an open 7-billion-parameter VLA trained on 970,000 real-robot demonstrations, combining pretrained visual/language components with action prediction. It also studies parameter-efficient fine-tuning and quantized serving.

Those scales are dramatically different from Microduck's small 50 Hz MLP. OpenVLA may help interpret language and visual manipulation tasks; placing a multi-billion-parameter model directly in a hard balance loop would require a completely different latency, reliability, and safety case.

16.8 π_0 and flow-matching action generation

The 2024 π_0 paper combines a pretrained vision-language model with **flow matching**, a generative approach for continuous action sequences. It evaluates diverse manipulation behaviors across several robot platforms.

The research direction is important: semantic visual-language knowledge and continuous dexterous action generation can be trained together. The correct engineering questions remain concrete:

- What data overlaps the target task and embodiment?
- What is the control horizon and replan rate?
- How are actions normalized and converted to hardware commands?
- What happens under instruction ambiguity or sensor shift?
- Which low-level and safety controllers remain outside the model?

16.9 Foundation policy versus local motor skill

These layers solve different timescales:

Layer	Example output	Typical concern
language/semantic agent	exact skill request and parameters	grounding, hallucination, network delay
task planner	ordered skill graph	feasibility and recovery
navigation/perception	local path or velocity	obstacles, localization, uncertainty
learned motor skill	joint/velocity targets	balance, contact, actuator mismatch
realtime controller	current/PWM/enable	deadlines and hard limits

A large model can choose "follow the person at 0.3 m/s." It should not invent a raw 20 kHz PWM stream. A small local policy can stabilize a commanded motion without understanding the sentence "follow Bruce to the workshop."

16.10 Hierarchical reinforcement learning

In **hierarchical RL**, a higher policy chooses a skill, goal, latent, or subtask; a lower policy executes it.

An option is often described by:

- initiation set: states where it can start;
- internal policy: actions while active; and
- termination condition: when it ends.

For a robot:

```
option: dock
preconditions: dock detected, localization confident, battery state valid
parameters: dock ID, approach side
termination: charging contact confirmed or timeout/fault
```

The option boundary is also an API and safety boundary. Exact typed parameters are safer than free-form text passed into control code.

16.11 Cloud agent: proposal, not authority

A cloud model can add:

- language understanding;
- long-horizon task decomposition;
- semantic interpretation of selected images/maps;
- retrieval of manuals or prior mission context; and
- human-facing explanation.

It also adds variable latency, disconnection, uncertain outputs, privacy/data flow, service/version drift, and adversarial or ambiguous input.

A robust flow is:

```
human request + summarized world state
                  |
                  v
cloud agent proposes exact option IDs/parameters
                  |
                  v
local schema validation + precondition check
                  |
                  v
local planner / behavior tree / safety filter
                  |
                  v
local motor skill -> realtime controller
```

The cloud does not send torque, PWM, motor current, balance correction, or unbounded wheel/servo targets. Loss of cloud must leave local stop and balance available.

16.12 Runtime contracts for learned skills

Package a skill with a manifest, not just a model file:

```
policy_id: walk-flat-v3
model_sha256: "..."
training_code_commit: "..."
robot_model_revision: "..."
input_schema_version: microduck-obs-61-v1
output_schema_version: joint-delta-14-v1
policy_rate_hz: 50
max_inference_ms: 6.0
normalization: embedded
command_bounds:
  vx_mps: [-0.3, 0.3]
```

```
  yaw_rps: [-1.0, 1.0]
validated_conditions: [flat_indoor, nominal_voltage]
known_exclusions: [stairs, obstacle_avoidance]
fallback_policy_id: stand-safe-v2
```

The manifest prevents a semantically wrong but shape-compatible model from being hot-swapped.

16.13 Perception uncertainty should change behavior

Do not reduce every perception output to a confident point estimate. Useful signals include confidence, age, covariance, valid-region mask, and source.

Example local rule:

```
if obstacle_map.age_ms > 200 or obstacle_map.confidence < 0.7:
    requested_speed = min(requested_speed, 0.05)
if obstacle_map.invalid:
    request_stop()
```

Thresholds need system-level validation. The principle is that uncertainty affects permitted behavior rather than being logged and ignored.

16.14 Worked Jump Rover architecture example

For a wheeled-leg rover with an SG2002 brain and a realtime motion MCU:

1. keep motor current/FOC, enable, limits, watchdog, and emergency behavior on realtime hardware;
2. run a bounded balance/drive policy only where worst-case timing is proven;
3. run perception, world state, local planning, and skill orchestration on the onboard brain;
4. let the cloud propose semantic goals or exact skills asynchronously; and
5. ensure network loss triggers no unsafe motor-state transition.

Chapter 17 turns this into a capstone. The Jump Rover repository's project handoff must additionally gate training on measured mechanics and accepted realtime-controller evidence.

16.15 Exercises

1. Explain why a visible simulated obstacle is not an actor observation.
2. Compare modular and end-to-end obstacle avoidance for a small biped.
3. Why is a VLA usually closer to imitation learning than online RL?
4. Design an exact JSON-like option request for "follow person," including speed and stop conditions but no raw motor command.
5. What should happen if cloud response arrives 3 seconds late?
6. List the latency budgets that must be measured before placing a visual model in a control loop.
7. Write a policy manifest for one Microduck behavior.
8. Give three held-out tests for a person-following perception system.

Continue with research literacy and capstone projects.

16.16 Folded solutions

Show reference answers to Section 16.15

1. Rendering affects what a human sees. The actor can respond only to tensors actually included in its observation. Microduck's 61D actor has no pixels, depth, range, or obstacle map.
2. A modular design lets local perception/planning convert geometry into twist commands for the existing fast policy; components are easier to test and the 61D contract survives. End-to-end depth-

to-joint control may coordinate tighter maneuvers but needs far more data, a new runtime schema, sensor randomization, latency proof, and independent safety.

3. A VLA is usually trained to predict actions from logged vision/language/ action demonstrations with a supervised sequence loss. A neural policy does not become RL unless reward-bearing interaction or an RL objective actually updates it.
4. One bounded future option could be:

```
{
  "skill": "follow_person",
  "track_id": "person.bruce",
  "following_distance_m": 1.2,
  "max_speed_mps": 0.15,
  "deadline_ms": 1500,
  "stop_if": {
    "track_age_ms_gt": 200,
    "map_age_ms_gt": 200,
    "minimum_clearance_m_lt": 0.35
  }
}
```

   A local signed schema, identity/consent rule, planner, and stop path must validate it. There is no PWM, torque, wheel speed, or servo target.
5. Reject a response that arrives after its bound or revalidate it against the current world and issue a fresh request. Never execute stale semantic intent merely because the JSON was valid when generated.
6. Measure capture/exposure, preprocessing, queueing, inference mean/tail/WCET, postprocessing, transport, planner, actuation delivery, total observation age, jitter, missed deadlines, and fallback transition.
7. A minimal manifest needs policy/model hashes, training commit, robot/model revision, input/output schemas, normalization, action scaling, policy rate, maximum input age, inference WCET, validated operating envelope, exclusions, calibration, evaluator report, and rollback artifact.
8. Hold out a person/appearance, lighting/background, and motion/occlusion pattern. Also test crossing identities and complete track loss; report false follow, missed follow, ID switch, age, localization error, and stop latency.

17. Research Literacy, Reproduction, and Capstone Projects

Being current in reinforcement learning does not mean collecting paper titles. It means being able to state what was tested, reproduce a meaningful slice, measure uncertainty, and know which conclusion the evidence does not support.

17.1 A source hierarchy

Prefer sources in this order:

1. peer-reviewed paper or clearly versioned preprint;
2. official project page and repository from the authors;
3. released configuration, code, checkpoints, data, and evaluation scripts;
4. independent reproductions that disclose deviations; and
5. summaries, talks, or social posts only as pointers.

An arXiv identifier proves a document exists; it does not prove peer review, correctness, reproducibility, or relevance to your robot. An open repository proves some artifact is visible; it does not prove it matches the paper result.

17.2 Read a paper in three passes

Pass 1: scope

Read title, abstract, figures, evaluation tables, limitations, and conclusion. Write one sentence:

```
The paper claims X on task/benchmark Y under protocol Z.
```

If you cannot fill Y and Z, do not repeat X yet.

Pass 2: mechanism

Read method and algorithm. Identify:

- inputs available during training and deployment;
- outputs/action representation;
- objective and constraint;
- data collection policy;
- model architecture and memory;
- simulator/hardware and rates; and
- differences from the strongest baseline.

Pass 3: evidence

Read experiment details, appendices, and repository configuration. Record:

- tasks and split;

- environment steps, data size, and compute;
- number of seeds and hardware trials;
- metric aggregation and uncertainty;
- hyperparameter tuning budget;
- ablations;
- failure examples; and
- artifact availability.

17.3 Claim taxonomy

Separate these claim types:

- **theoretical**: follows from stated assumptions and proof;
- **algorithmic**: mechanism is defined and implemented;
- **benchmark empirical**: result holds on evaluated benchmark/protocol;
- **sim-to-sim**: transfers between simulators;
- **sim-to-real**: evaluated on stated hardware conditions;
- **generalization**: evaluated on a specified held-out distribution;
- **scaling**: behavior changes with data/model/compute range;
- **systems**: latency, throughput, reliability, or resource result.

A benchmark empirical claim is not a theoretical guarantee. A ten-minute hardware video is not a reliability distribution. "Zero-shot sim-to-real" means no real-world policy fine-tuning under that setup; it does not mean no hardware calibration, system identification, or engineering.

17.4 Baselines answer "better than what?"

A useful baseline should be:

- relevant to the task and information setting;
- implemented competently;
- given a comparable tuning/compute budget;
- evaluated with the same metric and held-out cases; and
- simple enough to reveal whether complexity is necessary.

Robot baselines can include:

- zero/random policy for plumbing only;
- hand-designed or PD controller;
- MPC or state machine;
- behavior cloning;
- standard PPO/SAC implementation;
- prior method with authors' configuration; and
- an oracle using privileged information, clearly labeled as an upper bound.

A new RL controller that has not beaten an accepted scripted balance baseline is not ready for hardware because its training reward is high.

17.5 Ablation studies identify causes

An **ablation** removes or changes one component while holding other factors fixed.

Examples:

- BAM actuator model on versus ideal actuator;
- observation history 1 versus 4;
- privileged critic on versus off;
- action-rate curriculum versus full penalty from iteration 0;
- calibrated randomization versus arbitrary wide randomization;
- perception confidence supplied versus omitted.

An ablation needs multiple seeds when training is variable. Comparing one lucky run with one unlucky run does not identify a cause.

17.6 Random seeds and uncertainty

A seed initializes stochastic components: network weights, environment resets, commands, minibatch order, and sometimes GPU kernels. Deep RL can produce meaningfully different outcomes across seeds.

For returns $x_1, \ldots, x_n$, the sample mean is

$$\bar{x} = \frac{1}{n} \sum_i x_i.$$

The sample standard deviation is

$$s = \sqrt{\frac{1}{n-1} \sum_i (x_i - \bar{x})^2}.$$

Neither says everything about skewed failures. Also report median, quantiles, success rate, and named failure categories.

Bootstrap confidence interval intuition

To estimate uncertainty without assuming a normal distribution:

1. sample n results with replacement from the observed n results;
2. compute the chosen statistic;
3. repeat many times; and
4. take appropriate percentiles of the bootstrap statistics.

This quantifies sampling uncertainty given the observed runs. With only three seeds, the interval will be uncertain because the evidence is genuinely weak.

The RLiable evaluation paper explains why point estimates from a few RL runs can mislead and proposes interval estimates, performance profiles, and robust aggregate metrics.

17.7 Best checkpoint bias

If you evaluate 100 checkpoints on the same test conditions and report only the best, you have tuned on the test set. Similarly, choosing the best of many seeds and hiding the rest estimates luck, not expected performance.

Define selection before final testing:

```
training set: optimizer updates
validation set: checkpoint/hyperparameter selection
test set: one final locked evaluation
hardware acceptance: separately defined safety/performance protocol
```

17.8 Reproducibility levels

Repeatability

Same team, code, data, environment, and procedure obtains consistent results.

Reproducibility

Another team uses provided artifacts/procedure and obtains a compatible result.

Replicability

Another team independently implements the idea and obtains evidence supporting the claim.

Terminology varies by field, so define what you mean. In all cases preserve:

- source commit and dirty diff;
- lockfile/container and driver/CUDA versions;
- exact resolved environment/agent configuration;
- seeds and command lines;
- raw and aggregated metrics;
- checkpoints and model hashes;
- evaluator source/version;
- rollout video and failure labels; and
- hardware/calibration revisions.

The classic Deep Reinforcement Learning That Matters paper documents sensitivity to implementation and experimental choices. The lesson remains current: algorithm labels alone do not define an experiment.

17.9 Reproduction card template

```
# Reproduction: <paper/result>

## Claim being tested
One scoped sentence.

## Primary sources
Paper version, official repository commit/release, model/data identifiers.

## Original protocol
Task, observations, actions, data, algorithm, compute, seeds, metrics.

## Our deviations
Every hardware/software/config difference and expected consequence.

## Success criterion chosen before running
Numerical metric, uncertainty, and qualitative failure boundary.

## Baselines and ablations
What is held fixed and what changes.

## Results
All seeds/trials, not only best; confidence intervals and failure categories.

## Interpretation
What evidence supports, does not support, and next discriminating experiment.

## Artifacts
Commits, lockfiles, commands, configs, logs, checkpoints, videos, hashes.
```

17.10 Capstone A: tabular foundations

Goal: show you understand learning before deep networks.

1. Modify `examples/tabular_q_learning.py` to implement SARSA.
2. Compare SARSA and Q-learning in a grid with a costly cliff.
3. Sweep ϵ and three seeds.
4. Plot or tabulate return, cliff entries, and convergence episodes.
5. Explain why on-policy exploration can produce a safer route during learning.

Deliverable: two-page report plus runnable code and raw seed results.

17.11 Capstone B: PPO mechanism study

Goal: connect the clipped objective to observed optimization.

1. Use `examples/ppo_clip_demo.py` to map advantage sign and probability ratio to the surrogate objective.
2. Add cases at ratios 0.5 through 1.5.
3. Predict the flat regions before running.
4. In a small standard control environment, compare two clip values while holding seeds and budget fixed.
5. Record approximate KL, entropy, return, and update stability.

Deliverable: reproduction card explaining what clipping does and does not guarantee.

17.12 Capstone C: reproduce the Microduck pipeline

Goal: demonstrate end-to-end robot-RL understanding without claiming a five-iteration policy is trained.

1. run CPU tests and task registry;
2. run a 64-environment, five-iteration smoke train;
3. inspect resolved environment and agent configuration;
4. play zero agent and one available checkpoint;
5. classify the observation, action, command, and reward terms;
6. export via the official normalizer-baking path;
7. inspect ONNX shape and run CPU deployment rehearsal; and
8. state why random-looking viewer direction is command sampling, why obstacles are not avoided, and why body-pose intent is not trained by the main recipe.

Deliverable: exact commands, log/artifact paths, one annotated rollout, and a contract diagram.

17.13 Capstone D: modern locomotion ablation

Choose one:

- actuator calibration versus randomization width;
- privileged critic versus symmetric actor/critic information;
- observation history versus feed-forward policy;
- PPO versus SAC at matched environment steps and wall time; or
- existing planner + velocity policy versus perceptive end-to-end policy.

Before running:

1. identify one primary paper and official codebase;
2. freeze the baseline contract;
3. define at least three training seeds;
4. define held-out physics/commands;
5. define human-visible failure categories; and
6. estimate compute and stopping rule.

Deliverable: a mini-paper with negative results preserved.

17.14 Capstone E: Jump Rover staged handoff

This capstone starts with evidence, not RL installation.

Phase 1: hardware truth

- finish and measure mechanics;
- validate motor/servo sign, range, current, and thermal behavior;
- validate realtime watchdog, limits, stop, and communication;
- accept a tethered classical/scripted balance/drive baseline.

Phase 2: digital twin

- record mass, inertia, geometry, contacts, actuator response, delay;
- create simulator and four-action environment;
- replay hardware excitation in simulation;
- define held-out identification trials.

Phase 3: learning

- train bounded residual or command policy in simulation;
- compare against the frozen baseline;
- export a versioned model with schema and WCET;
- perform sim-to-sim, processor-in-loop, bench, tether, then floor gates.

Phase 4: autonomy

- onboard perception produces confidence-bearing world state;
- local planner/behavior tree selects exact skills;
- cloud agent proposes semantic plans only;
- realtime MCU retains motor safety and network-loss response.

Deliverable: signed gate evidence and rollback artifact at every phase. The project-specific handoff document in the Jump Rover repository contains the current interfaces and blockers.

17.15 A weekly research-study routine

For one paper per week:

1. Monday: scope pass and claim sentence.
2. Tuesday: derive one central equation in your own words.
3. Wednesday: inspect official code/config corresponding to that equation.
4. Thursday: run one tiny reproduction or dependency-free analogy.
5. Friday: write evidence, limitations, and one project-relevant experiment.

Reading ten abstracts is less useful than tracing one claim through equation, implementation, configuration, and result.

17.16 Final self-assessment

You are ready to begin independent robot-RL research when you can:

- reject an impossible task definition before training;
- derive a Bellman or policy-gradient update and implement a small version;
- justify algorithm choice from interaction/data constraints;
- separate simulator truth, actor observation, critic privilege, and runtime sensor data;
- design baselines and ablations before seeing results;
- report uncertainty and failure clusters;
- preserve a deployable observation/action/timing contract; and
- say “the evidence does not establish that” without losing enthusiasm for the next experiment.

Continue with the detailed paper seminars, then use Chapter 20 for terminology and worked checks before returning to the capstone whose deliverable you cannot yet produce.

17.17 Folded capstone reference checks

Show reference mechanisms and completion criteria for Capstones A–E

These are not single “correct” experiment results. They are reference checks that make an incomplete or unsupported submission visible.

Capstone A — SARSA. The on-policy target uses the action actually selected by the exploratory policy at the next state:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_t + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t) \right].$$

A minimal update helper is:

```
def sarsa_update(q, s, a, reward, next_s, next_a, alpha, gamma):
    target = reward + gamma * q[next_s][next_a]
    q[s][a] += alpha * (target - q[s][a])
```

Q-learning instead uses `max(q[next_s])`. Near a costly cliff, SARSA learns about the consequences of its continuing exploration, so it can prefer a route with more clearance while training. Completion requires all requested seeds and cliff-entry counts; one attractive trajectory is not the result.

Capstone B — PPO clipping. For positive advantage, the surrogate becomes flat above ratio $1 + \epsilon$; for negative advantage it becomes flat below $1 - \epsilon$. Clipping the objective does not impose a hard bound on the final network update because all samples share parameters and the optimizer takes multiple minibatch steps. A complete report shows the predicted piecewise curve, measured approximate KL and entropy, both clip settings under the same budget, and every seed.

Capstone C — Microduck pipeline. A passing submission distinguishes a smoke test from learned locomotion, records the actor input as 61 values and the action as 14 values, uses the official export path that bakes observation normalization into ONNX, and rehearses that artifact on CPU. It explicitly states that the main actor has no obstacle sensor or global-goal input and that its body-pose reward has zero weight. Missing any one of configuration, checkpoint, normalizer, command ordering, or timing leaves the deployment contract unproved.

Capstone D — locomotion ablation. The causal comparison changes exactly one named factor, holds training and evaluation budgets fixed, uses at least three seeds, and reports held-out conditions plus failure categories. If PPO and SAC receive different wall time, replay warm-up, observations, or reward code, label those as confounds rather than attributing the entire difference to the algorithm name. A negative result with complete artifacts passes; a best-seed claim without uncertainty does not.

Capstone E — Jump Rover. Each phase needs a measurable exit gate. Examples are a recorded watchdog stop bound and thermal envelope for hardware truth; held-out excitation error for the digital twin; matched-baseline success and processor-in-loop worst-case execution time for learning; and demonstrated safe behavior under cloud timeout, stale perception, and invalid skill IDs for autonomy. A cloud-generated plan is never evidence that the real-time motor safety layer works. Every gate must name its artifact and rollback version.

18. Detailed Paper Seminars: Impactful Robot-Intelligence Research

This chapter is a guided reading course, not a list of fashionable titles. The selected works introduced ideas that materially shaped open robot-learning practice, provided unusually useful experimental evidence, or opened a major current direction. They span locomotion, world models, large-scale real-robot RL, imitation, cross-embodiment data, foundation policies, and language-guided planning.

"Most impactful" is necessarily a judgment. The selection criteria are:

- the work changed how important robot-learning problems are formulated;
- the central mechanism can be explained and tested;
- the evidence is substantial and scoped clearly;
- a primary paper and preferably official artifacts exist; and
- the idea teaches something transferable to Microduck, Jump Rover, or another real robot.

For each seminar, read the paper itself after this guide. This chapter explains the map; it does not replace methods, appendices, or code.

18.1 Seminar 1 — Massively parallel PPO for locomotion

Primary work

- Learning to Walk in Minutes Using Massively Parallel Deep Reinforcement Learning (2021)
- Official `legged_gym`
- Official RSL-RL

The problem before the paper

Legged locomotion policies could take hours or days to train, which made reward, curriculum, and dynamics experiments slow. Physics simulation was often split between CPU environments while neural-network updates ran on GPU, creating data movement and throughput bottlenecks.

Central idea

Run thousands of physics environments in parallel on one GPU and collect short on-policy fragments from all of them:

```
environment 1:     o0 a0 r0 ... o23 a23 r23
environment 2:     o0 a0 r0 ... o23 a23 r23
...
environment 4096:  o0 a0 r0 ... o23 a23 r23
                              |
                              v
                    one large PPO batch
```

The time horizon per environment can be short because the aggregate batch is large. For Microduck:

$$4096 \times 24 = 98,304 \text{ transitions per iteration.}$$

This changes wall-clock economics. It becomes reasonable to discard stale PPO data because almost 100,000 fresh transitions arrive each iteration.

Curriculum insight

The work also uses a terrain curriculum framed like a game: environments move toward harder terrain when the policy makes sufficient progress and toward easier terrain when it fails. This concentrates samples near the current competence frontier.

The general principle is not "always increase difficulty." It is:

$$\text{sample where success is neither nearly zero nor nearly certain.}$$

If every rollout fails instantly, the learning signal cannot distinguish useful partial behavior. If every rollout succeeds, it supplies little pressure to improve robustness.

What the evidence establishes

The paper reports dramatic wall-clock training speed in its ANYmal/Isaac Gym setup, analyzes components in the massively parallel regime, and demonstrates sim-to-real locomotion. The open stack influenced a broad family of legged-robot projects.

What it does not establish

- PPO is always more sample efficient than off-policy methods.
- Any 4,096-environment recipe fits an 8 GB GPU.
- A fast training curve transfers without actuator/system identification.
- Reward and curriculum values copy across robot size or morphology.

Code-reading route

In RSL-RL, trace:

```
runner -> rollout storage -> returns/advantages -> PPO minibatches
       -> actor/critic update -> new rollout
```

In Microduck, match this to `num_steps_per_env=24`, environment count, PPO epochs, and minibatches. Calculate samples and optimizer exposures before launching a run.

Reproduction exercise

Run the same small locomotion task at 64, 256, and the largest feasible environment count. Keep total environment transitions approximately fixed. Report:

- simulator frames/second;
- GPU memory;
- wall time;
- policy updates;
- return across at least three seeds; and
- whether optimization behavior changes with batch size.

This tests the throughput insight without pretending to reproduce ANYmal.

18.2 Seminar 2 — Rapid Motor Adaptation

Primary work

- RMA: Rapid Motor Adaptation for Legged Robots (2021)
- Official project page

The problem

A robust policy trained under domain randomization must act reasonably across many dynamics. But the best action on ice differs from the best action on high friction. If the policy can infer the current condition, it can adapt rather than compromise.

The true environment parameter vector might contain friction, payload, motor strength, and terrain compliance:

$$e_t = [\mu, m_{payload}, k_{motor}, \ldots].$$

Those quantities are easy to read from simulation but usually unavailable on the real robot.

Architecture

RMA trains a base policy with a compact environment encoding:

$$a_t = \pi(o_t, z_t).$$

During a privileged training phase, an encoder can derive z_t from simulator environment information. A deployment adaptation module instead predicts it from recent observation/action history:

$$\hat{z}_t = g(o_{t-H:t}, a_{t-H:t-1}).$$

The history contains the consequence of prior actions. For example, the same motor command produces different acceleration under different payload or friction.

Why a latent instead of explicit parameter regression?

The policy needs a control-useful summary, not necessarily an accurate human parameter report. Several physical changes can have similar short-term effects, and some are not separately identifiable from proprioception. A learned latent can represent equivalence classes relevant to action.

Training logic

The idea is teacher-student distillation across information sets:

```
simulation parameters -> privileged encoder -> target latent
history                -> adaptation module -> predicted latent
                                          |
observation + latent -> base policy -> action
```

The deployment module learns to reproduce a latent whose use was already made valuable by the base policy.

Evidence and impact

The paper evaluates simulation-trained RMA on a Unitree A1 over diverse real terrains without real-world policy fine-tuning. The architecture popularized fast history-based adaptation as a practical alternative/complement to fixed robustness.

Limitations and audit questions

- Adaptation can only infer properties that affect observed history.
- The first moments after reset have insufficient history.
- Latent prediction can lag a sudden surface change.
- A training distribution missing a real failure mode cannot confer magic adaptation.
- History length, encoder latency, and sensor bias affect deployment.

Microduck experiment

Randomize one identifiable factor, such as actuator strength. Train:

1. a feed-forward robust baseline;
2. the same actor with observation history; and
3. a privileged-latent teacher plus history student.

Evaluate on held-out fixed strengths and a within-episode strength change. Plot tracking error versus time after the change. This distinguishes average robustness from adaptation speed.

18.3 Seminar 3 — Dreamer and physical robot world models

Primary works

- DayDreamer: World Models for Physical Robot Learning (2022)
- DreamerV3: Mastering Diverse Domains through World Models (2023)
- Official DreamerV3 code

The problem

Model-free RL may need many environment interactions. That is tolerable in a fast simulator but costly on physical hardware. A learned world model can use each transition to predict many aspects of future experience, then train a policy through imagined rollouts.

Recurrent state-space model

Dreamer-family methods use a compact latent state containing a deterministic memory and a stochastic component. At a high level:

$$h_t = f(h_{t-1}, z_{t-1}, a_{t-1}),$$

$$z_t \sim q(z_t \mid h_t, o_t).$$

- h_t carries recurrent history;
- z_t represents uncertain current information;
- posterior q incorporates the real observation.

The model learns to predict observations/features, rewards, and continuation. A prior predicts z_t without seeing o_t, which is what imagined rollouts need.

Model-learning objective in words

The objective balances:

1. reconstruct/predict information from observations;
2. predict reward and whether the episode continues; and
3. make the predictive prior agree with the observation-informed posterior without collapsing the representation.

The divergence term is commonly based on KL divergence. Here it asks how different two latent distributions are—not whether the robot's physical path is close in meters.

Imagination

Starting from a posterior latent inferred from real data:

```
latent now -> actor samples action -> world model predicts latent next
           -> predicted reward/value -> repeat in imagination
```

Actor and critic learn from these compact imagined trajectories. No future pixels must be rendered during policy learning.

DayDreamer evidence

DayDreamer applies the approach online to four physical robot settings, including legged recovery/walking, visual manipulation, and visual navigation. Its importance is showing that world-model learning can operate directly on real robot experience rather than only simulated/game benchmarks.

DreamerV3 evidence

DreamerV3 emphasizes a single configuration across more than 150 evaluated tasks and introduces scale-robust transformations/normalization. It provides strong general evidence for the family, not proof that one model captures every contact-rich robot accurately.

Failure analysis

Inspect separately:

- reconstruction/prediction quality;
- reward prediction calibration;
- continuation/termination prediction;
- latent rollout error versus horizon;
- imagined policy success versus real rollout success; and
- uncertainty under out-of-distribution actions.

A visually plausible prediction can have the wrong contact timing, while an unrecognizable decoded image might still retain adequate control information.

Project exercise

Before training a world model on a robot, fit a one-step predictor to logged proprioception. Evaluate one-step, 5-step, and 25-step open-loop errors on held- out action sequences. Compare against a persistence baseline $\hat{s}_{t+1} = s_t$. This establishes whether learning dynamics adds signal before adding actor optimization.

18.4 Seminar 4 — TD-MPC2: planning in a learned latent model

Primary work

- TD-MPC2: Scalable, Robust World Models for Continuous Control (2023)
- Official code

How it differs from Dreamer

Both learn latent dynamics, but TD-MPC2 emphasizes local trajectory optimization at decision time. It learns an encoder, latent dynamics, reward, value functions, and a policy prior. Candidate action sequences are refined and scored in latent space.

Conceptually, for horizon H:

$$\text{score}(a_{t:t+H-1}) = \sum_{k=0}^{H-1} \gamma^k \hat{r}(z_{t+k}, a_{t+k}) + \gamma^H \hat{V}(z_{t+H}).$$

The learned value estimates what happens after the short explicit planning horizon. This is a common model-based pattern: model near-term consequences, bootstrap the distant future.

Planning loop

1. encode current observation/history into latent state;
2. initialize a population of action sequences, helped by the learned policy;
3. roll sequences through latent dynamics;
4. score them with predicted reward and terminal value;
5. update the proposal distribution toward high-scoring sequences;
6. execute only the first action; and
7. observe and replan.

Evidence

The paper reports one set of hyperparameters across 104 online RL tasks in four domains and investigates scaling a multi-task agent. This supports the method's breadth within that suite.

Engineering tradeoff

A feed-forward PPO actor performs one network pass. TD-MPC2 performs several model/value evaluations over candidate sequences per action. Compare:

data efficiency versus runtime planning cost and model risk.

For a 20 Hz arm, planning may fit comfortably. For a 100 Hz balance loop on a small processor, measure worst-case execution before choosing it.

Reproduction exercise

Use the official code on one supported benchmark. Then halve and double the planning horizon while keeping training checkpoint fixed. Measure return and inference latency. Explain why a longer horizon can either help foresight or hurt through compounded model error and missed deadline.

18.5 Seminar 5 — QT-Opt and large-scale real-world RL

Primary work

- QT-Opt: Scalable Deep Reinforcement Learning for Vision-Based Robotic Manipulation (2018)

Why this older paper remains important

QT-Opt is an influential counterexample to the idea that robot policies must always learn in simulation or by imitation. It scales off-policy Q-learning to hundreds of thousands of real grasp attempts and closed-loop visual control.

Task formulation

The observation is camera-based; the continuous action describes gripper motion and gripper command; sparse success indicates a completed grasp. The learned Q-function estimates long-horizon success probability/value from observation and action.

Continuous action prevents enumerating $\arg\max_a Q(s, a)$. QT-Opt uses the Cross-Entropy Method (CEM), a sampling optimizer:

1. sample action candidates from a distribution;
2. evaluate each with Q;
3. retain an elite high-value fraction;
4. refit the action distribution to elites; and
5. repeat, then execute a high-value action.

This makes Q-learning possible without a separate deterministic actor, at the cost of several Q evaluations per control decision.

Distributed data and training

The system separates robot collection, replay storage, distributed training, and deployment. Off-policy learning lets old grasp attempts remain useful as new policies collect better data.

Evidence

The paper reports use of more than 580,000 real grasp attempts and high success on its unseen-object evaluation. It documents learned closed-loop behaviors such as regrasping and object repositioning that were not individually programmed.

Limitations and modern reading

- The data/robot fleet budget is far beyond a hobby project.
- Grasp success is narrower than general manipulation.
- The exact camera/action distribution defines generalization.
- A percentage on unseen objects does not describe every failure severity.
- CEM inference has a compute/latency cost.

The lasting lesson is a systems one: replay-based RL can turn a growing robot fleet log into policy improvement when task reward is reliable and collection is automated.

Small-scale exercise

In simulation, compare a discrete grid over a 2D continuous action with CEM optimization of the same learned/known Q surface. Measure solution quality versus Q evaluations. This isolates action optimization from the cost of training a deep visual Q-function.

18.6 Seminar 6 — ACT and Diffusion Policy

Primary works

- ACT: Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware (2023)
- Official ACT code
- Diffusion Policy: Visuomotor Policy Learning via Action Diffusion (2023)
- Official Diffusion Policy code

Shared problem: demonstrations are temporally and multimodally structured

A one-step squared-error behavior-cloning policy assumes a single average action label. Manipulation demonstrations often have:

- temporally coordinated submotions;
- several valid approaches;
- pauses and contact transitions;
- human timing variability; and
- errors that compound when predicting one action at a time.

Both ACT and Diffusion Policy predict **action chunks**, but represent their distribution differently.

ACT architecture

ACT uses a conditional variational autoencoder (CVAE) with a transformer-style policy. During training, an encoder sees the demonstration action sequence and infers a latent style z. The policy/decoder predicts an action chunk conditioned on observation and z.

The CVAE objective has two parts:

$$L = L_{action} + \beta D_{KL}\big(q_\phi(z \mid o, a_{t:t+H}) \| p(z)\big).$$

- L_{action} makes predicted chunks match demonstrations;
- KL regularization makes the training latent distribution compatible with a simple prior available at deployment;
- β controls the tradeoff.

ACT also temporally ensembles overlapping chunk predictions. If several past inferences predicted the action for the current time, the controller combines them with age-dependent weights. This can smooth transitions without making the policy completely open loop.

Diffusion Policy architecture

Diffusion starts with a noisy action trajectory x_K and repeatedly predicts how to remove noise conditioned on observation:

$$x_{k-1} = \text{denoise}_\theta(x_k, o, k) + \text{scheduled noise}.$$

Training corrupts real action sequences with known Gaussian noise and asks the network to predict the noise (or an equivalent denoising target):

$$L(\theta) = \mathbb{E}_{x_0,k,\epsilon}\left[\|\epsilon - epsilon_\theta(x_k, o, k)\|^2\right].$$

This is supervised generative modeling. It can retain distinct modes because sampling begins from different noise rather than forcing one mean action.

At deployment, receding-horizon control predicts a sequence but executes only a prefix before observing again.

Evidence

ACT reports fine-grained low-cost bimanual tasks with relatively small demonstration sets. Diffusion Policy reports results across 12 tasks from four manipulation benchmarks and real-robot evaluations. Each supports action sequence modeling in its task family.

Comparison

Question	ACT	Diffusion Policy
action distribution	latent-variable decoder	iterative generative denoising
inference passes	typically one policy decode	several denoising steps
temporal mechanism	chunks + temporal ensembling	chunks + receding horizon
strength	efficient fine-grained imitation	expressive multimodal trajectories
concern	latent use/collapse, chunk tuning	inference latency, denoising schedule

What to reproduce

On one official simulated task:

1. train a one-step BC baseline;
2. train action chunks at two horizons;
3. hold demonstration split and image encoder fixed;
4. report task success and correction after an injected perturbation;
5. measure inference latency; and
6. visualize predicted action sequences, not only final success.

The research question is whether temporal/multimodal representation explains the improvement—not whether one method has a newer name.

18.7 Seminar 7 — RT-1 and Open X-Embodiment

Primary works

- RT-1: Robotics Transformer for Real-World Control at Scale (2022)
- Open X-Embodiment: Robotic Learning Datasets and RT-X Models (2023)
- Official Open X-Embodiment repository

RT-1 question: does robot policy performance scale with diverse data?

RT-1 studies a high-capacity transformer policy trained on a large real-robot dataset covering many language-described tasks. Images are converted into tokens; language conditions the network; actions are discretized into tokens.

An action dimension a^j within range $[l_j, u_j]$ can be placed into one of B bins. The model predicts a categorical token rather than a raw floating number. De-tokenization maps the selected bin back to a command.

This turns policy learning into sequence classification:

$$L = -\sum_t \sum_j \log p_\theta(\text{action-token}_{t,j} \mid I_{\le t}, \text{instruction}).$$

The model is trained by imitation. It does not learn from a scalar reward in the PPO sense.

TokenLearner and efficiency

Full image patches across time create many tokens. RT-1 uses a learned spatial token reduction so the transformer processes a smaller set of image features. The design highlights a recurring systems problem: a robot policy needs enough visual context while meeting inference rate.

RT-1 evidence

The work studies data/model scaling and generalization on its Everyday Robots fleet/task setup. Its importance is the large, diverse real-robot policy study, not a claim that tokenized actions dominate continuous outputs everywhere.

Open X-Embodiment question: can data transfer across robots?

Robot datasets differ in cameras, arms, grippers, action frames, rates, and task language. Open X-Embodiment assembled standardized data from 22 robot embodiments across 21 institutions, covering 527 skills (reported as 160,266 tasks/variations in the paper's terminology).

The central hypothesis is positive transfer:

target robot performance with multi-robot pretraining > target-only training at comparable target data.

RT-X models adapt RT-1/RT-2-style policies to the combined data.

The hard part is normalization, not file concatenation

Cross-embodiment training must reconcile:

- observation names, image views, and missing sensors;
- absolute versus delta actions;
- joint versus end-effector coordinates;
- action dimension and gripper semantics;
- frequency and temporal horizon;
- language/task taxonomy; and
- dataset imbalance.

A common action space may discard embodiment-specific capability. A very loose space may prevent shared learning. This is an interface research problem.

Evidence and limitations

The Open X paper reports positive transfer for multiple evaluated robots and makes standardized datasets/models available. But 22 embodiments do not uniformly cover all robot types: dataset volume and task diversity are imbalanced, mostly toward manipulation.

Microduck relevance

The main lesson is not to feed manipulation actions into a biped. It is to version a policy-family schema so related Microduck skills can share useful representations. Microduck's stable 61D actor contract enables hot swapping; multi-task pretraining would still need task labels, compatible actions, and balanced sampling.

Reproduction exercise

Choose two compatible datasets from Open X. Write a schema table before training. Compare target-only BC with joint pretraining plus target fine-tuning, matching target examples. Report per-task transfer, including negative transfer. A global average can hide one task getting worse.

18.8 Seminar 8 — From RT-2 to open VLA and flow policies

Primary works and artifacts

- RT-2 (2023)
- Octo (2024) and official code
- OpenVLA (2024) and official code
- π_0 (2024) and official `openpi`
- $\pi_{0.5}$ (2025)
- GR00T N1 (2025) and official code
- SmolVLA (2025) and LeRobot

This is a fast-moving area. The purpose is to understand design axes, not name a permanent winner.

RT-2: action tokens inside a vision-language model

RT-2 co-fine-tunes a pretrained vision-language model on web vision-language tasks and robot trajectories. Robot actions are expressed as tokens in the same autoregressive vocabulary as text.

The hoped-for transfer is:

```
web vision/language knowledge (objects, concepts, relations)
                          +
robot action demonstrations (grounded motor behavior)
                          |
                          v
language-conditioned visual action tokens
```

The paper reports thousands of evaluation trials and improved semantic generalization in its robot setup. It demonstrates that web-scale semantic pretraining can influence robot action selection. It does not show that web text teaches unobserved robot dynamics.

Octo: reusable open policy initialization

Octo is trained on 800,000 trajectories from Open X-Embodiment and is designed to accept different observation/task modalities and adapt to new action spaces. Its transformer uses token groups and readout tokens so downstream users can attach suitable action heads.

Octo's scientific value includes open checkpoints/code and ablations on architecture/data choices. It frames a generalist policy as an initialization to fine-tune, not a universal zero-shot hardware controller.

OpenVLA: an open VLM-to-action recipe

OpenVLA starts from a pretrained language backbone and visual encoders, then trains on 970,000 Open X robot trajectories. It predicts discretized action tokens autoregressively. The 7B model makes modern VLA study possible outside the organizations that developed closed RT-2 models.

Parameter-efficient fine-tuning such as LoRA updates low-rank matrices rather than every full weight matrix:

$$W' = W + BA,$$

where A and B have much smaller rank than W. This reduces trainable parameters, but not necessarily base-model inference memory or latency.

π_0: continuous action chunks with flow matching

π_0 combines a pretrained vision-language backbone with an action expert that generates continuous action chunks using flow matching. Instead of classifying each action into a discrete token, it learns a velocity field that transforms noise into an action trajectory.

In a simplified conditional flow-matching view, interpolate between noise x_0 and data action x_1:

$$x_\tau = (1 - \tau)x_0 + \tau x_1.$$

Train a vector field $v_\theta(x_\tau, \tau, c)$, conditioned on vision/language c, to predict the direction toward data. At inference, numerically integrate the learned field from noise toward a structured action chunk.

The official `openpi` repository makes a particularly valuable systems lesson visible: model code is only one part. Data transforms, image masks, prompt tokenization, action padding, normalization statistics, embodiment config, checkpoint assets, inference server, and robot adapter define actual behavior.

$\pi_{0.5}$: heterogeneous co-training for open-world tasks

$\pi_{0.5}$ adds co-training on heterogeneous examples: robot actions, high-level semantic subtask prediction, object detections, multi-robot data, and web knowledge. The paper evaluates long-horizon manipulation in new homes.

The design suggests that low-level action data alone may not teach robust semantic decomposition. Mixing high-level prediction examples gives the model intermediate supervision. It also complicates attribution: ablations are needed to say which data source produces which capability.

GR00T N1: dual-system VLA for humanoid manipulation

GR00T N1 explicitly describes a dual system:

```
System 2: vision-language reasoning/understanding
                          |
System 1: diffusion-transformer continuous action generation
```

Training mixes real robot trajectories, human video, and synthetic data. The paper reports simulation benchmarks across embodiments and real humanoid bimanual tasks. "Humanoid foundation model" here centers primarily on language-conditioned manipulation; it should not be read as proof of autonomous locomotion safety.

SmolVLA: efficiency and asynchronous inference

SmolVLA studies a smaller, community-oriented model intended for affordable hardware. It also separates slower perception/action prediction from action execution using asynchronous inference.

Asynchrony introduces a control question: the predicted chunk may be based on an older observation. Measure

$$\text{observation age at action execution} = \text{capture} + \text{queue} + \text{inference} + \text{transport delay}.$$

Smooth throughput is not enough if stale actions meet a disturbed robot.

Comparing VLAs responsibly

Record:

Axis	Questions
pretraining	web data, robot hours/trajectories, embodiments, licenses?
action	tokens, regression, diffusion/flow, horizon, frame, rate?
adaptation	full fine-tune, LoRA, new head, frozen backbone?
evaluation	same tasks, splits, cameras, demonstrations, trials?
runtime	parameters, memory, average/WCET latency, asynchronous age?
openness	code, weights, data, exact training recipe, robot adapter?

Never compare one paper's percentage directly to another without protocol alignment.

Reproduction exercise

Use one official open VLA on a supported simulation or dataset. First run its released checkpoint/evaluator. Then fine-tune on a deliberately small target subset. Compare:

- from-scratch BC;
- frozen backbone + new action head;
- parameter-efficient fine-tuning; and
- full fine-tuning if compute permits.

Evaluate in-distribution and on one held-out object/layout. Report model memory, inference time, observation age, and task success.

18.9 Seminar 9 — SayCan, Code as Policies, and VoxPoser

Primary works

- SayCan: Do As I Can, Not As I Say (2022)
- Code as Policies (2022)
- Official Code as Policies code
- VoxPoser (2023)
- Official VoxPoser code

These works focus on high-level semantic planning/grounding. They complement, rather than replace, motor RL.

SayCan: combine usefulness with affordance

An LLM can score which skill description is linguistically useful for an instruction. A learned value/affordance function estimates whether the robot can successfully execute that skill in the current situation.

For candidate skill k:

$$\text{score}(k) \propto p_{LM}(k \mid \text{instruction, history}) \times p_{afford}(\text{success} \mid s, k).$$

The product embodies "Say" times "Can." A sponge-related skill may be useful for cleaning, but its affordance should be low if no sponge is reachable.

This is a powerful architecture lesson: semantic plausibility and physical feasibility are different estimators. Exact skill choices constrain the LLM's output space.

Code as Policies: language models compose APIs

Instead of selecting one listed skill, a code-capable LLM writes programs that call perception and control APIs. Programs can express loops, geometry, conditions, and reusable functions.

The strength is compositionality and inspectable structure. The danger is that generated code has real authority. A safe implementation needs:

- a restricted language/API;
- static validation and resource limits;
- no arbitrary shell/network/device access;
- typed physical units and bounded arguments;
- simulation/dry run;
- runtime monitor and timeout; and
- signed/approved primitives beneath it.

Generated Python with unrestricted motor access is not a safety architecture.

VoxPoser: language to spatial value maps

VoxPoser asks an LLM/VLM to compose 3D voxel maps encoding affordances and constraints. A model-based planner then optimizes a trajectory through those maps.

Example maps for "place the block in the bowl without crossing the laptop":

- attraction around bowl interior;
- grasp affordance around block;
- repulsion around laptop and table collision;
- orientation preference for gripper; and
- path smoothness/feasibility from the motion planner.

This grounds language into geometry rather than asking the LLM to emit every joint target.

Evidence boundary

These papers demonstrate important semantic/planning mechanisms in their robot settings. They do not make language-model output a hard guarantee, solve perception uncertainty universally, or remove the need for local feedback.

Worked Jump Rover design example

Define a fixed skill registry:

```
stop
turn_to_bearing(yaw_rad)
drive_local(distance_m, max_speed_mps)
follow_person(track_id, distance_m, max_speed_mps)
dock(dock_id)
```

For “follow Bruce,” let a cloud agent propose `follow_person(track_id="person.bruce", ...)` only after local identity/ consent resolution maps language to an exact track ID. Local perception updates the track; local planning checks obstacles; realtime control executes bounded motion. If any layer becomes invalid, `stop` remains local.

Reproduction exercise

Build a simulation-only skill selector with ten exact skills. Compare:

1. language-model likelihood only;
2. affordance score only; and
3. their product.

Construct commands where a useful object is absent. Report inappropriate skill selection separately from motor execution failure.

18.10 Seminar 10 — Perceptive locomotion and visual parkour

Primary works

- Learning Robust Perceptive Locomotion for Quadrupedal Robots in the Wild (2022)
- Extreme Parkour with Legged Robots (2023)
- SoloParkour (2024)

The problem

Proprioceptive locomotion reacts after the feet/body experience terrain. Exteroception can see a step or gap before contact, enabling deliberate foot placement or body posture. But real depth is incomplete, noisy, reflective, occluded, and delayed.

Robust fusion rather than blind trust

Perceptive locomotion needs a belief over terrain, not a perfect height map. History and attention/gating can learn when exteroception agrees with proprioception and when to rely more on contact evidence.

Conceptually:

$$b_t = f_\psi(b_{t-1}, o_t^{prop}, o_t^{ext}, a_{t-1}),$$

$$a_t = \pi_\theta(o_t^{prop}, b_t, c_t).$$

b_t is a learned belief state carrying temporal context.

Privileged teacher and student

Simulation can train a teacher from exact terrain geometry. A student receives noisy depth/history and distills the teacher or learns from its experience. The gap to audit is:

```
teacher truth: exact terrain/contact/state
student input: rendered/noisy depth + proprioception
real input: sensor-specific artifacts + calibration + delay
```

Constrained visual learning

SoloParkour explicitly formulates task reward and physical constraints, then uses privileged experience to initialize visual off-policy learning. This is a useful combination of Chapter 6 (off-policy), Chapter 7 (constraints), Chapter 14 (experience), and this chapter’s perception hierarchy.

Evidence

The cited works show increasingly demanding real quadruped terrain/parkour behaviors under their sensors and obstacle protocols. They establish that vision can participate in dynamic locomotion, including on low-cost platforms.

Failure taxonomy

Do not report only obstacle success. Label:

- perception missed obstacle;
- perceived geometry wrong;
- goal/contact plan infeasible;
- policy chose wrong motion despite adequate perception;
- actuator/contact mismatch;
- latency/stale frame;
- body collision other than intended contact;
- recovery succeeded/failed; and
- safety intervention.

Microduck experiment ladder

1. Add one obstacle and an oracle geometric local planner commanding the unchanged velocity policy.
2. Vary tracking accuracy and determine the locomotion controller's command response limit.
3. Add realistic perception noise/dropout to the planner input.
4. Only then compare a new depth-conditioned locomotion actor.
5. Evaluate held-out obstacle geometry and sensor faults.

This ladder determines whether failure belongs to perception, planning, command tracking, or joint-level control.

18.11 Synthesis: five enduring research themes

Across these papers, the durable ideas are:

1. **Scale data generation carefully.** GPU environments, robot fleets, and cross-embodiment datasets change what is learnable, but interfaces and data balance still decide value.
2. **Exploit information asymmetry during training.** Privileged teachers, critics, and world models can improve learning while deployable actors use realistic inputs.
3. **Represent time explicitly.** History, recurrent belief, action chunks, and predictive models address partial observability and temporal coherence.
4. **Separate semantic intent from physical authority.** VLA/LLM planning belongs above bounded local skills and hard realtime safety.
5. **Evaluate the system, not the model name.** Data, normalization, control rate, actuator, planner, hardware limits, and failure recovery define robot intelligence in operation.

Continue with the open-source ecosystem and reproduction labs, which turns these seminars into executable project choices.

18.12 Folded seminar-exercise guidance

Show reference experiment designs for Seminars 1–10

These are solution **structures**, because reproducible measurements—not a predetermined winning number—answer the research exercises.

1. **Parallel PPO:** keep total transitions, task, network, optimizer, and evaluator fixed. Vary only environment count and corresponding iteration count. Report throughput, memory, updates, wall time, and three-seed uncertainty. If return changes, batch/update geometry changed learning even though transition count matched.

2. **RMA:** compare feed-forward robustness, history, and privileged-teacher/ history-student variants with matched actor capacity where possible. Use held-out fixed dynamics plus a mid-episode change; plot tracking error versus time since the change to distinguish robustness from adaptation speed.
3. **World model:** split complete trajectories before creating windows, fit on training trajectories, and roll out held-out action sequences open loop. Compare one-, five-, and 25-step state error against persistence. Never leak overlapping future windows into validation.
4. **TD-MPC2 horizon:** freeze one checkpoint and evaluator, then vary planning horizon. Record success/return together with median, p95/p99, and maximum inference time. A horizon is unusable if it misses the control deadline even when its simulator return improves.
5. **QT-Opt action search:** define a known 2D Q surface with multiple peaks. Compare grid and cross-entropy method (CEM) using equal Q-evaluation budgets; repeat random CEM seeds and report distance/value regret from the known optimum.
6. **ACT/Diffusion:** hold dataset split and visual encoder fixed. Compare one- step BC and two chunk horizons, inject the same perturbation, and measure success, recovery time, action discontinuity, and inference age. Visualize full predicted chunks so "smooth" cannot hide stale open-loop action.
7. **Open X transfer:** first harmonize action frames, units, rates, cameras, and missing fields in a schema table. Match target examples across target- only and pretrain/fine-tune runs; report every task so negative transfer is visible.
8. **Open VLA:** reproduce the released evaluator before fine-tuning. Compare from-scratch BC, frozen backbone, parameter-efficient adaptation, and full adaptation only when compute allows. Fix target splits and report memory, tail latency, observation age, in-distribution success, and held-out object/ layout success.
9. **Language planning:** construct exact skill IDs and local preconditions. Evaluate LM relevance, affordance-only, and their product on present/absent objects. Score invalid selection separately from execution failure and prove timeout/network loss retains local `stop`.
10. **Perceptive locomotion:** progress from oracle geometry + existing velocity policy, through noisy/dropout planning, to a new visual actor. Use the same held-out obstacles and label perception, planning, command tracking, contact, actuator, recovery, and safety failures separately.

One compact run table prevents a throughput exercise from becoming a story:

```
seed,num_envs,total_transitions,updates,fps,gpu_peak_mb,wall_s,eval_return
0,64,REPLACE,REPLACE,REPLACE,REPLACE,REPLACE,REPLACE
0,256,REPLACE,REPLACE,REPLACE,REPLACE,REPLACE,REPLACE
0,MAX_FEASIBLE,REPLACE,REPLACE,REPLACE,REPLACE,REPLACE,REPLACE
```

19. Open-Source Robot-Learning Ecosystem and Reproduction Labs

Open source makes a research claim inspectable, but the repository name is not the experiment. This chapter maps major projects to their layer, shows what to read, and gives a low-cost sequence of reproductions.

Project capabilities and versions change. Links were checked on 2026-08-31; pin a release or commit before reproducing anything.

19.1 Separate the layers before choosing software

```
dataset / demonstrations
          |
environment + simulator + robot/controller model
          |
algorithm + networks + replay/rollout storage
          |
experiment tracking + evaluator
          |
exported model + inference runtime
          |
hardware adapter + realtime safety
```

One repository may cover several layers but rarely all of them. Installing an RL library does not provide a correct robot model; downloading a VLA checkpoint does not provide your camera calibration or action adapter.

19.2 Small general-RL learning tools

Gymnasium

- Official repository
- Role: standardized environment API and classic/control benchmarks.
- Read: reset/step return values, termination versus truncation, spaces, and seeding.
- Use: validate an algorithm idea cheaply before robot simulation.

CleanRL

- Official repository
- Role: single-file deep-RL implementations designed for readability and reproducibility.
- Read: one PPO or SAC file end to end; map each equation to a code block.
- Caution: a concise benchmark implementation is not a robot deployment stack.

Stable-Baselines3

- Official repository

- Role: maintained PyTorch implementations and a convenient baseline API.
- Use: establish a competent algorithm baseline on Gymnasium-compatible tasks.
- Caution: defaults and wrappers are part of the experiment; inspect them.

19.3 Locomotion and GPU simulation stacks

RSL-RL

- Official repository
- Role: robot-focused runners, PPO and related algorithms, models, storage, export utilities, and teacher-student components.
- Read in order: runner → storage → PPO → actor/critic model.
- Microduck use: this is the optimizer/training loop beneath `mjlab` tasks.

`mjlab`

- Official repository
- Role: manager-based robot-learning environments on MuJoCo Warp.
- Read: environment configuration, observation/reward/event managers, and task registration.
- Microduck use: direct environment framework.

MuJoCo Playground

- Official repository
- Paper
- Role: GPU MuJoCo environments across locomotion, manipulation, and vision; multiple learning backends.
- Study: compare one task's observation/action/randomization/export path with Microduck's.

Isaac Lab

- Official repository
- Role: robot-learning framework on Isaac Sim with sensors, actuator models, environment design, randomization, RL, and imitation workflows.
- Study: manager-based versus direct environments, actuator configuration, and the current supported version matrix.
- Caution: Isaac Sim, driver, CUDA, and extension versions form a large compatibility surface.

`legged_gym`

- Official repository
- Role: influential Isaac Gym vectorized locomotion tasks associated with Learning to Walk in Minutes.
- Study: reward preparation, terrain curriculum, command sampling, and task registry.
- Caution: it targets legacy Isaac Gym and a specific RSL-RL version. Follow its pinning instructions; do not combine newest packages by guesswork.

Genesis

- Official repository
- Role: Pythonic robotics physics platform with parallel simulation.
- Study: supported solvers, robot import, vectorized control API, benchmark scripts, and released—not announced—features.
- Jump Rover use: one candidate for the future isolated four-action sandbox, after measured hardware/digital-twin gates pass.

19.4 Manipulation environments and benchmarks

robosuite

- Official repository
- Paper
- Role: modular MuJoCo robot/controller/task framework with demonstrations and multi-modal sensors.
- Study: how action meaning changes among joint velocity, inverse kinematics, and operational-space control.

ManiSkill

- Official repository
- Role: GPU-parallel SAPIEN manipulation environments and baselines spanning RL, imitation, model-based learning, and VLA evaluation.
- Study: heterogeneous parallel scenes, state versus visual observations, and real-to-sim/sim-to-real protocols.
- Caution: asset licenses can differ from code license.

RLBench

- Official repository
- Paper
- Role: many language-described manipulation tasks in CoppeliaSim/PyRep.
- Study: task demonstrations, variation definitions, observation modes, and success conditions.

LIBERO

- Official repository
- Paper
- Role: lifelong/multitask manipulation benchmark with controlled spatial, object, goal, and task distribution shifts.
- Study: what a split is intended to test and whether a policy learns transfer or memorizes benchmark regularities.
- Caution: the official repository notes its main focus is imitation rather than sparse-reward RL.

19.5 Robot data and foundation-policy code

LeRobot

- Official repository
- Role: dataset format, hardware integrations, policy training/evaluation, and pretrained-policy ecosystem.
- Study: dataset features/video timing, normalization stats, policy config, robot adapter, and evaluation loop.

Open X-Embodiment

- Official repository
- Paper
- Role: standardized multi-institution robot datasets and RT-X study.
- Study: embodiment/action transforms and dataset sampling weights.

Octo

- Official repository
- Role: open generalist policy/checkpoints intended for downstream adaptation.
- Study: token groups, task definitions, action heads, and fine-tuning config.

OpenVLA

- Official repository
- Role: open VLA code/checkpoints and fine-tuning path.
- Study: visual encoder fusion, action tokenization, normalization, LoRA, and inference serving.

`openpi`

- Official repository
- Role: official open implementations/checkpoints for π_0, FAST variants, and $\pi_{0.5}$ support.
- Study: `Observation`/action tensor specs, image masks, data transforms, normalization stats, action horizon, config, and serving.
- Caution: the maintainers explicitly frame adaptation to new robots as an experiment, not a guaranteed drop-in result.

Isaac GR00T

- Official repository
- GR00T N1 paper
- Role: open foundation-model artifacts/workflows oriented toward humanoid and generalist manipulation.
- Study: data format, embodiment config, dual-system inference, and simulation benchmark protocol.

SmolVLA

- Paper
- Implementation in LeRobot
- Role: smaller VLA and asynchronous inference designed for more accessible training/deployment.
- Study: observation age, action-chunk queueing, and actual CPU/GPU latency.

19.6 How to inspect an unfamiliar repository

Do not start by running its most expensive command. Answer these questions:

1. What license applies to code, weights, data, and assets separately?
2. Which commit/release matches the paper?
3. What Python, framework, CUDA/driver, and simulator versions are required?
4. Is the environment API Gymnasium-old, Gymnasium-new, or custom?
5. What are observation/action shapes, units, frames, rates, and normalization?
6. Where are reward, termination, reset, and randomization defined?
7. What command recreates a released evaluation without training?
8. Are pretrained artifacts content-addressed or mutable links?
9. What external downloads are executed?
10. Does the code send telemetry or require an account?

Then create an isolated environment. Do not install a second simulator stack into a working Microduck lock environment.

19.7 Reproduction ladder

Use the cheapest rung that can falsify the current assumption:

```
read schema/config
      |
import/runtime smoke
      |
one environment + zero/random action
      |
released checkpoint evaluation
      |
short training smoke
      |
one benchmark reproduction across seeds
      |
one ablation
      |
new robot simulation
      |
processor-in-loop / hardware gates
```

Skipping from repository clone to hardware merges dependency, model, policy, timing, and safety failures into one mystery.

19.8 Lab sequence A — algorithm truth

1. Run this book's bandit, value iteration, Q-learning, and PPO clipping programs.
2. Read one matching CleanRL implementation.
3. Use a Gymnasium classic-control task and three seeds.
4. Reproduce a documented baseline before changing it.
5. Implement one controlled ablation.

Outcome: understand algorithm mechanics independent of robot complexity.

19.9 Lab sequence B — vectorized robot learning

1. Run Microduck CPU tests and zero-agent playback.
2. Run a 64-environment/five-iteration PPO smoke.
3. Trace one reward and observation through `mjlab` managers.
4. Inspect RSL-RL rollout batch shape and PPO update counts.
5. Export and run CPU ONNX rehearsal.
6. Repeat one supported MuJoCo Playground task in a separate environment.
7. Compare environment and deployment contracts, not reward numbers.

Outcome: understand a complete simulator-to-runtime motor-policy pipeline.

19.10 Lab sequence C — robot imitation and data

1. Inspect one LeRobot dataset in a browser/tool without training.
2. Write its schema/dataset card.
3. Train a simple BC baseline on a small split.
4. Evaluate on fixed held-out episodes.
5. Compare one-step versus chunked action prediction.
6. Inject an observation perturbation and measure recovery.
7. Only then try ACT, Diffusion Policy, or a VLA.

Outcome: separate data quality/action representation from model scale.

19.11 Lab sequence D — foundation-policy adaptation

Choose Octo, OpenVLA, `openpi`, GR00T, or SmolVLA based on supported hardware, compute, and license.

1. Pin paper-matching repository commit and checkpoint hash.
2. Run the official evaluator on one supported task.
3. Measure model memory, average latency, p95/p99 latency, and action age.
4. Visualize observation and de-normalized action samples.
5. Fine-tune on a tiny target dataset with a fixed validation/test split.
6. Compare against target-only BC at matched target data.
7. Test one held-out object/layout and one sensor fault.
8. Document which “generalist” behavior transferred and which did not.

Outcome: evaluate pretraining as a hypothesis rather than an identity.

19.12 Lab sequence E — language planning with safe skills

1. Create a simulator-only registry of exact, typed skills.
2. Implement local preconditions and deterministic stop.
3. Let an LLM rank/propose only registered skills.
4. Add an affordance/feasibility score.
5. Test absent objects, stale perception, conflicting instructions, timeout, and network loss.
6. Log proposal, validation decision, execution result, and fallback.
7. Do not connect generated code or free-form motor values to hardware.

Outcome: gain semantic planning without surrendering physical authority.

19.13 Dependency and provenance record

For every external project, preserve:

```
source_url: OFFICIAL_REPOSITORY_URL
commit: full_commit_sha
paper: exact_url_and_version
license: code / weights / data / assets
environment: lockfile_or_container_digest
accelerator: GPU, driver, CUDA/framework
command: exact_reproduction_command
config: resolved_config_path_and_hash
checkpoint: source_and_sha256
dataset: version/revision_and_split
local_changes: patch_or_clean
result: raw_metrics_and_evaluator_commit
```

This turns “I ran the GitHub project” into inspectable evidence.

19.14 Choosing a first open-source project

Goal	Start with	Avoid starting with
learn RL equations/code	book examples + CleanRL/Gymnasium	billion-parameter VLA
learn robot PPO/sim-to-real	Microduck + RSL-RL/mjlab	unmeasured custom hardware
learn manipulation environments	robosuite or ManiSkill	real-arm online exploration
learn demonstrations	LeRobot + BC	offline RL before dataset audit
study lifelong transfer	LIBERO	claiming real-world generality from one split
study VLA adaptation	Octo/OpenVLA/SmolVLA on supported benchmark	raw motor deployment
study world models	DreamerV3 or TD-MPC2 official benchmark	long-horizon hardware planning first
study language planning	SayCan-style typed skill simulator	unrestricted generated code on robot

The smallest project that answers your question is the fastest route to genuine understanding.

Continue with the glossary and worked problems, then select one reproduction ladder whose success criterion you can state before running it.

19.15 Folded lab completion rubric

Show reference evidence for Lab sequences A–E

- **A — algorithm truth:** the small implementation and reference implementation agree on hand-calculated updates; a documented baseline is reproduced across three seeds; one ablation changes a single mechanism and reports uncertainty.
- **B — vectorized robot learning:** tests, smoke training, resolved manager terms, rollout tensor shape, ONNX export, and CPU rehearsal are connected by exact paths/hashes. The five-iteration result is labeled pipeline evidence, not locomotion performance.
- **C — imitation and data:** a dataset card precedes training; splits prevent episode/scene leakage; one-step and chunked BC use matching data; the perturbation test reports recovery and failure, not only nominal loss.
- **D — foundation adaptation:** official checkpoint evaluation passes before fine-tuning; target-only BC and adaptation see the same target examples; model memory, tail latency/action age, in-distribution and held-out results, licenses, and failures are preserved.
- **E — safe language planning:** the model can select only offered exact IDs; local schema/preconditions reject absent/stale cases; every effect has timeout/cancel/fallback; generated prose/code and raw motor values have zero execution authority.

For every sequence, commit a provenance record like Section 19.13 and a one-command evaluator. “Repository installed” is setup evidence, not a lab result.

20. Glossary and Worked Problems

Use this chapter as both a reference and a self-test. Try each problem before reading its solution. The point is not to perform arithmetic quickly; it is to connect the arithmetic to a real robot decision.

20.1 Plain-language glossary

Action

The numeric decision produced by a policy at one time step. Microduck's main policy outputs 14 relative joint-position targets.

Actor

The neural network that selects actions. It is the part exported for deployment.

Actuator

Hardware that produces motion or force, plus its control electronics. A Dynamixel XL330 servo is an actuator. In simulation, an actuator model converts the policy's target into realistic force/torque behavior.

Advantage

An estimate of how much better or worse a sampled action was than the critic expected in that situation. Positive advantage encourages the action; negative advantage discourages it.

BAM

Better Actuator Models, the library/model used here to represent XL330 motor electrical behavior, firmware position control, friction, voltage, load sag, and delay more realistically than an ideal position actuator.

Behavior cloning (BC)

Supervised learning from demonstration pairs: given the observation, predict the demonstrator's action. BC does not require a reward or Bellman update and is therefore imitation learning, not by itself reinforcement learning.

Bellman backup

An update based on "immediate reward plus discounted value of what follows." Dynamic programming, TD learning, Q-learning, and many critics use different forms of this recursive idea.

Batch and minibatch

A batch is a collection of training samples processed together. PPO collects a full rollout batch, shuffles it, and divides it into smaller minibatches for optimization.

Bootstrapping

Updating an estimate using another current estimate, such as training $V(s_t)$ toward $r_t + \gamma V(s_{t+1})$. It enables learning before an episode ends but can propagate estimation error.

Checkpoint

A saved training state such as `model_2000.pt`. It normally includes actor, critic, observation normalizers, optimizer state, and counters, so it can be evaluated or resumed.

Command

A requested behavior supplied to the policy as input, such as forward speed, turn rate, or head pose. The command expresses intention; the action expresses how the joints should move now.

Coordinate frame

The origin and axis directions used to describe a vector. A velocity in the robot body frame is different from the same three numbers in the world frame.

Critic

The training-only neural network that estimates expected future return. It helps estimate advantages and may use privileged simulator information.

Curriculum

A schedule that changes difficulty, command range, reset distribution, reward weight, or another task property after the policy learns an easier stage.

Decimation

The number of physics substeps for which one policy action is held. A 5 ms physics step with decimation 4 produces one policy decision every 20 ms.

Degree of freedom (DoF)

One independent direction of movement. A simple hinge joint has one rotational DoF. “14 actuated joints” means the policy controls 14 motorized DoFs.

Domain randomization (DR)

Sampling plausible physical parameters and disturbances during training—such as mass, friction, voltage, sensor bias, and delay—so the policy learns a family of robots rather than one perfect simulation.

Distribution shift

A difference between the data distribution used to train a model and the situations encountered during evaluation/deployment. An imitation policy can create states absent from expert data; an offline critic can be queried on actions absent from its log.

Diffusion policy

A generative policy that starts from noise and iteratively denoises a conditioned action or action sequence. This can represent several distinct valid action modes but requires multiple inference steps.

Entropy

A measure of uncertainty/spread in the policy's action distribution. PPO uses an entropy bonus to preserve exploration early in learning.

Experience replay

A buffer of past transitions sampled for off-policy updates. Replay reuses expensive data and decorrelates adjacent samples, while creating distribution mismatch between older behavior and the current policy.

Environment

The complete executable task: simulated scene, robot, actions, observations, commands, rewards, reset events, terminations, and curricula.

Episode

One sequence from reset until timeout or another termination. The main walking episode lasts at most 20 simulated seconds.

Epoch

One complete optimization pass over a collected rollout batch. Microduck's main PPO configuration uses five epochs per rollout.

GAE

Generalized Advantage Estimation, a method that combines several temporal-difference errors. Its λ setting trades bias against variance.

Foundation policy / VLA

A broadly pretrained robot policy intended for adaptation across tasks or embodiments. A Vision-Language-Action (VLA) model conditions actions on images and language. Most VLA pretraining is demonstration-based imitation rather than online RL.

Flow matching

A generative method that learns a vector field transporting samples from a simple noise distribution toward a data distribution. Some modern VLAs use it to generate continuous action chunks.

Gradient and backpropagation

A gradient says how a small change in each network parameter changes the loss. Backpropagation applies the chain rule through all layers to compute those gradients efficiently.

Hyperparameter

A setting chosen by the engineer rather than learned as a network weight—for example learning rate, PPO clip value, hidden-layer widths, γ, or λ.

Inference

Running a frozen actor to compute an action. No PPO update, critic training, or backpropagation occurs during inference.

Action chunk

A sequence of several future actions predicted at once. Chunks can improve temporal coherence; executing too much of a chunk open loop can delay feedback correction.

KL divergence

Kullback–Leibler divergence, a measure of how one probability distribution differs from another. PPO monitors approximate KL to detect a policy update that moved too far from the rollout policy.

LoRA

Low-Rank Adaptation, a parameter-efficient fine-tuning method that adds trainable low-rank updates to frozen or mostly frozen weight matrices. It reduces trainable parameter count but does not automatically make base-model inference small or realtime.

Markov Decision Process (MDP)

The mathematical model containing states, actions, transition probabilities, rewards, and a discount factor.

MJCF

MuJoCo's XML model format. It describes robot bodies, joints, actuators, contacts, sensors, meshes, and related physics/model data.

Normalization

Transforming each observation feature using learned statistics so values with different units/scales are numerically comparable. The actor normalizer must be included in deployed ONNX.

Observation

The numeric information given to a policy. It may be only part of the complete environment state.

ONNX

Open Neural Network Exchange, a portable neural-network graph format used to move the trained actor from Python/RSL-RL to the runtime.

On-policy

Learning from experience generated by the current or very recent policy. PPO collects a rollout, updates for a few epochs, discards it, then collects fresh experience.

Off-policy

Learning about a target policy from experience generated by a different behavior policy. Replay-based SAC, TD3, and DQN are off-policy; offline RL is the special fixed-dataset setting with no new corrective interaction.

Partial observability / POMDP

A setting where the current observation omits information needed to predict the future. History, recurrent state, explicit estimation, or an adaptation latent can help when hidden properties leave observable traces.

Policy

The decision rule mapping observations (and commands) to action probabilities or inference actions. Here the policy is an MLP neural network.

PPO

Proximal Policy Optimization, the on-policy actor-critic algorithm used here. Its clipped objective reduces the incentive for one rollout to cause an excessively large policy change.

Privileged observation

Simulator information given to the training critic but withheld from the actor because the real robot cannot measure it equivalently.

System identification

Estimating physical model parameters from measured input/output trajectories. For sim-to-real work, identify and validate a nominal model before choosing randomization around its remaining uncertainty.

Teacher-student learning

A training scheme in which a teacher with richer information or capability provides targets for a deployable student. The student's sensor/input contract still determines what it can reproduce.

Reward and reward shaping

Reward is the scalar learning objective. Reward shaping adds measurable terms that make useful intermediate progress learnable. Poor shaping can create exploits or a different task.

Rollout or trajectory

A time-ordered sequence of observations, actions, rewards, and termination flags sampled by a policy. A rollout batch contains many short sequences from parallel environments.

RSL-RL

The training library that provides PPO, neural-network models, rollout storage, normalization, checkpoints, and ONNX export integration in this stack.

Sim-to-real

Transferring a policy learned in simulation to a physical robot. Success requires consistent interfaces and a simulation distribution that covers the important real dynamics and uncertainties.

State

The full variables needed to describe the environment dynamics. The simulator state is richer than the actor observation.

Tensor

A multidimensional numeric array. Shapes matter: one actor observation is `(61,)`; 4,096 observations form `(4096, 61)`; actor actions form `(4096, 14)`.

Termination

A condition that ends and resets an episode, such as timeout, fall, terrain bounds, or nonfinite state.

Value function

The critic's estimate of expected discounted future reward from a state or critic observation.

World model / model-based RL

A world model predicts future state or a learned latent representation under actions. Model-based RL uses such predictions for planning or policy/value learning. Longer imagination can amplify model error.

WCET

Worst-Case Execution Time: the longest execution time under the defined conditions. A realtime loop must budget a defensible upper bound or percentile and deadline policy, not only average inference speed.

Warp and MuJoCo Warp

Warp is NVIDIA's Python/CUDA kernel framework. MuJoCo Warp implements GPU-parallel MuJoCo-style simulation so thousands of environments can step together.

20.2 Worked problem 1: observation dimensions

Background

The actor concatenates proprioception (the robot's own measured motion/state) and commands. A wrong total means the network/runtime contract is broken.

Problem

Add the main terms: angular velocity 3, projected gravity 3, joint positions 14, joint velocities 14, previous actions 14, and command block 13.

20.3 Worked problem 2: timestep and policy rate

Background

Physics needs small steps for contacts. Neural policy inference can run less often using action decimation.

Problem

Physics timestep is 0.005 s and decimation is 4. Find the policy period and frequency. How long is a 500-step video?

20.4 Worked problem 3: discounted return

Background

Return adds current and future rewards, discounting each later step by γ. Use a short made-up reward sequence to see the calculation.

Problem

At time 0, the next three rewards are 1, 1, and 1. Let $\gamma = 0.99$ and stop after those three terms. What is G_0?

20.5 Worked problem 4: PPO rollout size

Background

One PPO iteration gathers the same number of time steps from every parallel environment.

Problem

How many transitions are collected with 64 environments and 24 steps? With 4,096 environments and 24 steps?

20.6 Worked problem 5: Gaussian tracking reward

Background

A common tracking term is $\exp(-(e/\sigma)^2)$, where e is error and σ is the error scale that still matters.

Problem

For head tracking with $\sigma = 0.5$ rad, calculate the per-joint reward at errors 0, 0.1, and 0.5 rad.

20.7 Worked problem 6: PPO clipping

Background

The probability ratio is new policy probability divided by old policy probability. For a positive advantage, PPO clips the benefit above 1.2 when $\epsilon = 0.2$.

Problem

Old action probability is 0.10, new probability is 0.13, and advantage is +2. Compare unclipped and clipped contributions.

20.8 Worked problem 7: penalty signs

Background

Configuration weight and function output multiply. Always reason about the product.

Problem

Function A returns squared error `+0.4`. Function B returns negative absolute error `-0.4`. What weight sign makes each a penalty?

20.9 Worked problem 8: why uniform sampling misses idle

Background

A continuous uniform random variable can produce values arbitrarily close to zero but has probability zero of producing one exact point.

Problem

Why does sampling each velocity uniformly from a range fail to train the exact deployment command `[0, 0, 0]`?

20.10 Worked problem 9: command versus action

Background

The planner states intent; the motor policy chooses immediate control.

Problem

Classify each value as command, actor observation, or actor action:

```
desired forward speed
measured head joint angle
new left-knee position target
desired head yaw delta
previous actor output
```

20.11 Worked problem 10: obstacle-avoidance architecture

Background

The renderer shows more than the actor receives. A visible object is not a numeric observation.

Problem

You add boxes to flat terrain and penalize collision, but keep the 61D actor unchanged. What can the policy learn, and what is missing?

20.12 Worked problem 11: actor versus critic information

Background

Asymmetric actor-critic training lets the critic see perfect simulator facts while the actor remains deployable.

Problem

Why can true base linear velocity improve the critic without being supplied to the actor? What would go wrong if the actor depended on it?

20.13 Worked problem 12: design one controlled experiment

Background

RL systems have many coupled variables. A good experiment changes one causal hypothesis and preserves a baseline.

Problem

Turn-in-place fails in 60% of trials. Propose a controlled experiment using the command distribution rather than changing PPO and rewards together.

20.14 Worked problem 13: a Q-learning backup

Background

Q-learning trains the current action value toward immediate reward plus the best estimated next action value.

Problem

The old value is $Q(s, a) = 1$. A transition gives reward -1. At the next state, the three action values are $[2, 5, 4]$. Let $\gamma = 0.9$ and learning rate $\alpha = 0.2$. Compute the target, TD error, and updated Q-value.

20.15 Worked problem 14: entropy changes the SAC objective

Background

SAC values expected reward plus α times policy entropy.

Problem

At one state, policy A has expected immediate score 5.0 and entropy 0.1. Policy B has score 4.8 and entropy 0.8. With $\alpha = 0.5$, which has the larger one-step soft objective?

20.16 Worked problem 15: behavior cloning averages modes

Background

Squared-error regression predicts the conditional mean when data contains uncertainty or several labels for the same input.

Problem

At an identical-looking observation, half the demonstrations steer left with action -1 and half steer right with action $+1$. What deterministic scalar action minimizes mean-squared error? Why may it be dangerous?

20.17 Worked problem 16: world-model error over a horizon

Background

Open-loop model predictions feed predicted state back into later predictions.

Problem

A simple position model has a consistent 5 mm forward error per predicted step. Ignoring all nonlinear amplification, what bias accumulates over 25 open-loop steps? Why is this a lower-complexity estimate rather than a guarantee?

20.18 Worked problem 17: mean return hides tail failure

Background

Expected return is not a per-episode safety guarantee.

Problem

Four hardware evaluations have returns [100, 100, 100, −100], where the last is a damaging fall. Compute the mean, median, and non-fall success rate. What must the report say?

20.19 Worked problem 18: stale asynchronous actions

Background

An asynchronous VLA can predict chunks while another loop executes prior actions. The relevant latency is observation age when an action is applied.

Problem

Image capture/preprocessing takes 20 ms, queue wait 15 ms, inference 70 ms, and transport 10 ms. How old is the observation when the first new action arrives? How many 50 Hz policy periods is that?

20.20 Open exercises

1. Derive the maximum 20-second episode length in policy steps.
2. At iteration 1,000, what is the curriculum environment-step counter?
3. Draw the path from `head_pose` command to a reward and to the ONNX input.
4. Explain why body-pose UI controls do not prove the velocity checkpoint learned body-pose control.
5. Design a five-case evaluation battery for exact-zero standing.
6. List three real measurements needed before transferring a policy to a new actuator.
7. Find one pure helper and its regression test in `tests/`; explain why the pure form is easier to test than a full simulator wrapper.

Return to the book index and repeat any lab whose terms or calculation are still unclear.

20.21 Folded solutions

Try each problem before opening its solution. The explanation—not only the final number—is the part to compare with your reasoning.

Problem 1: observation dimensions — show solution

$$3 + 3 + 14 + 14 + 14 + 13 = 61$$

The first 48 values are proprioception/history-like action context, and the last 13 are intention. With 4,096 environments the actor input tensor is `(4096, 61)`.

Problem 2: timestep and policy rate — show solution

$$\Delta t_{policy} = 0.005 \times 4 = 0.020 \text{ s}$$

Frequency is the inverse of period:

$$f = \frac{1}{0.020} = 50 \text{ Hz}$$

Five hundred policy steps last:

$$500 \times 0.020 = 10 \text{ s}$$

This is why a 500-step recorded rollout is a 10-second behavior sample.

Problem 3: discounted return — show solution

$$G_0 = 1 + 0.99(1) + 0.99^2(1)$$

$$G_0 = 1 + 0.99 + 0.9801 = 2.9701$$

The third reward still matters strongly because it is only two steps away. This truncated example is for arithmetic; real GAE also uses a critic estimate beyond a rollout boundary when appropriate.

Problem 4: PPO rollout size — show solution

$$64 \times 24 = 1,536$$

$$4096 \times 24 = 98,304$$

The five-iteration smoke train therefore performs real PPO updates but with a much smaller batch and duration than a full run. It validates plumbing, not locomotion quality.

Problem 5: Gaussian tracking reward — show solution

At zero error:

$$e^{-(0/0.5)^2} = 1$$

At 0.1 rad:

$$e^{-(0.1/0.5)^2} = e^{-0.04} \approx 0.961$$

At 0.5 rad:

$$e^{-(0.5/0.5)^2} = e^{-1} \approx 0.368$$

This shows why a wide standard deviation gives useful gradient far away but makes small errors relatively cheap.

Problem 6: PPO clipping — show solution

$$r = 0.13/0.10 = 1.3$$

Unclipped:

$$1.3 \times 2 = 2.6$$

Clipped ratio is 1.2:

$$1.2 \times 2 = 2.4$$

The objective uses the more conservative value 2.4, removing the incentive to increase this already-favored sample further during that update.

Problem 7: penalty signs — show solution

For A, use a negative weight; for example:

$$-1.0 \times +0.4 = -0.4$$

For B, use a positive weight:

$$+1.0 \times -0.4 = -0.4$$

Using a negative weight for B gives `+0.4`, rewarding the violation. Confirm all logged penalty contributions remain nonpositive.

Problem 8: why uniform sampling misses idle — show solution

The probability of any one exact real-number triple under continuous sampling is zero. “Very small” is also behaviorally different from an exact idle flag. The environment therefore needs an explicit zero-command bucket with nonzero probability.

Problem 9: command versus action — show solution

```
desired forward speed            command (also included in observation)
measured head joint angle        proprioceptive observation
new left-knee position target    action after scale/offset mapping
desired head yaw delta           command (also included in observation)
previous actor output            observation/history context
```

Commands become part of the actor input; they are not outputs chosen by the motor policy.

Problem 10: obstacle-avoidance architecture — show solution

It may learn post-contact reactions or a generally conservative gait. It cannot deliberately steer around an unseen box before contact because no pre-contact obstacle information reaches the actor. Add a local perception/planner that changes twist commands, or create a versioned exteroceptive policy input and matching runtime.

Problem 11: actor versus critic information — show solution

The critic is discarded after training, so privileged velocity can improve its value/advantage estimates without becoming a runtime input. If the actor used perfect velocity but the real robot could not reproduce it with matching noise, delay, frame, and accuracy, deployment observations would come from a different distribution and behavior could fail.

Problem 12: design one controlled experiment — show solution

```
hypothesis:
  turn-in-place examples are too rare

change:
  increase TURN_IN_PLACE_FRACTION from 0.15 to 0.25 only

preserve:
  task, robot, rewards, PPO config, training budget, evaluation commands

verification:
  config test -> CPU suite -> five-iteration smoke -> equal-budget train

metrics:
  turn success, yaw error, translation drift, fall rate, forward tracking

decision:
  keep only if turn improvement is repeatable and forward behavior remains
  inside its acceptance envelope
```

Changing entropy, yaw reward, command range, and sample fraction together would prevent a causal conclusion.

Problem 13: a Q-learning backup — show solution

The greedy next value is 5:

$$y = -1 + 0.9(5) = 3.5.$$

The TD error is target minus old estimate:

$$\delta = 3.5 - 1 = 2.5.$$

Update only partway using α:

$$Q_{new} = 1 + 0.2(2.5) = 1.5.$$

The estimate rises because this transition was better than the old prediction.

Problem 14: entropy changes the SAC objective — show solution

$$A : 5.0 + 0.5(0.1) = 5.05,$$

$$B : 4.8 + 0.5(0.8) = 5.20.$$

The soft objective prefers B despite slightly lower expected task score because it retains more action diversity. This does not mean deployment must sample unsafe random actions; the entropy term shapes training, and execution policy/ safety still must be specified.

Problem 15: behavior cloning averages modes — show solution

For prediction x:

$$L(x) = \tfrac{1}{2}(x + 1)^2 + \tfrac{1}{2}(x - 1)^2 = x^2 + 1.$$

The derivative $2x$ is zero at $x = 0$, the mean. If left and right both avoid an obstacle, the average can drive straight into it. More context or a multimodal policy must distinguish/represent the alternatives.

Problem 16: world-model error over a horizon — show solution

$$25 \times 5 \text{ mm} = 125 \text{ mm} = 0.125 \text{ m}.$$

Real error need not add linearly. A biased position changes future contact and policy inputs, which can amplify, cancel, or redirect error. Receding-horizon replanning replaces imagined state with observation before the full horizon.

Problem 17: mean return hides tail failure — show solution

$$\text{mean} = \frac{100 + 100 + 100 - 100}{4} = 50.$$

The sorted middle pair is 100, 100, so median is 100. Success is 3/4 = 75%. The attractive median hides one severe failure; even the mean does not label its consequence. Report every trial, success/failure category, damage/safety intervention, and uncertainty. Four trials are too few for a reliable tail-risk estimate.

Problem 18: stale asynchronous actions — show solution

$$20 + 15 + 70 + 10 = 115 \text{ ms}.$$

A 50 Hz period is 20 ms:

115/20 = 5.75 policy periods.

The model acts from information nearly six local-control ticks old. A manipulator may tolerate that under slow receding-horizon execution; a disturbed balance loop likely cannot. Measure age/jitter and keep fast stabilization local.

Open exercises 1–7 — show solutions and reference checks

1. At 50 Hz, a 20-second episode contains 20 s × 50 Hz = 1,000 policy steps.
2. Curriculum steps count environment steps per rollout fragment, so iteration 1,000 corresponds to 1,000 × 24 = 24,000 environment steps.
3. `CommandManager` samples `head_pose`; `generated_commands` appends its four values after the 48 proprioceptive values and 3D twist command, so they occupy actor indices 51–54. The `head_pose_tracking` reward reads the same command and the four physical head/neck joint positions. Export rebuilds this observation order, applies the saved normalizer, and embeds the actor in ONNX. This shared source and ordering are why command, reward, and deployment remain consistent.
4. Interface presence is not a learned objective. The six body-pose inputs and UI control can vary, but the main velocity recipe gives `body_pose_tracking` weight zero. A checkpoint might ignore those inputs or respond only through accidental correlations; testable command following requires nonzero task data, reward, and held-out evaluation.
5. A useful exact-zero battery includes: nominal quiet stand; randomized initial tilt; a bounded push followed by recovery; held-out friction and actuator-delay extremes; and a long-duration hold that exposes drift or heating. For every case report fall rate, tilt, position drift, action jitter, saturation, and recovery time across seeds—not just reward.
6. At minimum measure command-to-motion delay/bandwidth, torque or force versus command under load, and friction/deadband/backlash. Battery-voltage response, saturation, speed limits, thermal behavior, and sensor noise are also part of a defensible actuator model.
7. One example is the following pure helper and its focused test:

   ```
   mdp.py::spin_wheel_differential_from_values
   tests/test_spin.py::test_wheel_differential_from_values_is_pure
   ```

   The helper receives tensors and constants directly, so the test can cover its sign, scaling, and gating without constructing MuJoCo, managers, sensors, or GPU state. The simulator wrapper can then be tested separately for correct data extraction.

Primary Sources and Open-Source Study Index

Last reviewed: 2026-08-31.

This index supports the book's research chapters. It deliberately favors primary papers and official repositories. Inclusion means the source is useful for study and its identity/artifact was verified; it is not an endorsement of every claim or a declaration that the method is currently best for every task.

Inclusion and verification policy

A research entry should provide at least:

- a stable paper record (publisher, proceedings, OpenReview, or arXiv);
- identifiable authors and version/date;
- a clearly scoped empirical or theoretical claim; and
- preferably an official project page, source repository, data, or checkpoint.

For every source, readers should verify:

1. paper version and publication status;
2. task, embodiment, observation, action, and data setting;
3. training/evaluation compute and number of seeds/trials;
4. whether code/config/checkpoints/data reproduce the reported path; and
5. license and current dependency compatibility.

Links below point to papers or official author/project repositories, not search result pages.

Foundations and policy optimization

Reinforcement Learning: An Introduction, second edition (2018)

- MIT Press record
- Richard S. Sutton and Andrew G. Barto.
- Establishes: the standard progression from bandits and tabular prediction to function approximation and policy gradients.
- Study caution: it is a general RL foundation, not a current robot deployment manual.

Playing Atari with Deep Reinforcement Learning (2013)

- Paper
- Volodymyr Mnih et al.
- Establishes: an influential deep Q-learning result from pixel observations in discrete Atari action spaces.
- Does not establish: suitability of DQN for high-dimensional continuous robot commands.

High-Dimensional Continuous Control Using GAE (2015)

- Paper
- John Schulman et al.
- Establishes: the generalized advantage estimator and its bias/variance motivation in policy-gradient control.

Trust Region Policy Optimization (2015)

- Paper
- John Schulman et al.
- Establishes: the trust-region motivation that precedes PPO.

Proximal Policy Optimization Algorithms (2017)

- Paper
- John Schulman et al.
- Establishes: PPO surrogate objectives and benchmark evidence for repeated minibatch updates on fresh policy data.
- Study caution: clipping is a practical surrogate, not a universal monotonic improvement guarantee.

Addressing Function Approximation Error in Actor-Critic Methods (TD3, 2018)

- Paper
- Scott Fujimoto, Herke van Hoof, David Meger.
- Establishes: twin critics, delayed actor updates, and target smoothing as responses to continuous-control value error.

Soft Actor-Critic (2018)

- Paper
- Tuomas Haarnoja et al.
- Establishes: an off-policy maximum-entropy stochastic actor-critic method for continuous control.

Constrained Policy Optimization (2017)

- Paper
- Joshua Achiam et al.
- Establishes: policy optimization with expected constraints and stated theoretical properties.
- Does not replace: hard electrical, mechanical, or realtime safety.

Model-based and world-model learning

DreamerV3: Mastering Diverse Domains through World Models (2023)

- Paper
- Danijar Hafner et al.
- Establishes: latent world-model learning and imagined actor/critic training with one reported configuration across diverse evaluated domains.

DayDreamer: World Models for Physical Robot Learning (2022)

- Paper
- Danijar Hafner et al.
- Establishes: online world-model learning directly on four physical robots in the paper's tasks, including quadruped locomotion and manipulation.

- Study caution: real-world sample efficiency does not remove reset labor, hardware wear, latency, or safety constraints.

QT-Opt: Scalable Deep Reinforcement Learning for Vision-Based Robotic Manipulation (2018)

- Paper
- Dmitry Kalashnikov et al.
- Establishes: a distributed off-policy Q-learning system trained from a large mixture of autonomous and logged real-robot grasp attempts.
- Study caution: its data and infrastructure scale are part of the method's empirical setting and must accompany algorithm comparisons.

TD-MPC2 (2023)

- Paper
- Official code
- Nicklas Hansen, Hao Su, Xiaolong Wang.
- Establishes: scalable latent model-predictive control plus value learning across the paper's online continuous-control suite.

Imitation and offline learning

DAgger (2010/2011)

- Paper
- Stéphane Ross, Geoffrey Gordon, Drew Bagnell.
- Establishes: dataset aggregation to address sequential imitation's state distribution shift.

D4RL (2020)

- Paper
- Justin Fu et al.
- Establishes: offline-RL datasets/protocols designed to expose data-coverage challenges.

Conservative Q-Learning (2020)

- Paper
- Aviral Kumar et al.
- Establishes: conservative value regularization for fixed offline data.

Implicit Q-Learning (2021)

- Paper
- Ilya Kostrikov, Ashvin Nair, Sergey Levine.
- Establishes: offline improvement without evaluating arbitrary unseen policy actions during the primary value-learning step.

ACT: Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware (2023)

- Paper
- Official code
- Tony Zhao et al.
- Establishes: action chunking and transformer-style imitation on the reported bimanual tasks.

Diffusion Policy (2023)

- Paper
- Official code
- Cheng Chi et al.
- Establishes: conditional diffusion for multimodal visual action sequences on the paper's manipulation benchmarks.

Sim-to-real and locomotion

Domain Randomization for Visual Transfer (2017)

- Paper
- Josh Tobin et al.
- Establishes: a foundational visual domain-randomization demonstration for sim-to-real object localization/control.

Dynamics Randomization for Robotic Control (2017)

- Paper
- Xue Bin Peng et al.
- Establishes: dynamics-randomized simulation policies transferred to a real object-pushing setup.

Learning Agile and Dynamic Motor Skills for Legged Robots (2019)

- Paper
- Jemin Hwangbo et al.
- Establishes: an influential actuator-aware simulated-training/real-ANYmal deployment pipeline.

Learning to Walk in Minutes (2021)

- Paper
- Official `legged_gym`
- Nikita Rudin et al.
- Establishes: massively parallel GPU locomotion training and terrain curriculum results in the reported setup.

Rapid Motor Adaptation (2021)

- Paper
- Official project page
- Author-linked training code
- Ashish Kumar et al.
- Establishes: a base policy plus history-based adaptation module for the evaluated quadruped conditions.

Robust Perceptive Locomotion in the Wild (2022)

- Paper
- Takahiro Miki et al.
- Establishes: learned fusion of proprioception and unreliable exteroception for reported real quadruped deployments.

Extreme Parkour with Legged Robots (2023)

- Paper

- Cheng et al.
- Establishes: depth-conditioned parkour behaviors on the paper's low-cost quadruped and obstacle protocol.

Wheeled-Legged Navigation and Locomotion (2024)

- Paper
- Joonho Lee et al.
- Establishes: an integrated hierarchical learned locomotion/navigation system evaluated in the stated urban missions.

SoloParkour (2024)

- Paper
- Elliot Chane-Sane et al.
- Establishes: constrained visual locomotion initialized from privileged experience on Solo-12.

MuJoCo Playground (2025)

- Paper
- Official code
- Establishes: an open GPU-accelerated MuJoCo suite with reported robot-learning and sim-to-real examples.

Fast off-policy humanoid locomotion (2025 preprint)

- Paper
- Younggyo Seo et al.
- Establishes: reported FastSAC/FastTD3 recipes at massive simulation scale on the stated GPUs, humanoids, and protocols.
- Study caution: a training-time headline is hardware/configuration dependent.

PACE sim-to-real (active project; 2025 paper record in project)

- Official code and documentation
- Establishes: an open, measurement-driven workflow for identifying actuator and joint dynamics on supported legged systems.
- Study caution: active APIs and platform-specific models must be versioned.

Generalist robot policies and data

RT-1: Robotics Transformer for Real-World Control at Scale (2022)

- Paper
- Anthony Brohan et al.
- Establishes: a transformer policy trained on the paper's multi-task real robot dataset, with image and language inputs and discretized action tokens.
- Does not establish: that language alone supplies geometric safety or that the reported policy transfers unchanged to arbitrary embodiments.

RT-2: Vision-Language-Action Models Transfer Web Knowledge to Robotic Control (2023)

- Paper
- Anthony Brohan et al.

- Establishes: co-fine-tuning vision-language models on robot trajectories and web-scale visual-language data in the reported manipulation evaluations.
- Study caution: inspect which models, weights, robot data, and evaluation interfaces are available before calling a result reproducible.

Open X-Embodiment and RT-X (2023)

- Paper
- Official dataset organization
- Open X-Embodiment Collaboration et al.
- Establishes: a large cross-institution collection of robot datasets and cross-embodiment policy experiments under the paper's unified format.
- Study caution: common serialization does not make action semantics, camera geometry, control rates, or data quality identical.

Octo (2024)

- Paper
- Official code
- Establishes: an open generalist policy pretrained on a large multi-robot demonstration collection and fine-tuned in the evaluated settings.

OpenVLA (2024)

- Paper
- Official code
- Establishes: an open 7B vision-language-action model, training recipe, and evaluated fine-tuning/serving results.

π_0 (2024)

- Paper
- Kevin Black et al.
- Establishes: a VLM plus flow-matching action model evaluated on the stated multi-platform manipulation data and tasks.
- Availability caution: verify which weights, data, and code required for a claimed result are public before planning a reproduction.

$\pi_{0.5}$: A Vision-Language-Action Model with Open-World Generalization (2025)

- Paper
- Official `openpi` code
- Karl Pertsch et al.
- Establishes: the reported co-training and hierarchical action-generation approach for long-horizon manipulation evaluations.
- Study caution: distinguish the paper's full internal data recipe from the checkpoints and fine-tuning paths actually released in `openpi`.

GR00T N1 (2025)

- Paper
- Official code
- NVIDIA et al.
- Establishes: a dual-system vision-language/action architecture and released tooling for the humanoid manipulation settings described by the project.
- Study caution: check robot support, dataset license, inference hardware, and checkpoint scope at the pinned revision.

SmolVLA (2025)

- Paper
- Official LeRobot code
- Hugging Face et al.
- Establishes: a compact open VLA recipe evaluated with community robot data and asynchronous inference in the stated tasks.
- Study caution: small parameter count is not a timing or safety guarantee on a particular robot computer.

LeRobot

- Official code
- Establishes: an active open ecosystem for robot datasets, models, training, and hardware integrations.
- Study caution: the repository evolves quickly; pin a release/commit and read each policy's learning setting instead of calling all of it RL.

Language-grounded planning and skill composition

Do As I Can, Not As I Say / SayCan (2022)

- Paper
- Michael Ahn et al.
- Establishes: combining language-model skill relevance with learned affordance/value estimates for the reported mobile-manipulation tasks.
- Does not establish: that unconstrained model text should become motor commands; the candidate skill set and local feasibility model are essential.

Code as Policies (2022)

- Paper
- Jacky Liang et al.
- Establishes: generating compositional robot-policy programs from language in the demonstrated tabletop and mobile manipulation settings.
- Study caution: generated code still needs constrained APIs, validation, timeouts, and a trusted execution boundary.

VoxPoser (2023)

- Paper
- Wenlong Huang et al.
- Establishes: composing language-conditioned 3D value maps with a motion planner in the reported manipulation experiments.
- Study caution: the perception and geometric-planning assumptions are part of the result and can fail independently of the language model.

Evaluation and reproducibility

Deep Reinforcement Learning That Matters (2017)

- Paper
- Peter Henderson et al.
- Establishes: empirical evidence that implementation and experimental choices materially affect deep-RL comparisons.

Deep RL at the Edge of the Statistical Precipice / RLiable (2021)

- Paper
- Official code
- Rishabh Agarwal et al.
- Establishes: uncertainty-aware aggregate evaluation methods for the few-run deep-RL regime.

Case-study projects

Microduck RL

- Official training repository
- Microduck runtime repository
- Study use: complete small-biped PPO task definitions, BAM actuator modeling, domain randomization, ONNX export, and sim-to-real runtime contracts.

RSL-RL

- Official code
- Study use: compact robot-focused runners, PPO, actor/critic models, storage, and student-teacher mechanisms.

Isaac Lab

- Official code
- Study use: multi-modal simulator workflows, actuator/sensor models, domain randomization, RL and imitation integrations.

Genesis

- Official code
- Study use: a modern general robotics physics platform and an alternative vectorized simulation stack.
- Availability caution: distinguish released open simulator features from project-roadmap/generative-system descriptions.

Maintaining this index

When adding a source:

1. link the exact paper/version and official repository;
2. summarize the narrow evidence in original words;
3. add one “does not establish” or study-caution boundary;
4. record the review date if a project is fast-moving; and
5. never update a benchmark superlative without rechecking the protocol and newer primary results.